



REPÚBLICA DEL ECUADOR

Escuela Politécnica Nacional

" E S C I E N T I A H O M I N I S S A L U S "

La versión digital de esta tesis está protegida por la Ley de Derechos de Autor del Ecuador.

Los derechos de autor han sido entregados a la "ESCUELA POLITÉCNICA NACIONAL" bajo el libre consentimiento del (los) autor(es).

Al consultar esta tesis deberá acatar con las disposiciones de la Ley y las siguientes condiciones de uso:

- Cualquier uso que haga de estos documentos o imágenes deben ser sólo para efectos de investigación o estudio académico, y usted no puede ponerlos a disposición de otra persona.
- Usted deberá reconocer el derecho del autor a ser identificado y citado como el autor de esta tesis.
- No se podrá obtener ningún beneficio comercial y las obras derivadas tienen que estar bajo los mismos términos de licencia que el trabajo original.

El Libre Acceso a la información, promueve el reconocimiento de la originalidad de las ideas de los demás, respetando las normas de presentación y de citación de autores con el fin de no incurrir en actos ilegítimos de copiar y hacer pasar como propias las creaciones de terceras personas.

Respeto hacia sí mismo y hacia los demás.

ESCUELA POLITÉCNICA NACIONAL

FACULTAD DE INGENIERÍA ELÉCTRICA Y ELECTRÓNICA

RECONSTRUCCIÓN 3D MEDIANTE EL USO DE UN PAR DE CÁMARAS A MODO DE ESTEREOVISIÓN

PROYECTO PREVIO A LA OBTENCIÓN DEL TÍTULO DE INGENIERO EN ELECTRÓNICA Y CONTROL

JOSÉ FELIX BRAVO PIEDRA
josefelixbravo@hotmail.com

BRAULIO ESAUD OTAVALO ALBA
braulio.otavalo@gmail.com

DIRECTOR: Ing. JORGE ANDRÉS ROSALES ACOSTA, PhD.
andres.rosales@epn.edu.ec

CODIRECTOR: Ing. LUIS ALBERTO MORALES ESCOBAR, MSc.
l.morales@udlanet.ec

Quito, mayo del 2014

DECLARACIÓN

Nosotros, José Félix Bravo Piedra y Braulio Esaud Otavalo Alba, declaramos bajo juramento que el trabajo aquí descrito es de nuestra autoría; que no ha sido previamente presentado para ningún grado o calificación profesional; y, que hemos consultado las referencias bibliográficas que se incluyen en este documento.

A través de la presente declaración cedemos nuestros derechos de propiedad intelectual correspondientes a este trabajo, a la Escuela Politécnica Nacional, según lo establecido por la Ley de Propiedad Intelectual, por su Reglamento y por la normatividad institucional vigente.

José Félix Bravo Piedra

Braulio Esaud Otavalo Alba

CERTIFICACIÓN

Certificamos que el presente trabajo fue desarrollado por José Félix Bravo Piedra y Braulio Esaud Otavalo Alba bajo nuestra supervisión.

Ing. Andrés Rosales, PhD

DIRECTOR DEL PROYECTO

Ing. Luis Morales, MSc

CODIRECTOR DEL PROYECTO

AGRADECIMIENTOS

Un agradecimiento a mi familia que han sido un apoyo incondicional a lo largo de mi vida estudiantil.

Un agradecimiento a la Escuela Politécnica Nacional y a todos lo que la conforman por haber transmitido todos los conocimientos profesionales que han hecho posible la realización de este proyecto y que servirán en el transcurso de mi vida profesional.

Un agradecimiento al director, Dr. Andrés Rosales y co-director, MSc. Luis Morales de este proyecto, por su apoyo y comprensión a largo de todos los inconvenientes que se presentaron durante el desarrollo de este proyecto.

Gracias a todos mis amigos, compañeros que han sido apoyo para lograr los objetivos planteados.

José Félix Bravo Piedra

AGRADECIMIENTOS

Agradezco a mis padres por cuidarme y apoyarme siempre, por haberme enseñado a trabajar para conseguir mis objetivos, y agradezco también a cada uno de mis hermanos que siempre han sido ejemplo y apoyo.

Gracias al director y codirector de este proyecto, quienes han sabido guiarnos para la consecución de las metas que nos propusimos en el presente trabajo.

Gracias a mis amigos, que han sabido brindar su apoyo y alegría, con los cuales hemos compartido muchos momentos de nuestras vidas.

Braulio Esaud Otavalo Alba

DEDICATORIA

A mis padres y hermanas

José Félix Bravo Piedra

DEDICATORIA

*A mis padres, quienes con mucho esfuerzo y cariño han educado
a todos sus hijos para hacernos personas de bien, a cada uno de
mis hermanos quienes han confiado en mí.*

Braulio Esaud Otavalo Alba

Contenido

RESUMEN	iv
PRESENTACIÓN	v
CAPÍTULO 1 MARCO TEÓRICO	1
1.1 VISIÓN ARTIFICIAL	1
1.2 SISTEMAS DIGITALES DE VISIÓN	2
1.2.1 IMÁGENES DIGITALES	2
1.2.2 ADQUISICIÓN DIGITAL DE IMÁGENES	2
1.3 MODELOS DE CÁMARAS	4
1.3.1 TIPOS DE CÁMARAS	4
1.3.2 MODELO PINHOLE	5
1.3.3 DISTORSIÓN DE LENTES.....	9
1.3.4 PARÁMETROS INTRÍNSECOS	11
1.3.5 PARÁMETROS EXTRÍNSECOS	13
1.3.6 CALIBRACIÓN	15
1.3.7 LA MATRIZ DE HOMOGRAFÍA	15
1.4 VISIÓN ESTÉREO	16
1.4.1 TRIANGULACIÓN	17
1.4.2 GEOMETRÍA EPIPOLAR	19
1.4.3 CALIBRACIÓN ESTÉREO	20
1.4.4 RECTIFICACIÓN ESTÉREO	20
1.4.5 MÉTODO DE BOUGUET	22
1.4.6 CORRESPONDENCIA ESTÉREO	23
1.4.7 RECONSTRUCCION 3D	26
1.5 SENSORES DE RANGO	27
1.5.1 3D LASER SCANNER	27

1.5.2	CAMARAS 3D	28
1.6	FUNDAMENTOS DEL HMI	29
1.6.1	DEFINICIÓN	29
1.6.2	CARACTERÍSTICAS DE UNA INTERFAZ GRÁFICA	29
1.6.3	LIBRERÍA MFC.....	30
1.6.4	INTEGRACIÓN DE LIBRERIAS	31
CAPÍTULO 2	DESARROLLO DEL SOFTWARE DE VISIÓN ARTIFICIAL	33
2.1	INTRODUCCIÓN	33
2.2	JUSTIFICACIÓN DEL HARDWARE	34
2.3	ADQUISICIÓN DE IMÁGENES	38
2.4	CALIBRACIÓN DE IMÁGENES	40
2.4.1	DETECCIÓN DE ESQUINAS	41
2.4.2	CALIBRACIÓN	43
2.4.3	CALIBRACIÓN ESTEREO	47
2.4.4	VERIFICACIÓN DE RESULTADOS	48
2.5	RECTIFICACIÓN DE IMÁGENES	49
2.6	MAPA DE DISPARIDAD	51
2.7	RECONSTRUCCIÓN 3D	57
CAPÍTULO 3	DISEÑO E IMPLEMENTACIÓN DEL HMI	59
3.1	DISEÑO DEL GUI	59
3.1.1	PRINCIPIOS DE DISEÑO	59
3.1.2	PROCESO DE DISEÑO	60
3.2	IMPLEMENTACIÓN DEL GUI.....	62
3.2.1	VENTANA PRINCIPAL	62
3.2.2	VENTANAS SECUNDARIAS.....	63
3.2.3	FUNCIONAMIENTO DEL GUI.....	64
CAPÍTULO 4	PRUEBAS Y RESULTADOS	69

4.1	PRUEBAS DE CALIDAD DE IMÁGENES	69
4.2	PRUEBAS DE CALIBRACIÓN	70
4.3	PRUEBAS DEL MAPA DE DISPARIDAD	74
4.4	ERRORES DE DISTANCIA DEL SISTEMA ESTÉREO	79
4.5	PRUEBAS DE RECONSTRUCCIÓN 3D	83
4.6	PRUEBAS DE RECONSTRUCCIÓN 3D DESDE 2 VISTAS	85
CAPÍTULO 5 CONCLUSIONES Y RECOMENDACIONES		91
5.1	CONCLUSIONES	91
5.2	RECOMENDACIONES	93
REFERENCIAS BIBLIOGRAFICAS		95

RESUMEN

En el presente proyecto se usó dos cámaras *web* para medir distancias por medio de triangulación de puntos en un par de imágenes, para luego representar las distancias calculadas por medio de una nube de puntos a colores, de manera que se tenga una reconstrucción tridimensional.

Para el cálculo de distancias previamente se realizó una calibración tanto individual como del sistema estéreo, la cual permite obtener los parámetros intrínsecos y extrínsecos de las cámaras. El cálculo de estos parámetros simplifica los cálculos de distancia, y permite usar algoritmos más rápidos para los propósitos del proyecto.

La programación se realizó en C++, dentro del entorno de *Microsoft Visual Studio*. Se utilizó la librería abierta *OpenCV (Open Computer Vision)* para adquirir y procesar imágenes, mientras que para el tratamiento de datos en 3D se utilizó la librería abierta *PCL (Point Cloud Library)*.

Todos los algoritmos, los cálculos de distancia y la representación 3D son controlados por medio de un GUI (*graphical user interface*), desarrollado mediante un proyecto MFC (*Microsoft Foundation Classes*) en *Microsoft Visual Studio*.

Adicionalmente se realizaron experimentos uniendo nubes de puntos tomados desde dos perspectivas distintas, con parámetros de rotación y traslación en 2D conocidos, para poder tener una reconstrucción completa de objetos juntando datos, que con una sola toma el campo de vista del sistema no permite obtener. Los resultados se presentan como una única nube de puntos con mayor número de datos de un objeto, que es tomado como objetivo de reconstrucción. Este experimento sirve como aporte para posibles desarrollos de sistemas SLAM (*Simultaneous Localization And Mapping*) utilizando visión artificial.

PRESENTACIÓN

El presente proyecto ha sido dividido en 5 capítulos, los cuales se detallan inmediatamente.

El Capítulo 1 contiene el marco teórico, se presenta toda la información que se usó como referencia y es necesaria para la comprensión del proyecto.

El Capítulo 2 presenta, todos los procedimientos y algoritmos implementados para cumplir con los objetivos de este proyecto. En este capítulo también se hace mención a todas las consideraciones que se tomó en cuenta, para la obtención de resultados óptimos.

El Capítulo 3 presenta los detalles del GUI implementado, los recursos utilizados para su desarrollo, programación, su lógica de programación y su funcionamiento.

En el Capítulo 4 se presentan las pruebas que se realizaron durante la elaboración de este proyecto, las cuales fueron utilizadas para posteriores correcciones que ayudaron a cumplir los objetivos del proyecto. Adicionalmente, se muestra los distintos experimentos que ayudan a determinar los errores que presenta el sistema, y las futuras aplicaciones en las que representaría un aporte el sistema.

En el Capítulo 5 se presentan las conclusiones y las recomendaciones obtenidas, en el desarrollo del proyecto.

Finalmente incluimos Anexos en los cuales se detallan, el manual de usuario del sistema, los datos utilizados para calcular el error absoluto y relativo para distintas distancias del sistema, y la hoja de datos de las cámaras que se utilizaron en este proyecto.

CAPÍTULO 1 MARCO TEÓRICO

Este capítulo hace mención a los conceptos teóricos relacionados con visión artificial que han servido de base para la realización de este proyecto.

1.1 VISIÓN ARTIFICIAL ^[1]

La visión artificial, también conocida como visión por computador o visión técnica es un campo que mediante la adquisición, procesamiento y análisis de imágenes obtiene información digital del entorno que permite la toma de decisiones.

Las herramientas necesarias para la visión artificial incluyen hardware para adquirir y almacenar imágenes digitales, procesamiento de imágenes, y comunicación al usuario o al proceso automatizado.

La visión por computador es aplicada en campos como la inteligencia artificial, robótica, procesamiento de imágenes, reconocimiento de formas, teoría de control, neurociencia y otros campos.

Las áreas con mayor desarrollo investigativo son:

- Detección de características de imágenes.
- Representación de contornos.
- Segmentación basada en características.
- Análisis de imágenes de rango.
- Representación y modelado de figuras.
- Reconstrucción de figuras.
- Visión estéreo.
- Análisis de movimiento.
- Visión por color.
- Sistemas de auto-calibración.
- Detección de objetos.
- Reconocimiento de objetos 3-D.
- Localización de objetos 3-D.

1.2 SISTEMAS DIGITALES DE VISIÓN ^{[2], [3], [4], [6]}

1.2.1 IMÁGENES DIGITALES

Una imagen digital es un arreglo 2-D matricial de valores, dependiendo de la naturaleza de la imagen este valor puede representar intensidad de iluminación, distancias, o algún otro valor físico.

Las dos tipos de imágenes digitales que se usan con frecuencia en visión artificial son:

- Imágenes de intensidad: Son las imágenes fotográficas comunes adquiridas por cámaras comerciales como *webcams*. Éstas miden la cantidad de luz reflejada por los objetos en la escena, la misma que es captada por elementos sensibles a la luz.
- Imágenes de rango: Cada pixel de una imagen de rango expresa la distancia entre un plano de referencia y cada punto visible en la escena. Se puede representar imágenes de rangos como conjuntos de coordenadas x, y, z lo que permite construir estructuras 3D de una escena, el cual es uno de los objetivos de este proyecto.

En este proyecto se utiliza imágenes digitales de intensidad y se referirá a estas cada vez que se mencione a una imagen a no ser que se indique lo contrario.

1.2.2 ADQUISICIÓN DIGITAL DE IMÁGENES

La visión empieza por medio de una fuente de energía, cuando la luz emitida por esta fuente de energía al desplazarse a través del espacio es reflejada por el entorno, y es capturada por medio de la cámara.

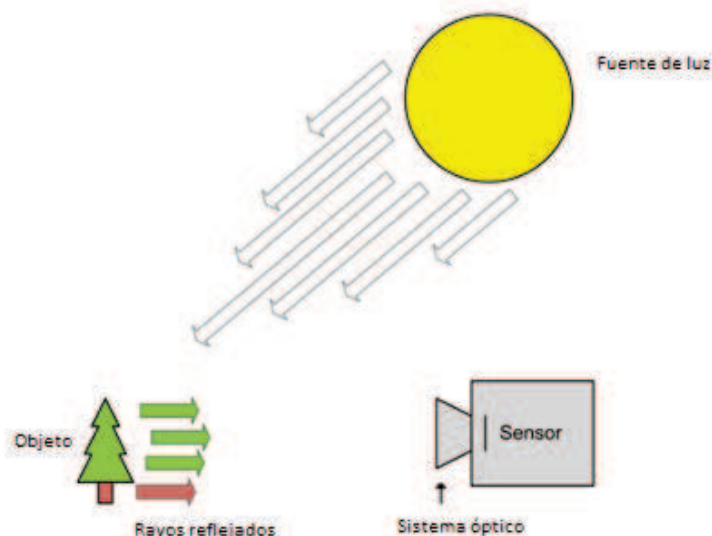


Figura 1.1: Esquema del proceso de adquisición de imágenes. [3]

La luz reflejada es capturada por medio de un sensor dentro de la cámara. Este sensor consiste en un arreglo matricial 2D, donde cada una de las celdas se la denomina pixel, y es capaz de cuantificar la cantidad de luz reflejada en la misma y transformarla en una medida de voltaje.

Cada uno de estos valores de voltaje es transformado por medio de un conversor análogo-digital en un valor digital en bits, para obtener finalmente una imagen en donde cada pixel representa un valor digital de la cantidad de luz adquirida en ese punto.

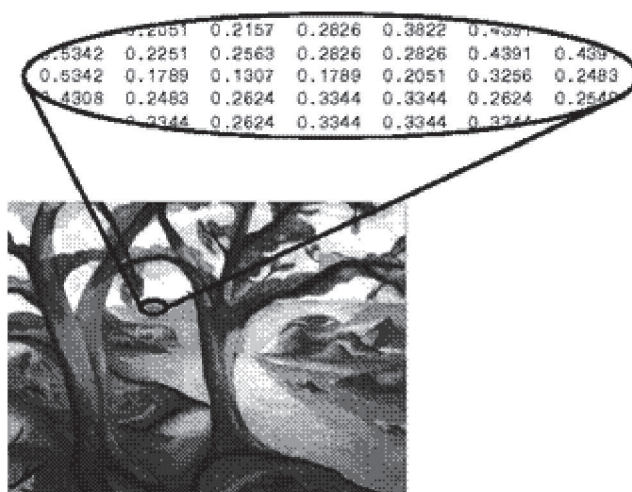


Figura 1.2: Representación de los valores en cada pixel en una imagen digital a escala de grises. [5]

Las imágenes de intensidad descritas anteriormente son imágenes a escala de grises, para obtener imágenes a colores se debe usar tres sensores que relacionen las longitudes de onda de los colores rojo, verde y azul. Con las combinaciones de estos tres colores se puede obtener casi cualquier color en una imagen.

Las imágenes digitales tienen el sistema de coordenadas como se muestra en la Figura 1.3, en este sistema de coordenadas la esquina superior izquierda es el centro de coordenadas y la dirección de los ejes son de la forma indicada.

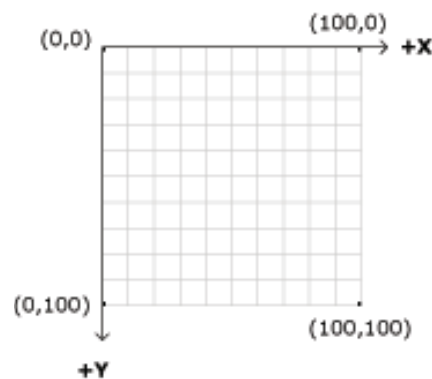


Figura 1.3: Sistema de coordenadas cartesianas de una imagen digital. [6]

La imagen es representada como una función $f(x, y)$, donde el valor de la función es el valor de intensidad de luz en valores digitales, contenido en la ubicación del pixel correspondiente. Para el caso de las imágenes a color la función $f(x, y)$ devuelve tres valores, correspondientes a los valores de los colores rojo, azul y verde.

1.3 MODELOS DE CÁMARAS ^{[1], [7]}

1.3.1 TIPOS DE CÁMARAS ^[1]

Un punto simple de la escena refleja la luz en varias direcciones, donde todos estos rayos reflejados entran en la cámara. Para obtener imágenes nítidas todos los rayos reflejados de un mismo punto deben converger en un mismo punto en el plano de proyección de la cámara, cuando esto sucede se puede decir que la

cámara está enfocada. Para obtener rayos enfocados de una escena se tiene dos posibilidades:

- Reducir el punto de apertura de la cámara llamado “*pinhole*”, de aquí se deduce el modelo que lleva el mismo nombre.
- Introducir un sistema óptico compuesto por lentes, aperturas y otros elementos.

1.3.2 MODELO PINHOLE [7], [8], [10]

Este modelo es una aproximación usada frecuentemente en la visión por computador como un modelo ideal.

El modelo *pinhole* describe la relación matemática que existe entre un punto 3D y su proyección en el plano de imagen, de una manera ideal sin usar ningún tipo de lente. En el campo de la visión artificial esta relación se la conoce como la transformación proyectiva.

En este modelo la apertura de la cámara se reduce a un punto infinitesimal, con lo que se consigue que solo un rayo de luz entre en la cámara, y cree una correspondencia uno a uno entre los puntos visibles, los rayos y el plano de proyección de la cámara. Esto da como resultado imágenes nítidas, sin distorsión de objetos a diferentes distancias de la cámara.

Definimos el centro óptico al punto en donde todos los rayos intersecan, y la línea perpendicular al plano de proyección que pasa a través del centro óptico como el eje óptico. Adicionalmente se define a la intersección entre el plano de la imagen y el eje óptico como el punto principal y la distancia entre el *pinhole* y el centro óptico a lo largo del eje óptico como la distancia focal.

Considerando que la imagen esta siempre enfocada, el tamaño de la imagen en relación con el objeto distante está dado por un único parámetro de la cámara, su longitud focal f .

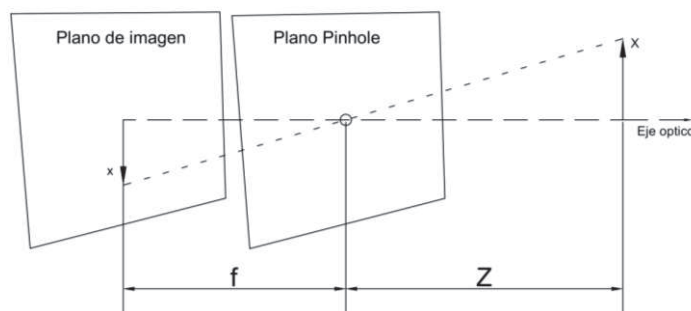


Figura 1.4: Modelo *Pinhole*. [9]

En la Figura 1.4, Z es la distancia desde el centro óptico al objeto a lo largo del eje óptico, " X " es la longitud respecto al eje óptico, y " x " es la distancia del objeto en el plano de la imagen con respecto del eje óptico, con esto se puede deducir: [9]

$$-x = f \left(\frac{X}{Z} \right) \quad (1.1)$$

Replantando el modelo anterior, a uno equivalente más conveniente en el que se intercambia el *pinhole* y el plano de la imagen como se muestra en la Figura 1.5 se tiene un modelo con las siguientes características:

- El objeto ya no aparece invertido en el plano de proyección.
- El *pinhole* se reinterpreta como centro de proyección
- Cada rayo reflejado termina en el centro de proyección.
- El punto principal se reinterpreta como el centro de coordenadas.

Con este nuevo modelo se puede imaginar un sistema de coordenadas, cuyo origen está en el centro de proyección y cuyo eje Z esta a lo largo del eje óptico, este sistema de coordenadas es llamado el sistema estándar de coordenadas de la cámara.

La intersección del rayo reflejado por un punto M con coordenadas X, Y, Z en el sistema estándar de coordenadas es el punto m con coordenadas x, y en el plano de la imagen. Las coordenadas del punto m están con respecto al punto principal como centro de coordenadas y los ejes x, y paralelos al los ejes X, Y, Z del sistema estándar de coordenadas.

En este nuevo modelo la imagen del objeto distante es exactamente del mismo tamaño como lo fue en el modelo anterior. La nueva fórmula es similar a la fórmula 1.1 excepto por el signo negativo. De manera analógica al eje x se puede deducir la coordenada para el eje y .

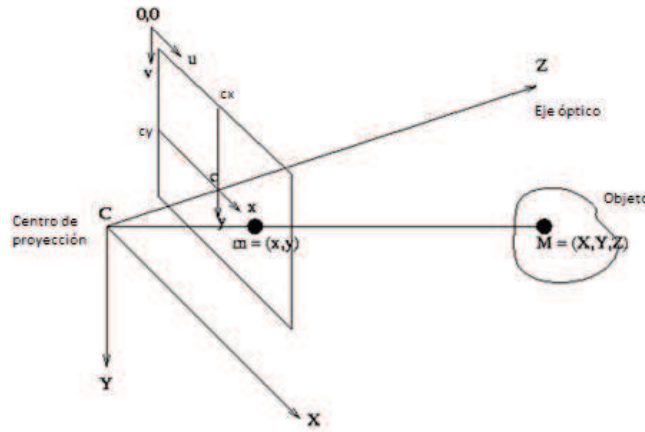


Figura 1.5: Modelo *Pinhole* modificado. [10]

$$x = f * \frac{X}{Z} \quad (1.2)$$

La coordenada en píxeles como se definió en 1.2.2 es con respecto al origen de coordenadas en la esquina superior derecha. Para pasar del sistema de coordenadas con el origen en el punto principal al sistema de coordenadas en píxeles con unidades en píxeles se usa las ecuaciones (1.3) y (1.4) respectivamente para los ejes x y y . [10]

$$x_{PIXELES} = \frac{x}{\text{Ancho del pixel}} + c_x \quad (1.3)$$

$$y_{PIXELES} = \frac{y}{\text{Largo del pixel}} + c_y \quad (1.4)$$

Los valores c_x y c_y representan los *offset* desde el centro de coordenadas en píxeles al punto principal, mientras el ancho y largo en píxeles se asigna en píxeles/mm.

Reemplazando la ecuación (1.2) en la ecuación (1.3) se tiene la relación entre el sistema de referencia estándar y el sistema de coordenadas en pixeles de una imagen digital. [9]

$$x_{PIXELES} = \frac{f}{\text{Ancho del pixel}} * \frac{X}{Z} + c_x = f_x \frac{X}{Z} + c_x \quad (1.5)$$

$$y_{PIXELES} = \frac{y}{\text{Largo del pixel}} * \frac{Y}{Z} + c_y = f_y \frac{Y}{Z} + c_y \quad (1.6)$$

Por simplicidad se asigna f_x y f_y en valores en pixeles, con esto se obtiene una distancia focal para cada eje. La razón para tomar esta simplificación es que por el proceso de calibración se puede determinar solamente f_x y f_y , mas no la distancia focal ni el ancho o largo en pixeles.

La desventaja del sistema *pinhole* es el tiempo de exposición, debido a que este modelo permite un paso de luz muy reducido por unidad de tiempo, los elementos fotosensibles necesitan un tiempo muy prolongado como para captar el mínimo de luz requerido para obtener una imagen legible.

La transformada proyectiva muestra que un punto " p ", proyección sobre un plano de imagen de un punto del mundo físico 3D " P " no proporciona información sobre la distancia a la que se encuentra el punto P con respecto al sistema de referencia de la cámara, ya que al ser una proyección en realidad podría estar ubicado en cualquier parte a lo largo de toda la línea de puntos del rayo reflejado.

Esta desventaja lo hace al sistema *pinhole* no utilizable en la práctica, en realidad se usan sistemas ópticos más complejos con lentes que enfocan una mayor cantidad de luz de cada punto, lo que permite adquirir imágenes más rápidamente pero al costo de tener distorsión en las mismas imágenes.

1.3.3 DISTORSIÓN DE LENTES ^{[9], [11]}

El uso de lentes ópticos hace que los rayos reflejados provenientes del mismo punto 3D tengan convergencia en el mismo punto en el plano de la imagen, de manera que se obtiene una imagen enfocada en corto tiempo de exposición a costa de introducir distorsión en la imagen capturada.

Ya que las cámaras comerciales utilizadas usan lentes se debe introducir el efecto de la distorsión de los mismos al modelo *pinhole* para obtener un modelo real práctico.

En teoría, es posible definir una lente que no introduzca distorsiones, pero en la práctica no hay ningún lente perfecto. Esto es principalmente por razones de fabricación, es mucho más fácil hacer una lente "esférica" que hacer una lente "parabólica" (matemáticamente ideal). A esto hay que sumar el hecho de que es difícil alinear exactamente el lector de imagen con el lente de la cámara.

Se tiene principalmente las distorsiones radiales y tangenciales, que son las usadas para este trabajo. Aunque existen otros tipos de distorsiones estas tienen muy poca incidencia y pueden ser despreciadas.

Para describir la distorsión de lentes se establece una relación entre las coordenadas físicas en píxeles de la imagen y de las coordenadas de los mismos píxeles en una proyección ideal.

1.3.3.1 Distorsión radial

Las lentes de cámaras reales a menudo distorsionan la ubicación de píxeles cerca de los bordes de las imágenes, fenómeno fuente del efecto "ojo de pez". Los rayos más lejos del centro de la lente curvan más que los más cercanos, en un lente de bajo costo es típica una distorsión más pronunciada a medida que se aleja del centro.

Los efectos de la distorsión radial se puede distinguir entre la distorsión de barril y de cojín:

- La distorsión de barril crea un efecto que la imagen ha sido proyectada sobre una esfera o barril.
- La distorsión de cojín crea el efecto visible de que las líneas que no pasan por el centro de la imagen se arquean hacia adentro, hacia el centro de la imagen, como un cojín.

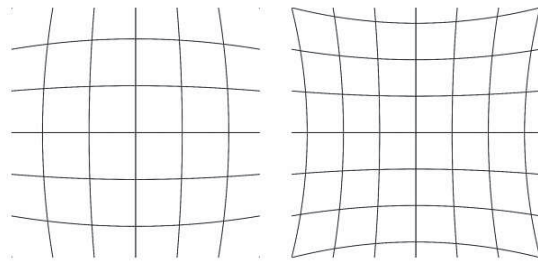


Figura 1.6: Efectos de distorsión, izquierda: efecto de barril, derecha: efecto de cojín. [11]

Al describir la distorsión de lentes se utiliza un sistema de coordenadas polares con el centro del mismo en el punto principal. Claramente los lentes son elementos simétricos axiales, por lo que los efectos de distorsión serán rotacionales simétricos.

Para distorsiones radiales, la distorsión es 0 en el centro (óptico) de la cámara y aumenta a medida que avanzamos hacia la periferia. En la práctica, esta distorsión es pequeña y se puede caracterizar por los primeros términos del desarrollo en serie de Taylor alrededor de $r = 0$. [9]

$$x_{\text{corregido}} = x(1 + k_1 r^2 + k_2 r^4 + k_3 r^6) \quad (1.7)$$

$$y_{\text{corregido}} = y(1 + k_1 r^2 + k_2 r^4 + k_3 r^6) \quad (1.8)$$

$$\text{donde } r^2 = x^2 + y^2$$

Aquí x e y son las coordenadas de los puntos distorsionados, y $x_{corregido}$ e $y_{corregido}$ las coordenadas corregidas, y r es la distancia radial.

1.3.3.2 Distorsión tangencial

Aunque los efectos de la distorsión radial son muchos mayores a los de la tangencial, y algunos modelos desprecian los efectos de la distorsión tangencial para el caso de este trabajo estos son considerados.

Esta distorsión es debida a defectos de fabricación que resultan al lente no estar exactamente paralela al plano de la imagen. Distorsión tangencial se caracteriza mínimamente por dos parámetros adicionales p_1 y p_2 : [9]

$$x_{corregido} = x + [2p_1y + p_2(r^2 + 2x^2)] \quad (1.9)$$

$$y_{corregido} = y + [p_1(r^2 + 2y^2) + 2p_2x] \quad (1.10)$$

1.3.4 PARÁMETROS INTRÍNSECOS ^{[9], [12], [13]}

Los parámetros intrínsecos pueden ser definidos como el conjunto de parámetros necesarios para definir las características ópticas, geométricas y digitales de la cámara. Estos parámetros son:

- f_x : Longitud focal en pixeles en dirección x
- f_y : Longitud focal en pixeles en dirección y
- C_x : La coordenada del punto principal en dirección x
- C_y : La coordenada del punto principal en dirección y
- γ : Coeficiente de asimetría que define el ángulo entre los ejes en pixeles x e y
- Los coeficientes de distorsión radial y tangencial

1.3.4.1 Matriz intrínseca

Un punto 2D en una imagen con coordenadas (x, y) puede ser representado como un vector en 3D $X = (x_1, x_2, x_3)$, donde $x = \frac{x_1}{x_3}$ y $y = \frac{x_2}{x_3}$ para valores de x_3 diferentes de cero. A esta relación se la conoce como la representación homogénea de un punto que se encuentra en el plano de imagen.

La transformada homogénea es una proyección inversa de las líneas formadas por los rayos de luz reflejados, que pasan por el centro de proyección y los puntos en el plano de la imagen.

Cabe notar que en la transformada homogénea los valores del vector proporcionales representan el mismo punto. Esto se justifica con el hecho de que todos los puntos de un rayo que pasa a través del centro de proyección representan el mismo punto en el plano de la imagen, por lo que todos los puntos a largo del rayo son iguales.

El uso de la transformada homogénea nos permite relacionar las coordenadas del mundo 3D, con el origen de coordenadas en el centro de proyección de la cámara con las coordenadas de los mismos en píxeles de manera sencilla por medio 1.11.

[9]

$$q = MQ \quad (1.11)$$

$$\text{donde } q = \begin{bmatrix} x \\ y \\ z \end{bmatrix}, \quad Q = \begin{bmatrix} X \\ Y \\ Z \end{bmatrix}$$

La matriz M organiza los parámetros intrínsecos que definen a la cámara en una sola matriz de 3x3, llamada la matriz intrínseca de la cámara: [9]

$$M = \begin{bmatrix} f_x & \gamma & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \quad (1.12)$$

Al usar coordenadas homogéneas se debe dividir x e y para w (o Z), a manera de obtener las coordenadas en pixeles.

1.3.5 PARÁMETROS EXTRÍNSECOS ^{[9], [12], [13]}

La ubicación del plano de la cámara es muchas veces desconocido, y un problema común es determinar la posición y orientación del plano de la cámara con respecto a una referencia conocida, usando solo información en la imagen.

Los parámetros extrínsecos se definen como el conjunto de parámetros geométricos, que identifican una transformada única entre el sistema coordenadas de la cámara y una referencia conocida en el mundo 3D.

Una manera de describir esta transformada es usar un vector de traslación, que describe la posición relativa de los orígenes, y una matriz de rotación, que da la correspondencia entre los ejes de ambas referencias.

1.3.5.1 Vector de traslación

El vector de traslación es la forma en que se representa un cambio del origen de coordenadas de un sistema a otro. En otras palabras, el vector de traslación es el *offset* entre los dos sistemas de coordenadas, por lo tanto para pasar del sistema de coordenadas del mundo físico 3D al sistema de la cámara el vector de traslación apropiado, es simplemente la coordenada al origen del sistema 3D en el sistema de la cámara. [9]

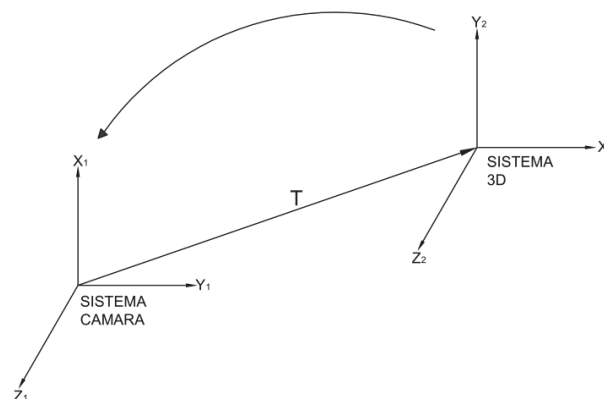


Figura 1.7: Traslación de sistema de coordenadas.

$$T = \begin{bmatrix} T_X \\ T_Y \\ T_Z \end{bmatrix} \quad (1.13)$$

1.3.5.2 Matriz de rotación

En el caso de la rotación de un sistema de coordenada se tiene dos posibilidades, la rotación de los ejes y la rotación del vector. Para el caso de la rotación del sistema de coordenadas del mundo 3D al sistema de la cámara se realiza una rotación de ejes.

La rotación en tres dimensiones se puede descomponer a rotaciones en torno a cada eje donde el eje de pivote se mantiene constante. Si hacemos girar alrededor de los ejes x, y y z en secuencia con la respectiva rotación de ángulos, Ψ, φ y θ con el sentido anti-horario como positivo, el resultado es una matriz de rotación R total que es dada por el producto de las tres matrices $R_x(\Psi), R_y(\theta)$ y $R_z(\varphi)$: [13]

$$R = R_x * R_y * R_z \quad (1.14)$$

$$\text{donde, } R_x(\Psi) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos(\Psi) & \text{sen}(\Psi) \\ 0 & -\text{sen}(\Psi) & \cos(\Psi) \end{bmatrix},$$

$$R_y(\theta) = \begin{bmatrix} \cos(\theta) & 0 & -\text{sen}(\theta) \\ 0 & 1 & 0 \\ \text{sen}(\theta) & 0 & \cos(\theta) \end{bmatrix}$$

$$R_z(\varphi) = \begin{bmatrix} \cos(\varphi) & \text{sen}(\varphi) & 0 \\ -\text{sen}(\varphi) & \cos(\varphi) & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

La matriz de rotación R tiene la propiedad de que su inversa es su transpuesta, por lo que se tiene: [13]

$$R^T * R = R * R^T = I \quad (1.15)$$

Se suman los efectos de rotación y traslación para expresar un punto de coordenadas de un sistema de coordenadas a otro de referencia. Para un punto con coordenadas P_o en el sistema de coordenadas en el mundo 3D, se tiene la

coordenada P_c en el sistema de coordenadas de la cámara según la ecuación 1.16. [9]

$$P_o = R(P_c) + T \quad (1.16)$$

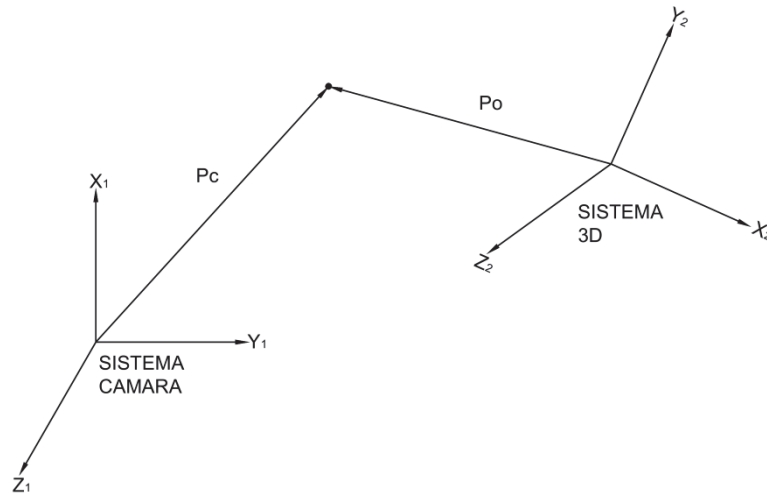


Figura 1.8: Vectores P_o y P_c , representación del mismo punto desde dos sistemas de referencia diferentes.

1.3.6 CALIBRACIÓN ^[14]

El proceso de calibración es la obtención de los parámetros intrínsecos y extrínsecos de la cámara, por medio de un conjunto de imágenes adquiridas. La calibración permite tener los parámetros que nos indican como un objeto 3D, se proyecta en el plano de la cámara para así obtener medidas métricas de las imágenes.

1.3.7 LA MATRIZ DE HOMOGRAFÍA ^[9]

Usando coordenadas homogéneas para expresar tanto un punto Q en el mundo 3D como el punto proyectado en el plano de la cámara q , es posible expresar como una multiplicación de matrices la relación entre ambos. Para expresar esta relación matricial es necesario tener tanto las características intrínsecas y extrínsecas organizadas en forma matricial.

Las características intrínsecas se encuentran reunidas en la matriz intrínseca M definida en 1.3.4.1. Para el caso de los parámetros extrínsecos definimos la

matriz extrínseca W , la misma que expresa la transformación física entre el sistema de referencia de la cámara y la del sistema de referencia del mundo 3D. Esta matriz extrínseca contiene la suma de efectos de la matriz de rotación R y traslación T : [9]

$$W = (R|T) \quad (1.17)$$

Con esto se define la ecuación que relaciona el punto Q y q , ambos en coordenadas homogéneas. [9]

$$s * \tilde{q} = M * W * \tilde{Q} \quad (1.18)$$

$$\text{donde } \tilde{q} = [x \ y \ 1]^T, \tilde{Q} = [X \ Y \ Z \ 1]^T$$

s es un factor de escala arbitrario que muestra que la homografía es definida bajo un factor de escala.

Una homografía plana es la correspondencia proyectiva de un plano u otro. Una homografía plana es la correspondencia de puntos de superficies planas en 2-D (puntos con igual coordenada z) en el mundo 3D, al plano de proyección de la cámara.

Sin pérdida de generalidad, cuando $Z = 0$: [9]

$$s * \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = M * \begin{bmatrix} r_1 & r_2 & t \end{bmatrix} \begin{bmatrix} X \\ Y \\ 1 \end{bmatrix} \quad (1.19)$$

Este punto podemos definir la que se conoce como la matriz de homografía H : [9]

$$H = M * \begin{bmatrix} r_1 & r_2 & t \end{bmatrix} \quad (1.20)$$

1.4 VISIÓN ESTÉREO ^{[1], [9]}

La visión estéreo se refiere a la habilidad de tener información 3D de una estructura, y de distancia de una escena a partir de dos a más imágenes tomadas

por diferentes vistas, es el mismo principio usado por el cerebro para realizar reconstrucciones 3D de lo que ve.

Los 4 pasos para realizar una reconstrucción 3D son:

1. Eliminar la distorsión radial y tangencial de lentes de las imágenes adquiridas.
2. Alinear las imágenes derecha e izquierda, este proceso es llamado rectificación.
3. Hallar los mismos puntos característicos de una imagen en la otra, este proceso es llamado correspondencia, el resultado de esto es el mapa de disparidad que contiene la disparidad de cada punto característico.
4. Con la geometría de las cámaras, se transforma los valores de disparidad en distancias por medio de triangulación, el resultado de este procedimiento es el mapa de profundidad.

1.4.1 TRIANGULACIÓN ^{[9] [15]}

El objetivo de la triangulación es calcular la distancia de puntos en una escena, basándose en la posición de un par de cámaras separadas la una de la otra por una distancia conocida.

Inicialmente se considera un modelo simple de dos cámaras idénticas, las cuales se encuentran separadas por una distancia en la dirección del eje x , con planos de proyección coplanares (con ejes ópticos paralelos), distancias focales iguales las cuales adquieren imágenes sin distorsión.

En este modelo simple las filas de píxeles están alineadas, esto quiere decir que cada fila de píxeles de una cámara se alinea exactamente con la fila correspondiente en la otra cámara.

Un punto característico en el mundo físico real 3D que se puede ver en ambos planos de proyección, es visto en diferentes posiciones por cada una de las cámaras. Esta diferencia de posiciones horizontales entre los planos de

proyección es llamada disparidad, el mismo que nos da la información sobre la distancia a la que se encuentra el punto desde la cámara.

Como referencia el origen de coordenadas al centro de proyección de la cámara izquierda se puede deducir geoméricamente, por medio de la imagen 1.9 la ecuación 1.21. [9]

$$\frac{T - (x_I - x_D)}{Z - f} = \frac{T}{Z} \quad (1.21)$$

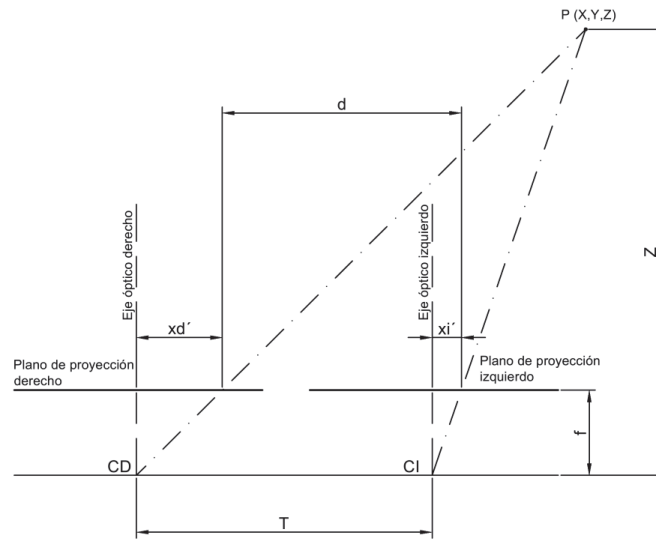


Figura 1.9: Proceso de triangulación. [15]

En la Figura 1.9 x_I y x_D representan las coordenadas de los puntos proyectados p_l y p_d con respecto al sistema de referencia correspondientes.

Por despeje se puede calcular la profundidad Z . [9]

$$Z = f \frac{T}{x_I - x_D} \quad (1.22)$$

Conociendo la distancia focal y la distancia entre las cámaras, se puede calcular la distancia desde el centro de coordenadas de la cámara izquierda, por medio del cálculo del valor de disparidad. Para esto se debe encontrar el mismo punto en ambas imágenes, problema conocido como correspondencia.

En la práctica no se tiene estas condiciones ideales, lo que dificulta el proceso de búsqueda de correspondencia. Obtener estas condiciones ideales por software permite encontrar los puntos correspondientes entre ambas imágenes de una manera simplificada, ya que en caso de no tener este modelo la búsqueda de correspondencia se expande a toda la imagen.

1.4.2 GEOMETRÍA EPIPOLAR ^{[16], [9]}

Geometría epipolar, se refiere a la geometría de un sistema estéreo de imágenes con 2 cámaras *pinhole*, y unos nuevos puntos llamados epipolos. En esta sección se considera un sistema estéreo donde los planos de imágenes no son coplanares, y representan una situación de un sistema estéreo real.

Los nuevos puntos de interés son los epipolos. El epipolo e_l en el plano de la imagen Π_l se define como la proyección del centro de la proyección de la otra cámara C_D . Para el epipolo e_r se cumple el mismo principio sobre el plano Π_r y el centro de proyección C_l .

El plano formado por los dos epipolos, e_l y e_r y el punto físico P se lo conoce como el plano epipolar, y las líneas $p_l - e_l$ y $p_r - e_r$ se llaman las líneas epipolares. Véase Figura 1.10.

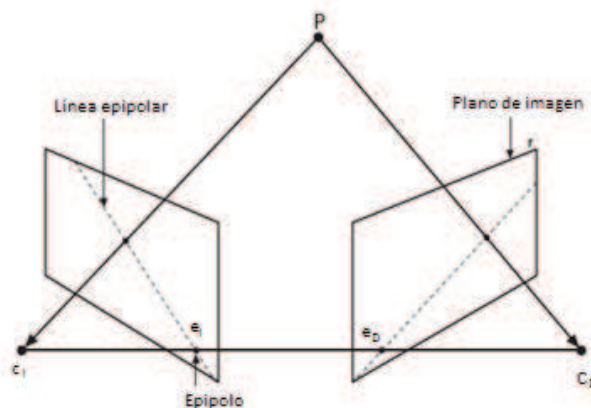


Figura 1.10: Geometría epipolar. [9]

Al tomar por ejemplo un punto físico con el sistema de referencia en el centro de proyección de la cámara derecha p_D , es útil proyectar la línea $C_D - p_D$ sobre el plano de proyección izquierdo Π_l , al hacer esto se puede notar que esta proyección es en realidad la línea epipolar e_l . Esto hace notorio que todas las

posibles ubicaciones de un punto de vista en el plano de proyección, es la línea que pasa por el punto correspondiente y el punto epipolar por el otro plano de proyección.

Las conclusiones a las que se pueden llegar con la geometría epipolar son las siguientes:

- Cada punto 3D P , visto por ambas cámaras está contenido en un plano epipolar que interseca a cada imagen en la línea epipolar.
- Dado un punto característico en una imagen, el mismo punto en la otra imagen tiene que estar en la línea epipolar correspondiente. Esto se conoce como la restricción epipolar.
- La restricción epipolar significa que la búsqueda de correspondencia se reduce a una búsqueda unidimensional, a lo largo de las líneas epipolares correspondientes.
- El orden se conserva. Si los puntos A y B son visibles en ambas imágenes, y se producen horizontalmente en ese orden en un plano de imágenes, se presentan horizontalmente en igual orden en el otro plano de imagen.

1.4.3 CALIBRACIÓN ESTÉREO

Es el proceso de cálculo de la relación geométrica entre las dos cámaras en el espacio. La calibración estéreo consiste en encontrar la matriz de rotación R y el vector de traslación T entre los sistemas de coordenadas de ambas cámaras. Con las matrices R y T se realiza el proceso de rectificación a explicarse a continuación.

1.4.4 RECTIFICACIÓN ESTÉREO ^{[18], [19]}

El modelo estéreo planteado en 1.4.1 es ideal, en la realidad no se obtiene un sistema con tales características. Tener un sistema estéreo donde los dos planos

de proyección estén perfectamente alineados no es posible, he aquí la razón de realizar el proceso de rectificación para obtener un sistema más simplificado.

El objetivo de la rectificación estéreo, es proyectar los planos de imagen de ambas cámaras, de manera que residan en el mismo plano con las filas de las imágenes perfectamente alineadas en una configuración paralela frontal, tal que la correspondencia estéreo es más fiable y computacionalmente tratable.

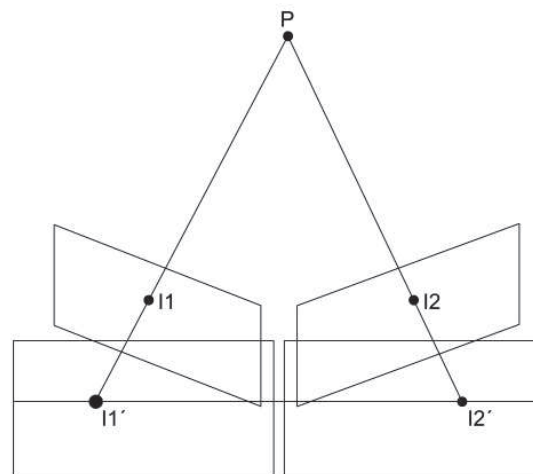


Figura 1.11: Planos antes y después de la rectificación. [17]

El resultado de la alineación de las filas horizontales dentro de un plano que contiene a ambos planos de imagen, es que los propios epipolos se encuentran en el infinito, y las líneas epipolares se disponen horizontalmente a lo largo de las filas de las imágenes ubicadas en iguales posiciones en ambas imágenes, ver Figura 1.11.

Luego de realizar el proceso de rectificación, la búsqueda de puntos correspondientes entre las imágenes derechas e izquierda, para el cálculo del valor de disparidad, se reduce a la búsqueda a lo largo de la misma coordenada y en la imagen correspondiente, con esto se simula el sistema ideal planteado en 1.4.1.

Los algoritmos de rectificación se pueden dividir en métodos que necesitan una calibración estéreo previa, y los que no la necesitan.

Los métodos que no necesitan calibración previa causan mayor distorsión en el par de imágenes rectificadas, pero con la ventaja de velocidades de procesamiento más altas y son ideales para cálculos de distancia por medio de una sola cámara en movimiento.

Los métodos que utilizan una calibración previa son la mejor opción cuando se tiene un par estéreo calibrado, y son los que se usa con preferencia. El método más común para estos sistemas es el de Bouguet, el mismo que es el usado en este proyecto.

1.4.5 MÉTODO DE BOUGUET ^[20]

Este método realiza una re-proyección de ambos planos de la imagen, de manera que se minimiza los efectos de distorsión debidos al proceso de rectificación, al tiempo que se maximiza el área de visualización permitiendo tener valores de distancia en escala reales.

Para minimizar la distorsión de re-proyección, cada cámara gira la mitad de la rotación, para esto la matriz R se divide por la mitad entre las dos cámaras, llamamos a las dos matrices de rotación resultantes r_l y r_r para la cámara izquierda y derecha respectivamente.

Las matrices r_l y r_r cumplen la siguiente ecuación: [20]

$$R = (r_l)^T r_r \quad (1.23)$$

Tal rotación pone las cámaras en la alineación coplanar, pero no en la alineación de filas, la matriz que logra esto es R_{rect} , la cual establece el punto epipolar de la cámara izquierda en el infinito y alinea las líneas epipolares horizontalmente. La matriz R_{rect} se define como: [20]

$$R_{\text{rect}} = \begin{bmatrix} (e_1)^T \\ (e_2)^T \\ (e_3)^T \end{bmatrix} \quad (1.24)$$

R_{rect} se define de manera que, el nuevo eje x este en dirección del vector traslación T entre los puntos principales tomando como centro de coordenadas al punto principal de la cámara izquierda, según esto e_1 se define acorde a la ecuación 1.25. [20]

$$e_1 = \frac{T}{||T||} \quad (1.25)$$

e_2 se define de manera que el nuevo eje y sea ortogonal al eje nuevo eje x y al antiguo eje z , el cual se encuentra a lo largo del eje óptico. Para esto se aplica el producto cruz entre e_1 y la dirección del eje óptico. [20]

$$e_2 = \frac{[-T_y \quad T_x \quad 0]^T}{\sqrt{T_x^2 + T_y^2}} \quad (1.26)$$

e_3 se define de manera que el nuevo eje z es ortogonal a los ejes x e y , por lo que e_3 es el producto cruz de e_1 y e_2 . [20]

$$e_3 = e_1 \times e_2 \quad (1.27)$$

La rectificación se obtiene mediante el efecto combinando de ambas matrices, por lo que las matrices de rotación para cada imagen son: [20]

$$R_l = R_{rect} * r_l \quad (1.28)$$

$$R_r = R_{rect} * r_r \quad (1.29)$$

1.4.6 CORRESPONDENCIA ESTÉREO ^[21]

A partir de las imágenes rectificadas izquierda y derecha, se toma la posición de un pixel en la imagen izquierda y se calcula la locación del pixel correspondiente en la imagen derecha.

Dado que en las imágenes rectificadas cada fila es una línea epipolar, la ubicación coincidente del pixel en la imagen derecha, debe ser a lo largo de la

misma fila (la misma coordenada y) del punto buscado en la imagen de la izquierda.

El resultado de realizar esto es el mapa de disparidad, en el cual constan los valores de disparidad en escala de grises de cada pixel, donde los puntos más claros representan distancias cercanas a la cámara, y medida que se oscurecen representan distancias más alejadas de la cámara.

1.4.6.1 Métodos de correspondencia ^[21] ^[40]

Los métodos de correspondencia estéreo se pueden clasificar como locales o globales. Los métodos locales intentan igualar pequeñas regiones de una imagen a otra en base a las características intrínsecas de la región. Mientras los métodos globales complementan los métodos locales, considerando las limitaciones físicas, como continuidad de la superficie.

Los métodos locales pueden clasificarse adicionalmente, por si coinciden con características discretas entre imágenes, o por si se correlacionan por un patrón dentro de un área pequeña.

En los métodos por coincidencia discreta la característica usualmente elegida es la luz, por ejemplo las esquinas son una característica natural a usar, porque se mantienen como esquinas en todas las proyecciones.

Algoritmos basados en características compensan los cambios de punto de vista y diferencias de cámara, y pueden producir una rápida y robusta correspondencia, pero tienen la desventaja de requerir un algoritmo potente de extracción de características.

Dado que los métodos de área no necesitan calcular características, y tienen una estructura algorítmica extremadamente regular, se puede tener implementaciones optimizadas de la misma. Este método es el implementado en este trabajo usando

pequeñas ventanas SAD (*Sum of Absolute Difference*), este algoritmo encuentra puntos sólo muy coincidentes (de textura) entre las dos imágenes.

El método de las ventanas SAD consiste en que para encontrar el mismo punto tanto en la imagen izquierda como derecha, considerando una ventana alrededor del mismo preestablecida, se calcula las diferencias absolutas entre los valores de cada pixel alrededor del punto objetivo, dentro de la ventana y de los valores de cada pixel alrededor una ventana móvil, a lo largo de la misma línea epipolar en la imagen destino.

Luego se suman todas las diferencias obtenidas, para cada una de las ventanas móviles a lo largo la línea epipolar en la imagen derecha, y la ventana que da como resultado el mínimo valor corresponde al pixel buscado, en caso de tener éxito al obtener un valor que cumplan con el mínimo preestablecido.

Para entender este proceso de mejor manera, consideremos una imagen origen de 3x3 y una imagen destino de 3x5, el objetivo es encontrar cual es el punto más correspondiente al punto en I_o con coordenadas (2,2) en la imagen destino:

$$I_o = \begin{bmatrix} 5 & 9 & 8 \\ 7 & 9 & 8 \\ 8 & 5 & 9 \end{bmatrix} \quad I_{de} = \begin{bmatrix} 9 & 5 & 7 & 8 & 2 \\ 7 & 6 & 1 & 3 & 8 \\ 8 & 9 & 3 & 6 & 1 \end{bmatrix}$$

Las restas de los valores de los pixeles considerando la parte derecha, central e izquierda de la imagen destino dan como resultado:

$$I_d = \begin{bmatrix} 4 & 4 & 1 \\ 0 & 3 & 7 \\ 0 & 4 & 6 \end{bmatrix} \quad I_c = \begin{bmatrix} 0 & 2 & 0 \\ 1 & 8 & 5 \\ 1 & 2 & 3 \end{bmatrix} \quad I_i = \begin{bmatrix} 2 & 1 & 6 \\ 6 & 6 & 0 \\ 5 & 1 & 8 \end{bmatrix}$$

Sumando todos los términos de cada matriz se obtiene los 3 valores SAD 29, 22, 35 correspondientemente a cada matriz, por lo que la porción de la imagen destino más próxima a la imagen origen es la de menor valor SAD, para este caso la porción del centro.

Como se puede ver la probabilidad de encontrar el punto correspondiente deseado, depende del tamaño de la ventana utilizado y del mínimo valor que se acepta para considerar que un punto es el deseado, estos valores deben ser elegidos considerando que el tiempo de procesamiento no sea demasiado alto, de modo que se obtenga un balance entre tiempo y rendimiento.

Aunque existen más métodos de área para los cálculos de correspondientes, como ZSAD (*Zero-mean SAD*), LSAD (*Locally scaled SAD*), SSD (*Sum of Squared Differences*), etc..., el método SAD es el utilizado por ser un algoritmo rápido que presenta características ideales para aplicaciones en tiempo real.

Por lo tanto, en una escena con mucha textura tal como podría ocurrir al aire libre en un bosque, se podría calcular profundidad de cada pixel, mientras en una escena muy bajo de textura, como un pasillo interior, muy pocos puntos pueden registrar profundidad.

1.4.7 RECONSTRUCCION 3D ^[9]

Se calcula las coordenadas (x, y, z) con respecto al sistema de coordenadas de la cámara izquierda con el uso de los parámetros de las cámaras, y de las medidas de disparidad trianguladas entre los puntos correspondientes de las 2 vistas de cada cámara.

1.4.7.1 Matriz reproyectiva

Los puntos en dos dimensiones son reproyectados en tres dimensiones dadas sus coordenadas en pixeles y los parámetros intrínsecos de las cámaras por medio de coordenadas homogéneas mediante la matriz reproyectiva: [9]

$$Q = \begin{bmatrix} 1 & 0 & 0 & -c_x \\ 0 & 1 & 0 & -c_y \\ 0 & 0 & 0 & f \\ 0 & 0 & -1/T_x & (c_x - c_x')/T_x \end{bmatrix} \quad (1.30)$$

Todos los parámetros pertenecen a la imagen de la izquierda, excepto para c_x' . Si los rayos principales se cruzan en el infinito entonces $c_x = c_x'$, y el término en la esquina inferior derecha es 0, esto se da cuando se establezca por programación que $d = 0$ a una distancia finita desde la cámara.

Dado un punto homogéneo de dos dimensiones y su disparidad d asociado, podemos proyectar el punto en tres dimensiones por medio de la ecuación:[9]

$$Q \begin{bmatrix} x \\ y \\ d \\ 1 \end{bmatrix} = \begin{bmatrix} X \\ Y \\ Z \\ W \end{bmatrix} \quad (1.31)$$

Donde las coordenadas 3D son $(X/W, Y/W, Z/W)$.

1.5 SENSORES DE RANGO

Varios de los sensores de rango como laser, ultrasonidos, etc. permiten obtener datos de distancia y ángulos que con un adecuado tratamiento de los mismos se puede utilizar dichos datos para reconstrucción 3D, aunque para representación a colores necesitan de otros sensores solamente para obtención de fuente de color.

1.5.1 3D LASER SCANNER ^[36]

Estos sensores se basan en el principio de TOF (*time of flight*, por sus siglas en inglés), en el cual se envía un haz laser en una dirección fija y se espera su retorno para ser captado por un dispositivo receptor. El principio de medición de distancia se basa en medir el tiempo de vuelo del haz de luz, y conociendo que viaja a la velocidad de luz se puede determinar la distancia a la que se encuentra el punto desde el sensor.

Para superar la limitación de medición de distancias de un solo punto, y poder tener mediciones en 3 dimensiones, se realiza una serie de adecuaciones que permiten construir *scanner* 2D, para luego construir los *scanner* 3D.

Los *scanner* 2D usan un espejo rotativo, el cual desvía el haz laser en varias direcciones alrededor de un eje, con lo que se tiene los valores de mediciones en un plano.

Para tener la tercera dimensión, se ha desarrollado varias alternativas, algunas son desviar la señal laser en dos direcciones por un espejo en lugar de una dirección, o usar más de una fuente laser, o usar un *scanner* 2D que rote alrededor de un eje.

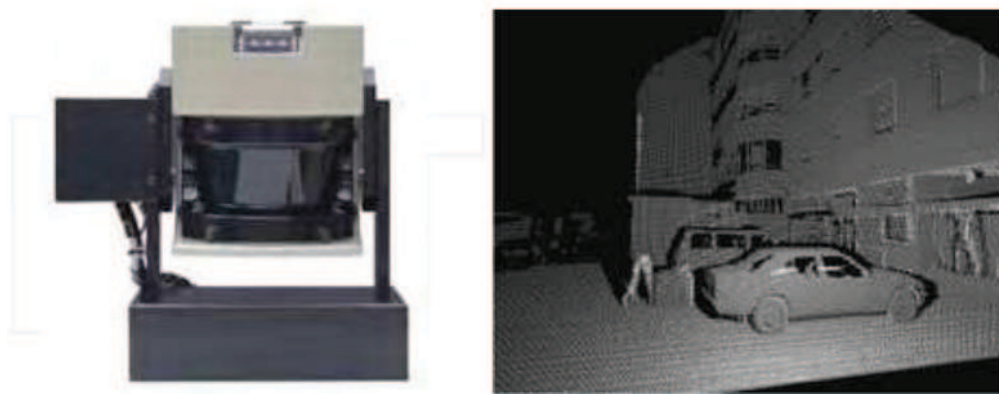


Figura 1.12: Der: Scanner 3D comercial 3DLS. Izq: Reconstrucción 3D realiza por 3DLS [36]

1.5.2 CAMARAS 3D ^[36]

Al igual que los Scanner 3D estos sensores usan el principio de TOF. El ambiente a ser reconstruido es iluminado con *flashes* de luz infrarrojos, y la luz reflejada por los mismos es capturada por sensores CCD, CMOS o alguna tecnología combinada.

La principal diferencia de este sensor con el *scanner* 3D, es que en lugar de realizar un barrido con el láser para obtener todos los valores de distancia punto a punto, la medición de toda la escena se realiza en paralelo obteniendo todos los valores de distancia en una sola toma. Esta característica permite aumentar la velocidad de captura (*frame rate*), y poder usarlo en ambiente dinámicos con objetos en movimiento.

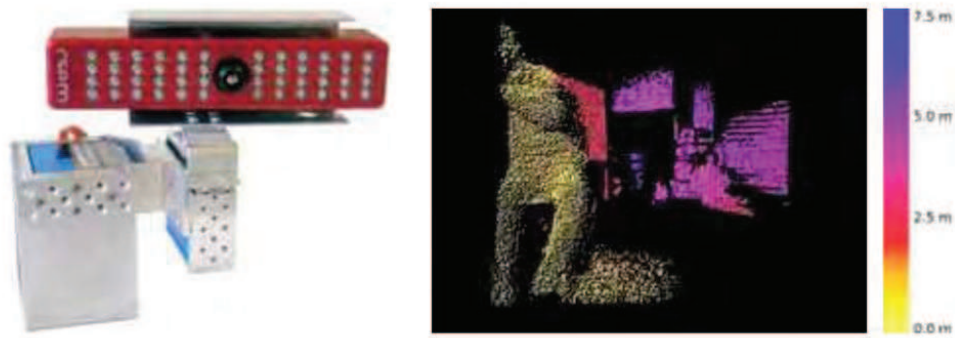


Figura 1.13: Der: Camara 3D SwissRanger SR-2 Izq: Reconstrucción 3D con SwissRanger SR-2 [36]

1.6 FUNDAMENTOS DEL HMI ^{[31], [32], [33]}

1.6.1 DEFINICIÓN

Una interfaz gráfica de usuario GUI, es un programa computacional que está compuesto por ciertos iconos, botones, cuadros de diálogo, etc., en una o más pantallas que trabajan de manera coordinada para la presentación de datos y ejecutar acciones, el usuario controla el programa principalmente por mover un puntero en la pantalla (típicamente controlada por un ratón), y la selección de ciertos objetos por la pulsación de botones, etc.

1.6.2 CARACTERÍSTICAS DE UNA INTERFAZ GRÁFICA

Una interfaz gráfica debe tener las siguientes características:

- Presentación visual es el aspecto visual de la interfaz. Es lo que el usuario observa en la pantalla. La presentación de un sistema gráfico permite visualizar las líneas, incluyendo dibujos e iconos. También permite la visualización de una variedad de fuentes de caracteres, incluyendo diferentes tamaños y estilos. Los gráficos también permiten la animación, y la presentación de fotografías y vídeo en movimiento.
- Los elementos de la interfaz significativos presentados visualmente al usuario en un sistema gráfico, están provistas de ventanas (primarias, secundarias o cuadros de diálogo), de los iconos, de controles, de un ratón u otro dispositivo apuntador, y de el cursor. El objetivo es mostrar

visualmente al usuario el contenido en la pantalla, de manera significativa, de forma sencilla y clara posible.

- Promueve la consistencia de la interfaz entre programas.
- Permite la transferencia de información entre programas.
- Se puede manipular en la pantalla directamente los objetos y la información.
- Existe una muestra visual de la información y los objetos (iconos y ventanas).
- Proporciona respuesta visual a las acciones del usuario.

1.6.3 LIBRERÍA MFC

La librería MFC, es un conjunto de clases interconectadas por múltiples relaciones y herencias, provee un código C++ orientado a objetos sobre Win32 y COMM API (*Application Programming Interface*). Mediante esta librería se pueden crear aplicaciones de escritorios muy simples, pero con gran integración de librerías como las que nosotros utilizamos.

1.6.3.1 Elementos de MFC Utilizados

Para este proyecto se ha utilizado los recursos que son partes de MFC, obteniendo una presentación ordenada de los resultados que se ha logrado del tratamiento de imágenes con *OpenCV*, y PCL en la Figura 1.14 se pueden observar los elementos de MFC utilizados para el diseño de nuestro GUI.

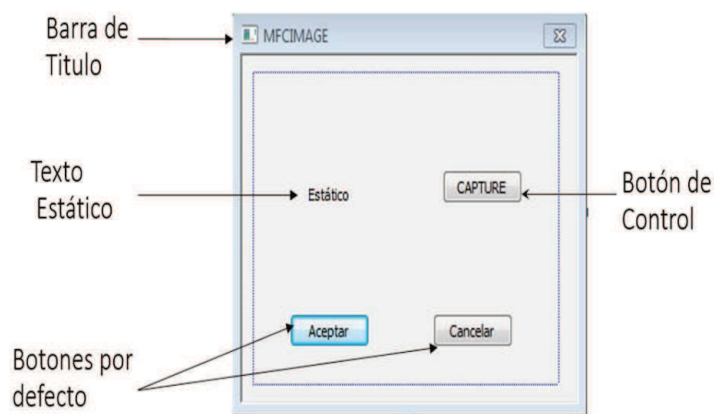


Figura 1.14: Elementos del cuadro de dialogo de MFC

1.6.3.1.1 *Cuadro de Dialogo*

Un cuadro de dialogo (*Dialog Box*) es una ventana que se crea en la cual se puede ubicar texto, controles e imágenes, elementos con los cuales se puede controlar la presentación, y brindará la información necesaria para interpretar la información que se presenta.

En el cuadro de dialogo principalmente aparecen por defecto el cuadro de icono en la esquina superior izquierda y los botones cerrar, minimizar y maximizar en la esquina superior derecha, en las propiedades del cuadro de dialogo se pueden desactivar los botones minimizar y maximizar, excepto el botón cerrar que siempre permanece activo.

1.6.3.1.2 *Botón de Control*

Los botones (*buttons*) permiten ejecutar acciones al presionarlo o hacer doble *click*, dentro de ellos se puede configurar acciones como desplegar o cerrar ventanas, mostrar información o quitarla, y cambiar el valor de alguna variable.

1.6.3.1.3 *Texto Estático*

El texto estático (*Static Text*), permite poner títulos y palabras en el cuadro de diálogo, de forma básica porque en sus propiedades solamente se puede cambiar su contenido y la ubicación con respecto al cuadro de dialogo y su propio espacio, mas no modificar características de forma como su tamaño y su color, para poder cambiar su tamaño se debe programar su cambio mediante líneas de comando. También se puede utilizar para mostrar valores de variables en el cuadro de dialogo.

1.6.4 INTEGRACIÓN DE LIBRERIAS

Para usar las librerías de *OpenCV 2.4.3* y *PCL 1.6.0* junto a la librería *MFC* se debe integrarlas de forma similar a cuando las se integra para aplicaciones de consola.

Pero para mostrar imágenes directamente en los cuadros de dialogo de MFC, se necesita una biblioteca o archivo de *OpenCV* 1.0 denominado *CvImage*, tanto el archivo de código fuente .cpp como el archivo de cabecera o *header* .h, esta biblioteca está diseñada para trabajar directamente con MFC, la cual permite mostrar de manera fácil fotografías en los cuadros de dialogo, con la diferencia que hacerlo directamente desde MFC es complicado.

CAPÍTULO 2 DESARROLLO DEL SOFTWARE DE VISIÓN ARTIFICIAL

2.1 INTRODUCCIÓN ^{[22], [23]}

El desarrollo del software consta de dos partes, la obtención de las coordenadas x,y,z de los puntos característicos de la escena, y el desarrollo de la interfaz la cual controla la ejecución de código y muestra los resultados de una manera interactiva.

Aunque todo el programa principal que controla el cálculo de las coordenadas se encuentra dentro del programa del HMI (*Human Machine Interface*), se explican éstos de manera separada para facilitar la comprensión del texto. Este capítulo se limita a los procedimientos utilizados para la obtención de las coordenadas, el desarrollo del HMI y cómo se interrelaciona con los algoritmos de este capítulo se tratará en el capítulo 3.

Tanto el procesamiento de imágenes como la interfaz se desarrollaron en C++ compiladas con *Microsoft Visual Studio Profesional*.

Ya que C++ no consta por si solo de algoritmos para la adquisición y procesamiento de imágenes por medio de *Microsoft Visual Studio*, se usó dos librerías abiertas muy conocidas en el campo de la visión artificial para estos propósitos, estas son *OpenCV* y *Point Cloud Library*.

OpenCV es creado bajo licencia BSD (*Berkeley Software Distribution*) por lo que su uso es gratuito, tanto como para usos académicos como comerciales. Contienen interfaces C++, C, Python y java compatibles con Windows, Linux, Mac OS,IOS y Android desarrolladas para tener una eficiencia computacional enfocado en aplicaciones en tiempo real.

Point Cloud Library al igual que *OpenCV* es un *software* abierto y gratuito para usos de investigación y comerciales. PCL permite el procesamiento de imágenes

2D/3D y de nube de puntos con algoritmos de filtrado, estimación de características, reconstrucción de superficies, ajuste del modelo y segmentación.

PCL se utilizó para mostrar la reconstrucción 3D de una manera dinámica e interactiva agradable para el usuario, donde se pueda apreciar con detalles la reconstrucción.

Se usa programas auxiliares para obtener toda la información necesaria, para que el programa principal se pueda ejecutar sin complicaciones, estos programas auxiliares son el de adquisición de imágenes y de calibración, serán explicados en las secciones 2.3 y 2.4.

2.2 JUSTIFICACIÓN DEL HARDWARE ^{[37], [38], [39]}

Para la realización de reconstrucciones 3D existen actualmente varias técnicas, y sensores que lo permiten hacer, según el tipo de sensor utilizado se pueden dividir entre sensores pasivos y activos.

Los sensores pasivos son los que usan solo receptores para el cálculo de las distancias, y los métodos activos son los que usan emisores y receptores para el cálculo de distancias.

Los sensores pasivos más utilizados son el de cámaras estero, como es el caso este proyecto, mientras que para los sensores activos se refieren a los sensores que utilizan el principio de TOF, como es el caso de los *scanner 3D*. Para este subcapítulo nos centramos en estos dos tipos de sensores, al ser estos los más comunes en los sistemas actuales de reconstrucción 3D.

Muchas de las características que presentan este tipo de sensores son complementarias, presentan ventajas diferentes que determina el uso de cada uno dependiendo del tipo de resultado que se quiere obtener.

Los sensores activos presentan una mayor exactitud que los sistemas de visión estéreo, aunque tienen la desventaja de presentar una baja resolución en las reconstrucciones que producen.

Las principales desventajas del sistema estéreo de cámaras son que funcionan solamente con objetos de alta textura, debido al problema de correspondencia, y que tienen un alto costo computacional debido al procedimiento de correspondencia.

De manera contraria a los sensores pasivos, los sensores activos tienen un alto rendimiento con objetos de baja textura pero presentan problemas con objetos de alta textura. Razón por lo que dependiendo de la escena bajo reconstrucción los resultado cambian dependiendo del tipo de sensor utilizado.



Figura 2.1: Imagen de la escena reconstruida en las Figuras 2.2 y 2.3 [37]

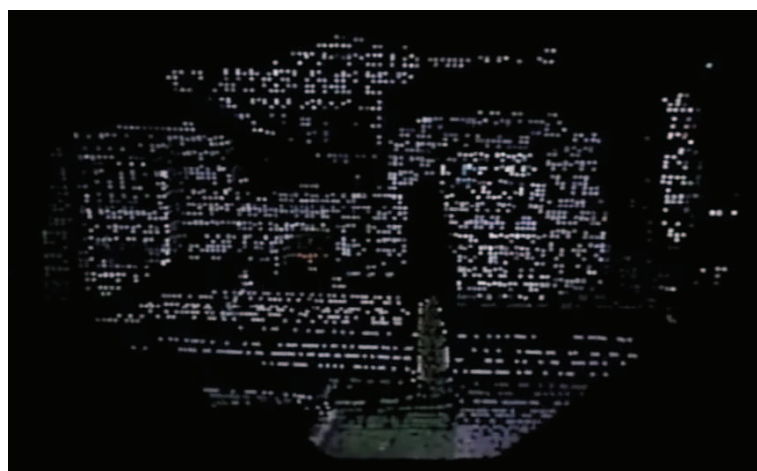


Figura 2.2: Reconstrucción 3D con el uso de un sensor activo. [37]

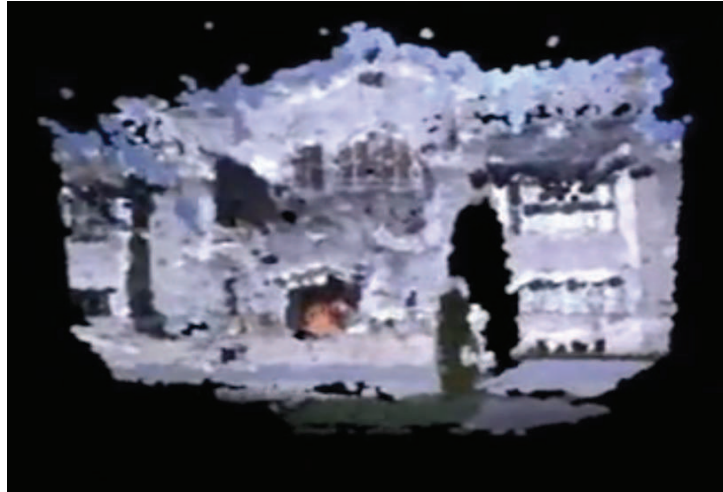


Figura 2.3: Reconstrucción 3D con el uso de un sensor pasivo. [37]

Para una reconstrucción con un sensor activo, se nota la baja resolución al tener pocos puntos en la nube, pero al mismo tiempo son exactas las posiciones de las mismas, sin tener rastros ni sombras en las esquinas.

Para la reconstrucción con el uso del sensor pasivo de la misma escena, se tiene la principal característica de que la misma es a colores, características ausente con el sensor pasivo, con abundantes puntos en su nube, pero con la gran desventaja de tener puntos falsos dentro de su nube debido a la falta de precisión del sistema,

Se han diseñado sensores activos que tienen incluida una cámara, de las usadas en los sensores pasivos, que combinan las ventajas de ambos tipos de sensores para tener un balance entre resolución – exactitud y obtener reconstrucciones claras de alta resolución.



Figura 2.4: Reconstrucción 3D realizada con la combinación de sensores activos y pasivos.[36]

Debido a que el objetivo de este proyecto, es realizar reconstrucciones de escenas interiores con objetos de abundante textura se uso un sistema de visión estéreo. El costo de este sistema es mucho menor que el uso de sensores láseres activos, estos sistemas se siguen mejorando constantemente pero su precio es todavía elevado en comparación con el costo de cámaras *web*.

Las cámaras *web* utilizadas han sido montadas sobre un soporte mecánico para tener la configuración estéreo. Las cámaras usadas fueron Logitech HD Pro Webcam C920, las cuales cuentan con las siguientes características:

- Resolución de hasta 1920 x 1080 píxeles, para el proyecto se usó 640x480 píxeles.
- Tecnología Logitech Fluid Crystal™.
- Lente Carl Zeiss® con enfoque automático de 20 pasos.
- Corrección automática de iluminación escasa.
- Certificación USB 2.0 de alta velocidad (compatible con USB 3.0).
- Requisitos del sistema Windows Vista®, Windows® 7 (32 bits o 64 bits) o Windows® 8.



Figura 2.5: Cámaras utilizadas en el proyecto.

Estas cámaras por configuración de fábrica poseen autoenfoco, lo cual no es deseable, debido a que la distancia focal cambia cada vez que se realiza un nuevo enfoque. Para solucionar este problema por medio del software para Windows de las cámaras esta característica fue desactivada.

2.3 ADQUISICIÓN DE IMÁGENES

Esta sección describe el desarrollo del programa auxiliar para la adquisición de imágenes. El objetivo de este programa auxiliar es obtener pares de imágenes, izquierda y derecha, que sirvan posteriormente tanto para la calibración individual de cada cámara como para la calibración estéreo del sistema de visión y usar estos datos en el programa principal de este proyecto.

Para la adquisición de imágenes se usó un programa en C++ en donde se ingresa por medio de teclado, el número de pares de imágenes que se desea tomar, posteriormente aparecen dos ventanas en donde se muestra todo el tiempo lo que las cámaras izquierda y derecha están adquiriendo en ese momento.

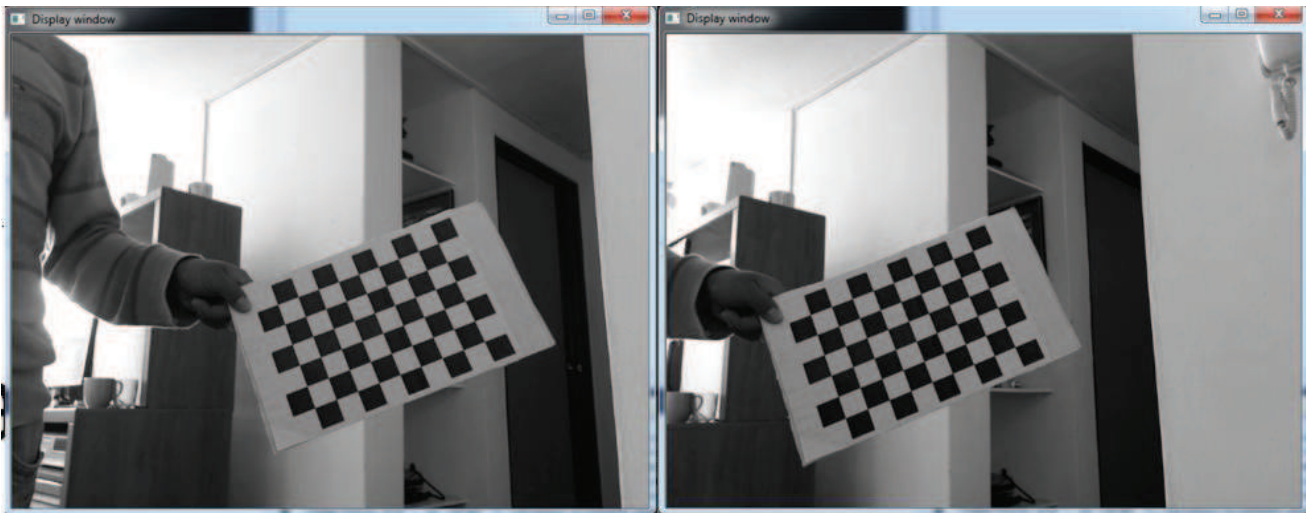


Figura 2.6: Par de imágenes adquiridas el proceso de calibración.

Cada vez que se presiona la barra espaciadora se toma un par de imágenes, mientras las ventanas siguen apareciendo, hasta que se adquieran el total de imágenes ingresadas anteriormente por teclado.

Las imágenes adquiridas para el uso posterior de la calibración son de un tablero de ajedrez de 10x7 cuadros de 2,5cm, cada uno con una resolución de 640x480 pixeles, y un total de imágenes adquiridas de 15. Estas imágenes se almacenan en una dirección preestablecida dentro del computador, y puede ser cambiada a cualquier dirección deseada.

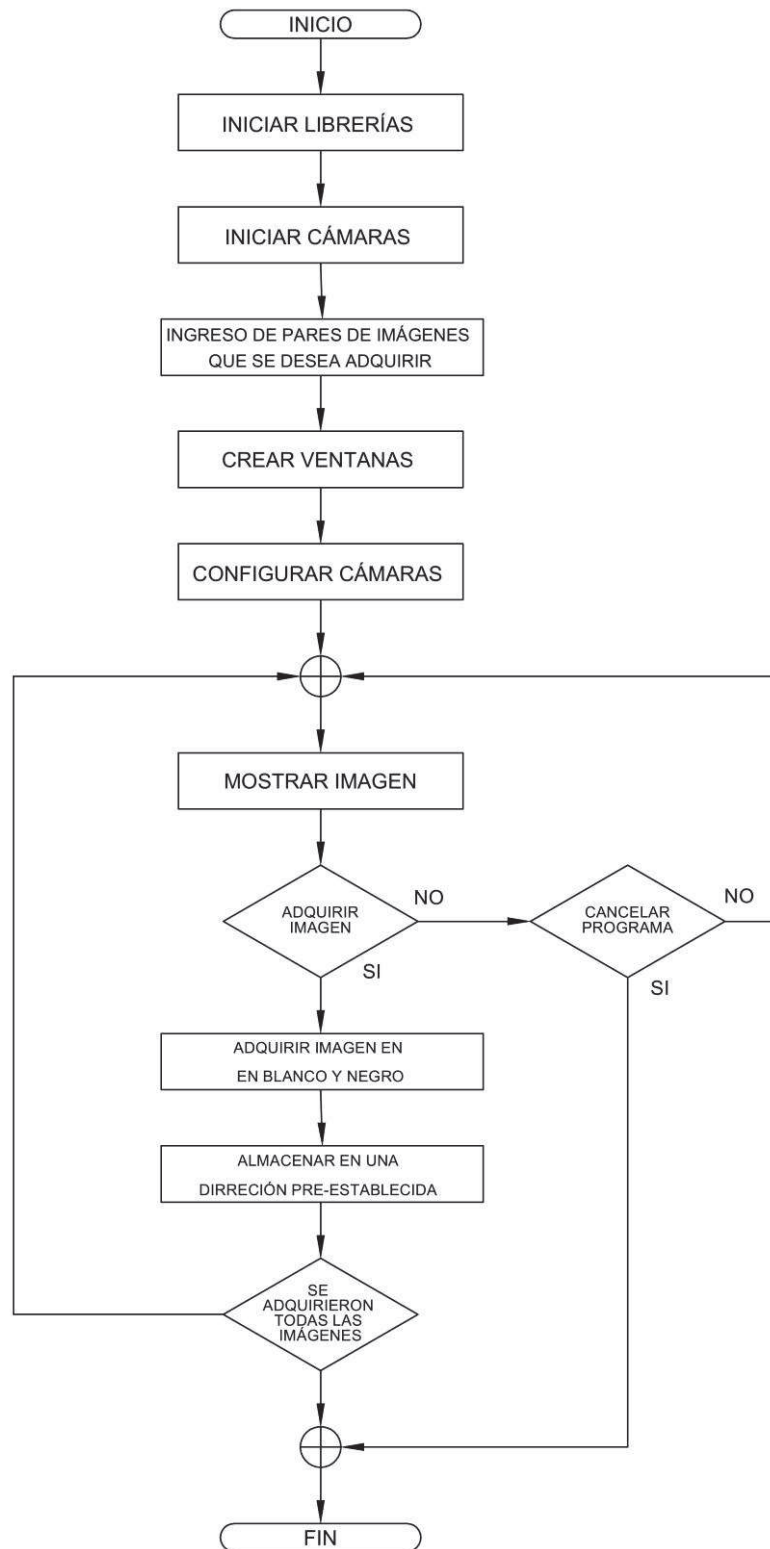


Figura 2.7: Diagrama de flujo de programa de adquisición de imagen

2.4 CALIBRACIÓN DE IMÁGENES ^[25]

Como se mencionó en 1.3.6 llamamos calibración al proceso de calcular los parámetros intrínsecos, extrínsecos y de distorsión de la cámara por medio de imágenes adquiridas por las cámaras a calibrar. En esta sección se describe el desarrollo del segundo programa auxiliar, el de calibración individual y estéreo.

Con suficientes imágenes desde diferentes ángulos de vista de un patrón conocido, que consta de muchos puntos fácilmente identificables, es posible reconstruir la ubicación y orientación de la cámara en el momento de la toma de cada imagen, como los parámetros intrínsecos de la cámara.

El cálculo de estos parámetros se hace por medio de ecuaciones geométricas básicas, que dependen del patrón utilizado para la calibración. El patrón usado para la calibración en teoría puede ser cualquier objeto con suficientes características, entre estos se encuentran patrones tridimensionales y patrones planos.

Los patrones tridimensionales son difíciles de construir con suficientes precisión, por esta razón los patrones que maneja *OpenCV* son planos. Actualmente *OpenCV* es compatible con tres tipos de patrones para la calibración:

- Tablero negro-blanco clásico
- Patrón de círculo simétrico
- Patrón de círculo asimétrico

Se utilizó un tablero de ajedrez por ser un patrón con abundantes puntos característicos fáciles de identificar, al existir alto contraste entre el blanco y negro entre cada casillero. El tablero de ajedrez es un elemento generalizado para la mayoría de procedimientos de calibración, y el más usado para este tipo de propósitos.

Este programa calcula las matrices necesarias para poder utilizar el programa principal en su totalidad, para esto se lee un conjunto de par de imágenes del tablero de ajedrez y se calcula las coordenadas en pixeles de las esquinas del tablero de ajedrez, para luego introducir todos estos valores a un algoritmo que nos va a dar como resultado, los parámetros buscados en el proceso de calibración.

Para el proceso de calibración estéreo se realiza primeramente la calibración individual de cada una de las cámaras, para luego por medio de estos datos, calcular la matriz de rotación R y la de traslación T buscados en la calibración estéreo. Cabe recalcar que aunque se realiza el proceso de calibración individual en cada cámara, estas deben ser calibradas con tomas de las mismas escenas tomadas al mismo tiempo.

Al final del programa se muestra el par de imágenes rectificadas, donde se puede apreciar, el resultado del proceso de rectificación con los valores de calibración calculados.

Para el proceso de calibración se usa los pares de imágenes, que previamente fueron almacenadas en una dirección en el disco duro del computador.

2.4.1 DETECCIÓN DE ESQUINAS ^[26]

Luego de adquirir una imagen y proceder a la adquisición de la siguiente imagen, se calculan las coordenadas en pixeles de las esquinas del tablero de ajedrez en cada una de las imágenes, y se realiza una comprobación visual de las mismas.

Antes del cálculo de coordenadas en sí, se realiza un procesamiento de imágenes previo para aumentar el rendimiento de la operación. Este consiste en normalizar el brillo y aumentar el contraste de la imagen, para seguidamente realizar un “*thresholding*” de la imagen.

El “*thresholding*” es el proceso de segmentar una imagen estableciendo todos los píxeles, cuyos valores de intensidad están por encima de un umbral a un valor de plano, y todos los píxeles restantes a un valor de fondo.

El cálculo del valor de umbral se realiza en cada pixel en función del nivel de intensidad de luz en los pixeles vecinos, lo que permite tener un sistema robusto a los cambios de iluminación.

El cálculo de coordenadas se realiza con una precisión de subpíxeles, lo cual permite tener, suficiente exactitud como para tener resultados óptimos en los procesos posteriores de calibración.

Antes de continuar con el proceso de calibración es importante comprobar los resultados obtenidos, para esto se dibujan círculos alrededor de las esquinas detectadas. Para una detección exitosa se puede observar círculos unidos con líneas en las esquinas detectadas como se muestra en la Figura 2.8.

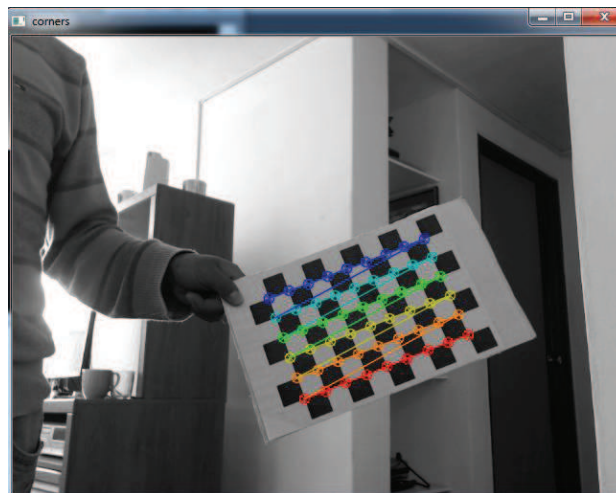


Figura 2.8: Detección exitosa de esquinas.

Por lo contrario cuando la detección de esquinas fue errónea se observan solamente círculos de color rojo en las esquinas como se observa en la Figura 2.9.

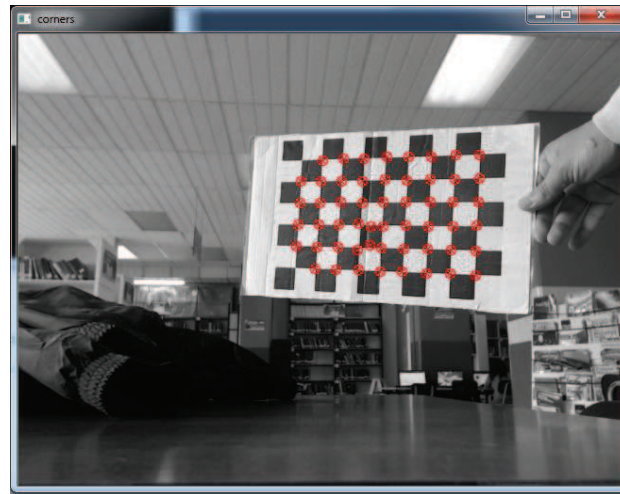


Figura 2.9: Detección errónea de esquinas.

En caso que no se haya detectado todas las esquinas, se debe repetir el procedimiento de adquisición de imágenes como el de calibración. Para evitar este tipo de errores, se debe tener cuidado al momento de adquirir la imagen, el tablero de ajedrez debe aparecer completamente en la imagen.

Se debe mencionar que una detección exitosa de esquina en las imágenes, no garantiza que la calibración y rectificación tenga buenos resultados. Pero si la detección de esquinas no es exitosa, es seguro que no se tendrá un buen resultado de calibración y rectificación.

2.4.2 CALIBRACIÓN ^{[27], [28], [29]}

El método de calibración utilizado usa las coordenadas de las esquinas obtenidas mediante el procedimiento descrito anteriormente, para luego con estos datos y asumiendo un sistema de referencia global calcular la localización y orientación de la cámara en cada una de las tomas como los parámetros intrínsecos de la misma.

Los valores extrínsecos buscados se pueden describir por medio de tres ángulos de rotación y tres valores de traslación, llegando a un total de 6 parámetros extrínsecos.

Los valores intrínsecos se mantienen constantes entre toma y toma, ya que depende de las características de la cámara, estos son los parámetros en la matriz intrínseca M y los coeficientes de distorsión radial y tangencial.

De aquí en adelante se asume $\gamma = 0$, con lo que se tiene un total de 4 parámetros intrínsecos a calcular dentro de la matriz M . *OpenCV* asume $\gamma = 0$ para todos sus cálculos de calibración y posteriores operaciones de la calibración.

Para este proceso *OpenCV* considera 3 parámetros de distorsión radial k_1, k_2 y k_3 y dos parámetros de distorsión tangencial p_1 y p_2 .

2.4.2.1 Coordenadas físicas

Para poder calcular los parámetros antes mencionados, necesitamos tener un sistema de referencia global en el mundo real 3D, que nos permita relacionar las coordenadas físicas de las esquinas fijadas en el tablero de ajedrez, con las coordenadas en píxeles adquiridas previamente.

Las coordenadas de estos puntos, son establecidos en el proceso de calibración por medio algún criterio que influirá en el resto del proceso. La manera en que se defina estos puntos definirá las unidades físicas, y la estructura del sistema de referencia usado a partir de este punto en adelante.

En este proyecto se utilizó puntos en donde todos tienen valor de $z = 0$, es decir están en un plano sin componente en z , y la unidad es la distancia del cuadrado del tablero de ajedrez. Elegir coordenadas con $z = 0$ permite usar la matriz de homografía para el proceso de calibración.

Al elegir la unidad como la distancia del cuadrado del tablero, las coordenadas de la esquinas en el tablero son enteros, en función de la columna y fila en la que se encuentran. Si decimos que S_{fila} y $S_{columna}$ son el número total de filas y columnas respectivamente, las coordenadas de las esquinas interiores que usamos para nuestro procedimiento de calibración son:

$(0,0,0); (0,1,0); (0,2,0); \dots (1,0,0); (2,0,0); \dots (1,1,0); \dots (S_{fila} - 1, S_{columna} - 1, 0)$

2.4.2.2 Cálculo de la matriz homogénea

El método que *OpenCV* utiliza para la calibración, es calcular múltiples matrices homogéneas de múltiples tomas sin conocer nada acerca de los parámetros intrínsecos y extrínsecos, para luego a partir de este conjunto de matrices calcular los valores buscados.

Como se definió en la ecuación 1.23 la matriz homogénea se establece a una escala. El valor de la escala de la matriz no afecta a la ecuación, por lo que solo 8 parámetros son importantes, por lo tanto se establece $H_{33} = 1$ para nuestros cálculos prácticos.

OpenCV calcula la matriz homogénea, de manera que el error de re-proyección es minimizado, por el método de soluciones aproximadas de sistema sobredeterminados (mas ecuaciones que incógnitas) de los mínimos cuadrados. Dado h_{ij} como los elementos correspondientes de la matriz homogénea, el error de re-proyección a minimizar es:

$$\sum_i \left(x_i - \frac{h_{11} * X_i + h_{12} * Y_i + h_{13}}{h_{31} * X_i + h_{32} * Y_i + h_{33}} \right)^2 + \left(y_i - \frac{h_{21} * X_i + h_{22} * Y_i + h_{23}}{h_{31} * X_i + h_{32} * Y_i + h_{33}} \right)^2 \quad (2.1)$$

Cada esquina interior del tablero de ajedrez nos proporciona dos ecuaciones, una por la coordenada X y una por la Y , por lo que el método de los mínimos cuadrados necesitaría como mínimo 4 esquinas para tener una solución de H . Por razones de tener mayor robustez se utilizo 54 esquinas por toma, con 15 tomas lo que da un total de 810 esquinas.

2.4.2.3 Cálculo de los parámetros

El conjunto de matrices homogéneas se utilizan para tener un sistema de ecuaciones que nos permita calcular, los parámetros intrínsecos, extrínsecos y los coeficientes de distorsión.

En este punto el procedimiento siguiente es, calcular una primera aproximación de los parámetros intrínsecos y los parámetros extrínsecos de la cámara, considerando que no existe distorsión en el sistema óptico, para luego recalcular los valores obtenidos previamente y mejorar la exactitud en las mismas.

El método que *OpenCV* utiliza para el cálculo inicial de las parámetros intrínsecos y extrínsecos, es el propuesto por Zhang en su paper “*A Flexible New Technique for Camera Calibration*”. En este *paper* Zhang forma un sistema de ecuaciones, donde el número de ecuaciones depende del número de imágenes adquiridas.

El método de solución que se plantea en el *paper* mencionado, es el cálculo de *eigenvector* y *eigenvalue* de una matriz que se forma por las ecuaciones planteadas. En este método se calcula las incógnitas de un sistema con más o igual número de ecuaciones que incógnitas.

Cómo se ha considerado $\kappa = 0$ el mínimo número de imágenes necesarias para el cálculo de la soluciones es 2, pero considerando el ruido en la adquisición de datos y la estabilidad numérica normalmente se necesitan más de 2 imágenes, para obtener resultados aceptables. Para el caso que se mencionó anteriormente se usó 15 imágenes del tablero.

El resultado obtenido de la solución de este sistema de ecuaciones son los parámetros intrínsecos y extrínsecos. Por efectos de ruido en la adquisición de imágenes la matriz de rotación obtenida contiene errores, por lo que se efectúa un procedimiento previo para la corrección de la misma antes de continuar.

Los resultados obtenidos hasta el momento no consideran los efectos de distorsión de la cámara, por lo que hay error en los parámetros obtenidos. Para solucionar esto con los valores preliminares obtenidos hasta el momento, se calcula los parámetros de distorsión radial k_1, k_2 y k_3 y tangencial p_1, p_2 para posteriormente recalculan los valores intrínsecos y extrínsecos. Estos valores recalculados de cada una de las cámaras son lo que se usa para la calibración estéreo.

2.4.3 CALIBRACIÓN ESTEREO ^[30]

Para obtener los parámetros de la calibración estéreo, primero se realiza el proceso de calibración individual antes descrito en cada una de las dos cámaras. Como resultado de la calibración individual obtenemos los parámetros intrínsecos de cada una de las cámaras, y los parámetros extrínsecos en cada una de las tomas en cada una de las cámaras.

Una vez que se obtiene los parámetros extrínsecos de ambas tomas del mismo par de imágenes, se puede calcular por medio de simples ecuaciones las matrices de rotación y traslación de una cámara respecto a la otra, que son de hecho los valores deseados en el proceso de calibración estéreo.

Con el sistema de coordenadas en el centro de proyección de la cámara izquierda, la relación entre las proyecciones en los planos de imagen derecho e izquierda está dada la ecuación 2.1. Los resultados de la calibración estéreo son la matriz R y T descritos en la ecuación mencionada. [30]

$$P_l = R^{-1}(P_r - T) \quad (2.2)$$

OpenCV calcula las matrices R y T de manera que se satisface las ecuaciones: [30]

$$R_r = R * R_l T_r = R * T_l + T \quad (2.3)$$

Debido a ruido y errores en la adquisición de imágenes, los valores de R y T que se obtiene de cada par de imágenes no son todos iguales, por lo que se calcula las matrices finales R y T minimizando el error de re-proyección, de igual forma como se realizó para la calibración individual de cada cámara.

2.4.4 VERIFICACIÓN DE RESULTADOS

Existe la posibilidad de que el proceso de calibración no se complete con la suficiente exactitud, por lo que es importante verificar los resultados y realizarlos nuevamente en caso de no ser correctos.

Para comprobar el proceso de calibración se realiza la rectificación de los pares de imágenes usadas para la calibración y se las muestra de manera continua en una misma ventana. Adicionalmente se muestra líneas horizontales igualmente espaciadas a través de ambas imágenes que nos ayudan a comprobar el proceso de calibración.

El proceso de rectificación se explicará posteriormente en este proyecto, por el momento nos interesa, tener un procedimiento gráfico que nos permita comprobar el proceso de calibración fácilmente antes de continuar.

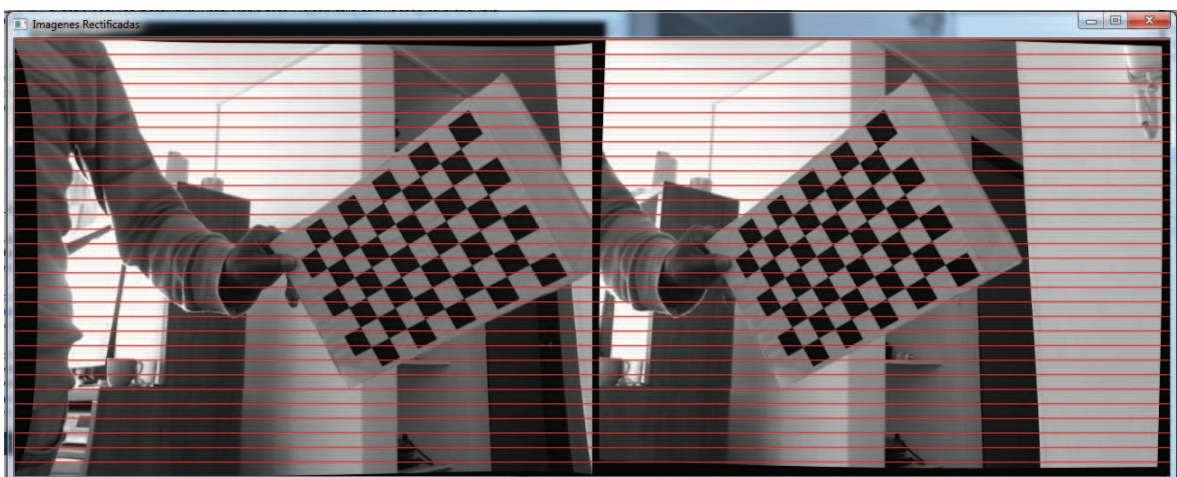


Figura 2.10: Rectificación de imágenes exitosa

Para comprobar que el proceso fue satisfactorio, se debe verificar que los mismos puntos en ambas imágenes se encuentran en la misma línea horizontal, y que las

imágenes rectificadas no presenten distorsiones considerables con respecto a las originales.

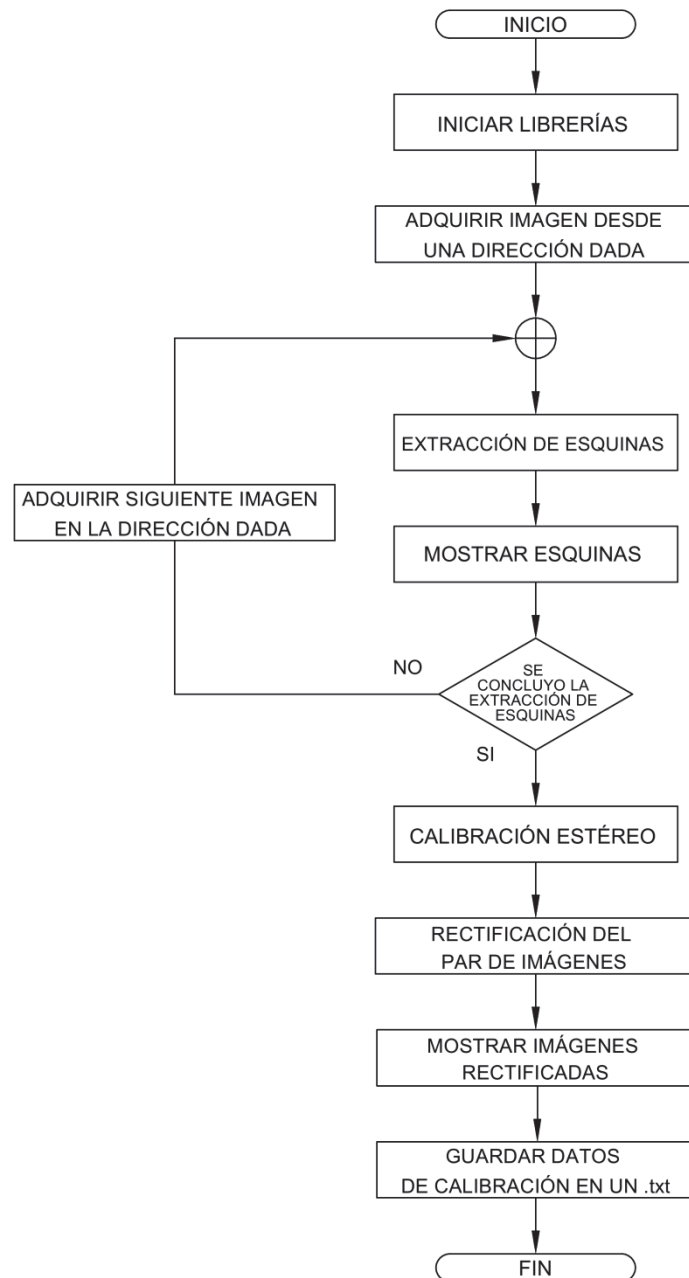


Figura 2.11: Diagrama de flujo del programa de calibración

2.5 RECTIFICACIÓN DE IMÁGENES

En este subcapítulo se hace referencia a los algoritmos propios del proceso de rectificación, que se encuentran dentro del programa principal del HMI, mas no a

cómo interactúan los mismos dentro de la interfaz, en el capítulo 3 se hace referencia a los mimos y a su uso dentro de la interfaz.

El proceso de rectificación consiste en el proceso de re-proyectar las imágenes, de manera que luego de este proceso ambas se encuentran en un arreglo paralelo frontal con líneas horizontalmente alineadas, como se muestra en la Figura 2.10.

Se utilizó el algoritmo de Bouguet para el proceso de rectificación, este el método más usado para rectificaciones estéreo cuando se conoce los parámetros de la calibración estéreo, y es el que ofrece *OpenCV*.

Para facilitar y obtener mejores resultados en este proceso, se ubicaron las cámaras lo más aproximado a un arreglo alineado horizontalmente. Al hacer esto los cálculos de la calibración estéreo se facilitan, y los resultados nos dan imágenes rectificadas con menor distorsión.

Para este proceso se calcula las matrices R_l y R_D indicadas en 1.4.4.1, y se recalculan las matrices de re-proyección para cada una de las cámaras con respecto a los nuevos planos asignados para la misma.

Con los parámetros de distorsión obtenidos de la calibración individual de cada cámara, se corrige los efectos de los mismos para posteriormente rotar los planos acorde a R_l y R_D . Luego de estos dos procesos, obtenemos las ubicaciones de cada pixel en la imagen destino correspondientes en la imagen obtenida por la cámara (este proceso es conocido como *mapping*). Este proceso de *mapping* se realiza individualmente para cada imagen.

Antes de proceder a realizar el proceso de *mapping*, se debe considerar el hecho que las ubicaciones destino pueden tener coordenadas no enteras en pixeles, lo que provocaría que estos puntos no sean reubicados al tener coordenadas que no existen dando como resultados muchos espacios vacíos.

Para solucionar esto *OpenCV* trabaja de manera inversa, para cada localización de pixel en la imagen destino se calcula la localización en la imagen fuente, en donde lo más probable es que se obtenga una coordenada no entera, y luego se realiza un proceso de interpolación con los pixeles vecinos para obtener una localización de valores enteros de pixeles.

Para este caso se realiza este proceso dos veces, uno entre la imagen rectificada y la imagen no distorsionada, y la otra entre la imagen no distorsionada y la imagen original. Luego de este proceso obtenemos como resultado las matrices, tanto para x como para y , que ubican los pixeles de las imágenes orígenes a las imágenes destino sin tener espacios vacíos.

Estas matrices nos son útiles para todos los pares de imágenes que deseamos rectificar, por lo que son almacenadas y llamadas cada vez que se desee realizar una rectificación.

Este proceso de calibración descrito es el utilizado por uno de los programas ejemplos de *OpenCV* que se obtiene al instalar el mismo.

2.6 MAPA DE DISPARIDAD

En este subcapítulo, se hace referencia a los algoritmos propios del proceso del cálculo del mapa de disparidad, que se encuentran dentro del programa principal del HMI mas no a cómo interactúan los mismos dentro de la interfaz, en el capítulo 3 se hace referencia y a su uso dentro de la interfaz.

El proceso de cálculo del mapa de disparidad corresponde de 3 procedimientos, primero un pre-filtrado de las imágenes, segundo el proceso de correspondencia a lo largo de la línea epipolar y tercero un post-filtro que elimina las falsas correspondencias.

En el proceso de pre-filtrado lo que se busca es reducir las diferencias de luz, y aumentar las texturas en las imágenes de manera que el algoritmo de

correspondencia encuentre mayor número de correspondencias, sin aumentar el tiempo de procesamiento.

Para el proceso de correspondencia, en cada punto en la imagen izquierda buscamos el punto correspondiente en la imagen derecha por medio de una ventana SAD, método explicado en 1.4.7.1, a lo largo de la línea epipolar hacia la izquierda de la coordenada del punto en la imagen izquierda.

La razón para buscar a lo largo de la izquierda de la línea epipolar, y no en toda la línea epipolar, se debe al arreglo frontal paralelo utilizado con el cual no obtenemos valores de disparidad negativos, lo que nos permite ahorrar tiempo de procesamiento que agiliza nuestro programa.

La búsqueda de correspondencia a lo largo de toda la línea epipolar, demanda un gran tiempo de procesamiento al tener que calcular el valor SAD alrededor de cada pixel, por lo que se limita la máxima y mínima distancia de búsqueda a lo largo de la línea epipolar, de manera de obtener un algoritmo rápido óptimo para aplicaciones en tiempo real.

Finalmente antes de mostrar los resultados del mapa de disparidad, se realiza un post-filtrado para eliminar falsas correspondencias (ya sea porque el punto no era suficientemente característico para que lo detecte el algoritmo SAD, por efectos de ruido, o porque el punto se encontraba oculto en la imagen correspondiente), y tener un mapa de disparidad más real.

El mapa de disparidad es una imagen a escala de gris con los respectivos valores de disparidad en pixeles (de los puntos que fue posible determinar), en donde los puntos más claros representan puntos más cercanos y los más oscuros representan los puntos más alejados desde la cámara.

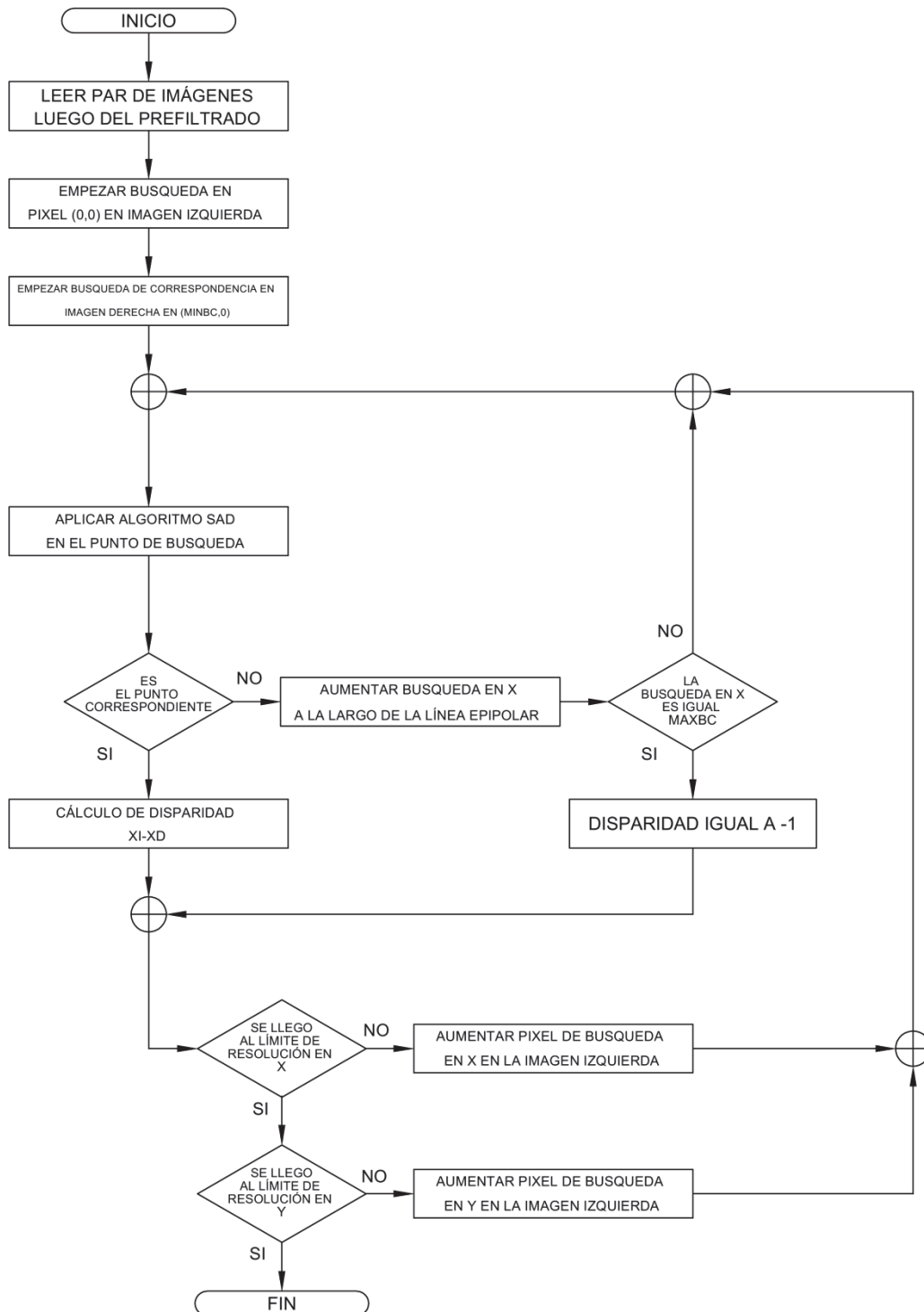


Figura 2.12: Diagrama de flujo del algoritmo de correspondencia.

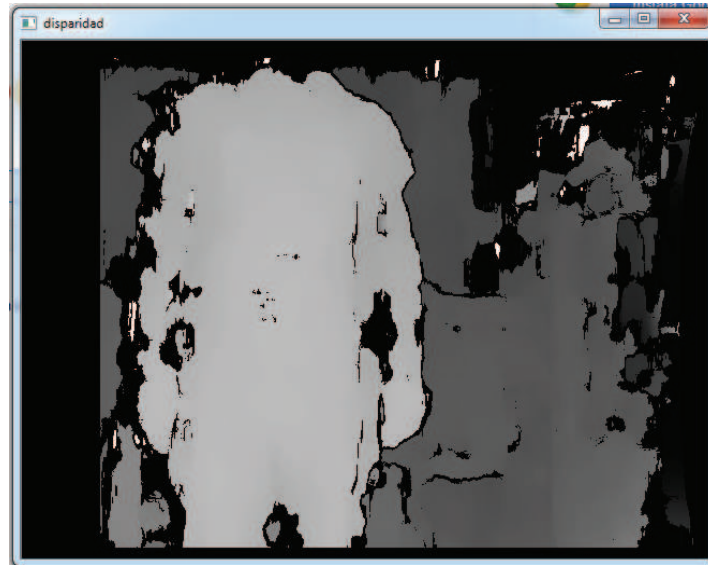


Figura 2.13: Mapa de disparidad.

En donde no fue posible calcular los valores de disparidad, se asigna valores de disparidad de -1 los mismos que aparecen como “vacíos” de color negro en el mapa de disparidad, y no se tiene información de los mismos acerca de la distancia a la que se encuentran desde la cámara.

La generación del mapa de disparidad se controla por medio de parámetros de distancia, nitidez y velocidad de cálculo del mismo. Estos parámetros fueron calibrados de manera que se pueda distinguir distancias entre 1 y 4 metros, y se tenga suficientes puntos reconocidos en el mapa de disparidad como para tener una reconstrucción 3D aceptable. Se hablara más sobre este tema en el subcapítulo 4.2.

El mapa de disparidad obtenido es válido para proceder a calcular distancias y la posterior reconstrucción 3D, pero no es factible para ser presentado al usuario final, debido a que no se tiene suficiente contraste como para poder apreciar las diferentes distancias, y no se puede distinguir distancias por simple inspección del mismo.

Para solucionar este problema y obtener imágenes que muestren de manera dinámica los resultados obtenidos, se utilizó el mapa de disparidad a blanco y negro para obtener una imagen a colores, en donde cada color representa un rango de distancias diferentes.

Se discrimino 10 diferentes rangos de distancias en función de su valor de disparidad por medio de la ecuación 1.18, y a cada uno se le asignó un color diferente. Para las distancias más lejanas se asignó el color azul y a medida que el valor de distancia disminuye, el color va cambiando paulatinamente hasta llegar al color rojo para las distancias más cercanas.



Figura 2.14: Gama de colores en el mapa de disparidad.

En la tabla 2.1 se muestra los rangos de distancia utilizados, con sus respectivos valores de disparidad y la mezcla de colores correspondientes.

	Z		d		COLORES		
	MIN	MAX	MAX	MIN	R	G	B
COLOR 1	0	100	64	54	255	0	0
COLOR 2	100	137,5	54	39	255	0	51
COLOR 3	137,5	175	39	31	255	0	102
COLOR 4	175	212,5	31	25	255	0	153
COLOR 5	212,5	250	25	22	255	0	204
COLOR 6	250	287,5	22	19	255	0	255
COLOR 7	287,5	325	19	17	192	0	255
COLOR 8	325	362,5	17	15	129	0	255
COLOR 9	362,5	400	15	14	100	0	255
COLOR 10	400	inf	14	0	0	0	255

Tabla 2.1: Rangos de distancia utilizados en el mapa de disparidad a colores.

Para los valores donde la disparidad no se pudo calcular, y no se conoce el valor de la misma, se asigna el color blanco de manera que exista contraste con el resto de colores.

En la Figura 2.15 se muestra una imagen con el mapa de disparidad a colores, seguido por la imagen de la cámara derecha, la cual se utilizó para la obtención del mismo.

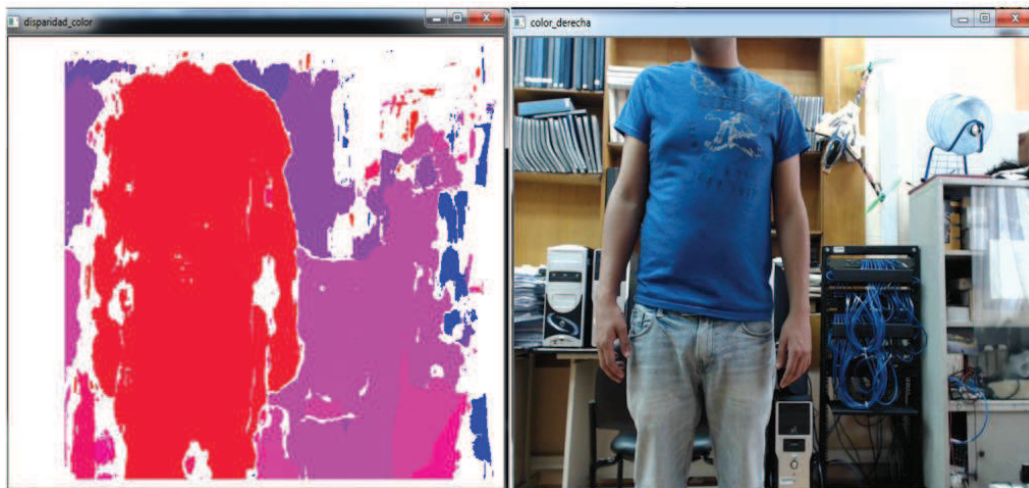


Figura 2.15: Mapa de disparidad a colores, Izquierda: Imagen derecha sin rectificar

Es importante saber los mínimos y máximos valores de distancia que el sistema puede detectar, de manera que tenemos una idea de lo que podemos esperar de nuestro sistema, las siguientes ecuaciones nos dan idea de esto:

- Mínima distancia detectable.

$$z = \frac{F * T}{d} = \frac{605.99 \text{ px} * 9.07 \text{ cm}}{2 * 480 \text{ px}} = 5,72 \text{ cm}$$

- Máxima distancia detectable.

$$z = \frac{F * T}{d} = \frac{605.99 \text{ px} * 9.07 \text{ cm}}{1 \text{ px}} = 54.9 \text{ m}$$

Para el caso de este proyecto se detecta las disparidades mínimas de 1 pixel por lo que en teoría, se puede llegar a medir hasta 54,9 m si el sistema SAD logra identificar correspondencia de 1pixel.

La máxima disparidad que se detecta es de 64 pixel, ya que la búsqueda de correspondencia está limitada a este valor, por lo que la mínima distancia que se puede detectar es de 85.88 cm, la cual cumple con los objetivos propuestos para este proyecto.

2.7 RECONSTRUCCIÓN 3D

En este subcapítulo se hace referencia a los algoritmos propios para el cálculo de coordenadas x, y y z , que se encuentran dentro del programa principal del HMI, mas no a cómo interactúan los mismos dentro de la interfaz, en el capítulo 3 se hace referencia a los mimos y a su uso dentro de la interfaz.

Una vez que se obtiene los valores de disparidad solo estamos a un paso de calcular las distancias para cada coordenada x, y en la imagen, para esto utilizamos la ecuación 1.30 con los valores del mapa de disparidad en escala de grises.

Se aplica esta ecuación considerando que los valores donde no fue posible calcular la disparidad, deben de ser excluidos del cálculo de coordenadas.

El resultado que se obtiene es una imagen de 3 canales en donde cada canal contiene las coordenadas x, y y z que se usan en la reconstrucción 3D.

Para la reconstrucción se toma la coordenada x, y y z de cada punto, discriminando las distancias mayores a 8m. Se discriminó las distancias mayores a 8m para evitar graficar las distancias con valores infinitos, que en realidad corresponden a los puntos donde la disparidad no pudo ser calculada.

Se eligió una distancia de 8m como límite, considerando que distancias mayores a las mismas serán difícilmente detectadas por el sistema SAD y en mayoría son distancias infinitas que no son de interés para la reconstrucción.

Para obtener la información de color se aplicó a la imagen izquierda a color el proceso de *mapping* mencionada en 2.6 y se extrajo de la misma el color correspondiente para cada coordenada x, y y z .

En la Figura 2.16 se muestra el resultado obtenido de la reconstrucción mostrada por medio de PCL:



Figura 2.16: Imagen .jpg de reconstrucción 3D.

CAPÍTULO 3 DISEÑO E IMPLEMENTACIÓN DEL HMI

Durante el desarrollo del HMI, el cual incluye el diseño e implementación, siendo el diseño una parte muy importante para la creación del HMI, que en este caso será una interfaz gráfica de usuario GUI. Si bien es cierto, no existe un procedimiento específico para el diseño de GUI, se pueden seguir varios caminos en el diseño de un GUI, antes de adentrarse a la programación del código del mismo.

En este proyecto se desarrollo una interfaz hombre máquina (HMI), la cual permita presentar al usuario, los resultados de manera ordenada para su mejor entendimiento.

3.1 DISEÑO DEL GUI ^[35]

Para el diseño del GUI se ha integrado las librerías *OpenCV* 2.4.3, *PCL* 6.0, *MFC*, y *CvImage* de *OpenCV* 1.0, dentro del entorno de programación de *Microsoft Visual Studio* 10, se ha creado un cuadro de dialogo como ventana principal en la cual se ubica botones de control, y un patrón de colores para poder entender de mejor manera el mapa de disparidad calculado con el software de estéreo visión, para esto se ha utilizado un computador de 4GB de memoria RAM, y un procesador Intel Core i7, con una pantalla de 1600x900 pixeles, sistema operativo *Windows 7 Home Premium*, dentro de la cual se ubica nuestras ventanas principales y secundarias.

3.1.1 PRINCIPIOS DE DISEÑO

Se ha encontrado una diversidad de documentos para el diseño de un GUI, en la mayoría de ellos coinciden con temas fundamentales, y son las que se aplicará para el diseño del GUI para este proyecto.

Las características que se requieren de un GUI son simplicidad, consistencia y familiaridad. Estas características serán el principio de diseño del GUI, resaltando en gran parte la simplicidad, porque estará enfocado a mostrar solamente

información que sea de mayor utilidad, y a su vez se utilice los recursos del *hardware* de manera óptima, no siempre un GUI lleno de menús y objetos da la información que el usuario requiere, y muchos objetos innecesarios consumen recursos de hardware.

Con esto se pretende que el usuario, desde la primera vez que interactúe con el GUI pueda entender el proceso que se lleva a cabo, teniendo objetividad en la muestra de resultados obtenidos.

3.1.2 PROCESO DE DISEÑO

Como se mencionó anteriormente el diseño del GUI debe hacerse previo a su implementación, si nos adentramos a la programación sin este paso previo, lo más probable es que surjan inconvenientes, y no se llegue completamente a la consecución de los objetivos, y se termine invirtiendo mayor tiempo.

En la fase de diseño consideraremos los siguientes procedimientos a seguir, requerimientos iniciales en cuanto a recursos y la tarea que se asignará a cada recurso, y un croquis inicial del GUI esperado.

3.1.2.1 Requerimientos iniciales

Luego de haber analizado nuestro principio de diseño, vamos a describir una serie de requerimientos que se espera del GUI para conocer los recursos de software que utilizaremos:

- Ventana principal para ubicar controles, imágenes y texto estático.
- Ventanas secundarias para mostrar: el mapa de disparidad, una imagen a color de la escena, una imagen del doble de resolución de las dos anteriores para mostrar la rectificación del par de imágenes.
- Un visualizador 3D para mostrar la reconstrucción.
- Botones de control, uno para iniciar el proceso, uno para capturar la escena deseada, y uno para finalizar el proceso.
- Imágenes: sello de la institución, y patrón de colores del mapa de disparidad.

- Texto estático, para identificar la institución, el título del proyecto, identificar los diferentes rangos e distancia.

3.1.2.2 Croquis del GUI

Después que se ha realizado un análisis previo de los recursos, y asignar las funciones que llevarán a cabo se va a representar en un croquis la ubicación de cada elemento que conforma el GUI:

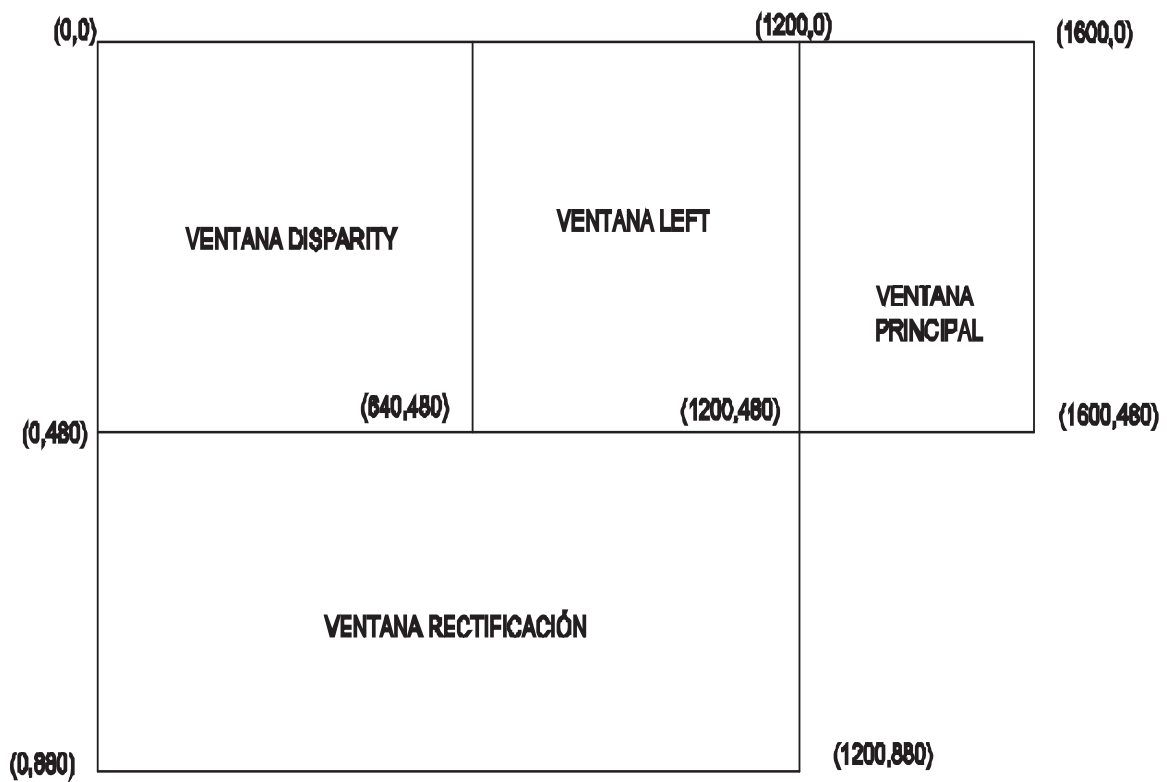


Figura 3.1: Croquis del GUI.

En el croquis se puede ver como se han distribuido la ventana principal, la cual contendrá los tres botones de control, las 2 imágenes, y el texto. Las ventanas secundarias han sido ubicadas de manera que, se pueda ver la información en su totalidad, y no interfiera ninguna.

En este croquis no se observa el visualizador 3D, debido a que se considera que será un objeto que trabajara independiente a las ventanas secundarias, y podrá ser ampliado para un mejor análisis del resultado final.

Con este proceso de diseño esperamos obtener un GUI, que nos brinde de la manera clara la información relevante que se obtiene en este proyecto.

3.2 IMPLEMENTACIÓN DEL GUI

Luego de las consideraciones llevadas en el proceso de diseño, se procede a programar el GUI, en esta parte será de gran ayuda el software considerado para la implementación, habiendo integrado en el entorno de programación de *Microsoft Visual Studio 10* tanto las librerías para visión como las que se utiliza específicamente para el GUI, es decir, reuniendo todo los recursos en un proyecto MFC.

Creamos un proyecto MFC asistido por cuadro de dialogo, el cual directamente nos da una ventana principal, a donde vamos a arrastrar tres botones, y los textos estáticos necesarios. Para programar la función de cada botón se da doble *click* en cada uno de esto y se incluirá el código fuente que queremos que se ejecute.

Para incluir las imágenes dentro de la ventana principal nos ayudaremos de la biblioteca "*CvImage*", dentro de la función *OnPaint()*, cambiaremos esta variable local como variable global "*CPaintDC dc(this)*", para poder utilizar la función: *CvImage::Show(HDC dc, int x, int y, int w, int h, int from_x, int from_y)*, la cual permite fijar las coordenadas (x,y) en las que se ubicara la imagen dentro del cuadro de dialogo, la altura y el ancho (w,h) que se desea que se muestre la imagen y las coordenadas iniciales de la imagen (x, y).

3.2.1 VENTANA PRINCIPAL

La ventana principal es el cuadro de dialogo que tiene una resolución de 400x480 que aparece en el centro de la pantalla, pero la arrastraremos hasta las coordenadas (1200,0) para poder visualizar todas las ventanas secundarias, esta ventana contiene el título principal, el botón START, el botón CAPTURE, el botón STOP y el patrón de colores para el mapa de disparidad, esta ventana la denominamos "3DRVS", en la Figura 3.2, se muestra la ventana principal.

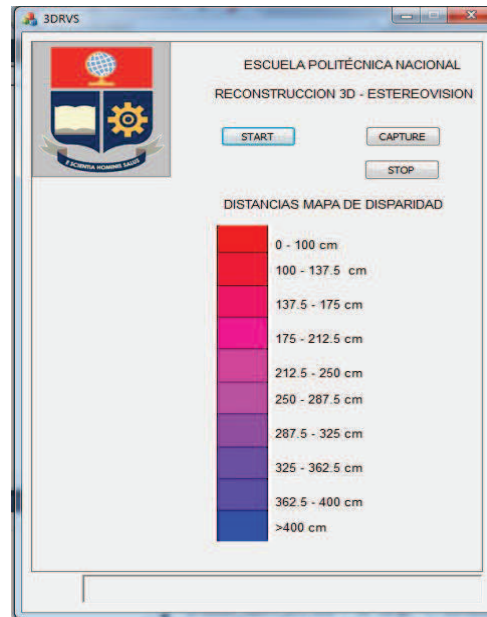


Figura 3.2: Ventana Principal

3.2.2 VENTANAS SECUNDARIAS

Utilizando la biblioteca de *OpenCV* “highgui”, se despliega la ventana “*DISPARITY*” para mostrar el mapa de disparidad en las coordenadas (0,0) con una resolución de 640x480, según el Patrón de colores de profundidad. Se despliega la ventana “*LEFT*” con la imagen a color de la cámara izquierda en las coordenadas (640,0) con una resolución de 560x480 pixeles. Se despliega la ventana “RECTIFICACIÓN” para mostrar la imagen rectificada izquierda y derecha en escala de grises en la coordenadas (0,480) con una resolución de 1200x400 pixeles.

Utilizando la biblioteca de PCL “pcl_visualizer” desplegamos el *viewer* 3D para mostrar la reconstrucción 3D en forma de nubes de puntos a color RGB.

La Figura 3.3. muestra la disposición de los componentes en la pantalla, el *viewer* 3D no tiene posición definida, debido a que se despliega solo en ciertas condiciones que se explica en las siguientes secciones.

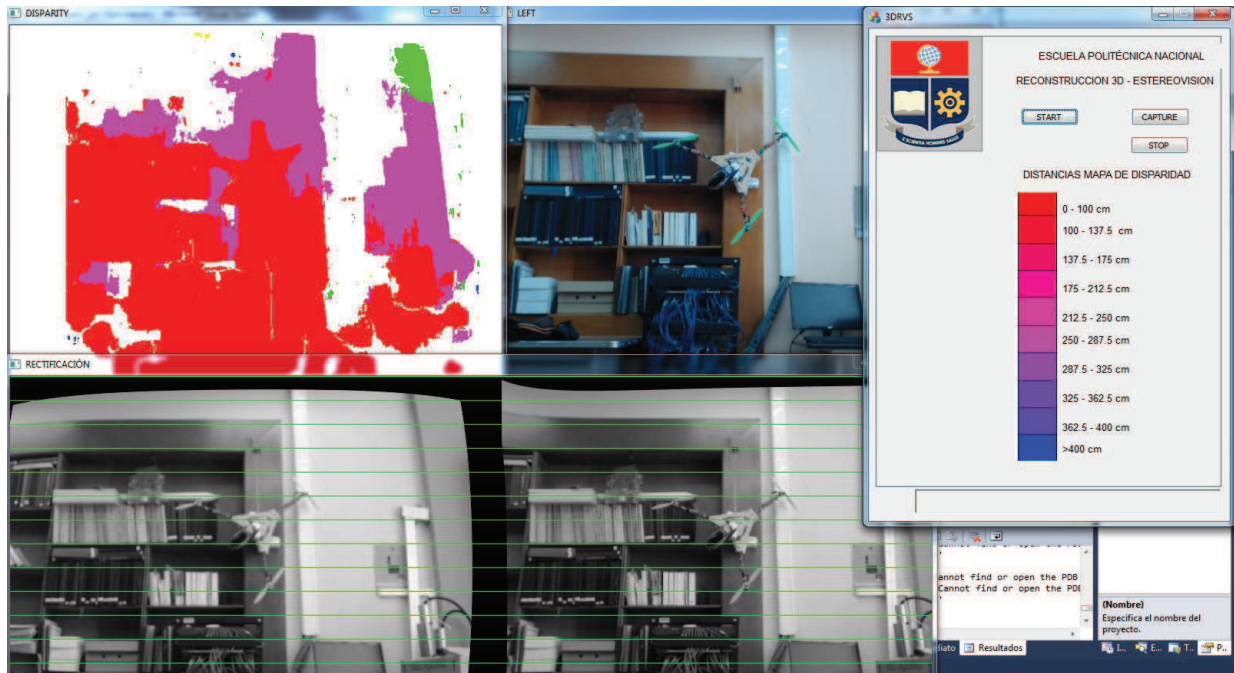


Figura 3.3: GUI con Ventanas Principal y Ventanas Auxiliares

3.2.3 FUNCIONAMIENTO DEL GUI

Primeramente se debe conectar las dos cámaras en los puertos USB, del computador, luego se ejecuta el programa desde el *Microsoft Visual Studio 10*, de aquí se despliega la ventana principal con los tres botones de control la función de cada botón se describe en los siguientes párrafos, se puede ver su diagrama de flujo en el Anexo C en la Figura C.4.

3.2.3.1 Botón “START”

Al presionar este botón inmediatamente se encienden las dos cámaras, luego se despliega las ventanas “DISPARITY”, la ventana “LEFT” y la ventana “RECTIFICACIÓN”, el algoritmo de visión estéreo empieza a ejecutarse, y se actualiza cada 5 ms.

3.2.3.2 Botón “CAPTURE”

Al presionar el botón “CAPTURE”, las cámaras capturan las imágenes izquierda y derecha en ese instante y luego se procede a ejecutar el programa para la representación de la reconstrucción 3D en forma de nubes de puntos,

inmediatamente se despliega el viewer 3D y utilizando el mouse podemos empezar a ver la reconstrucción desde varias perspectivas.

3.2.3.3 Botón “STOP”

Al presionar el botón “STOP”, se cierran las ventanas “RECTIFICACIÓN”, la ventana “LEFT”, y la ventana “DISPARITY”, y la ventana principal. Deja de ejecutar todo el código de visión y se apagan las cámaras.

3.2.3.4 Ventana “RECTIFY”

En la ventana “RECTIFY” se muestra la imagen del mapa de disparidad, para el programa se ha hecho una modificación para mostrar el mapa de disparidad según un patrón de colores que representan rangos de distancias de profundidad pues generalmente el mapa de disparidad se representa solamente en escala de grises, el Patrón de colores se puede observar en la ventana principal y mediante ellos se puede ir comprobando los objetos que se encuentran en esos rangos de distancia desde la ubicación de las cámaras. En la Figura 3.4 se puede observar la imagen de un mapa de disparidad y el patrón de distancias.

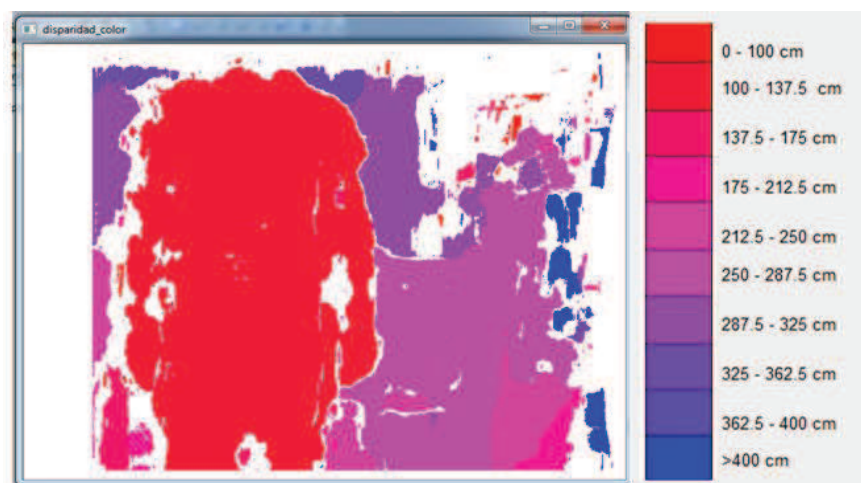


Figura 3.4: Mapa de disparidad y Patrón de colores

3.2.3.5 Ventana “LEFT”

La ventana “LEFT” solamente muestra una imagen a color de la cámara izquierda en la cual se tiene el punto de origen de las coordenadas dentro del tratamiento

de visión estéreo, esta foto sirve de referencia para poder ver los colores de la fuente en ese instante en caso de presionar el botón START, y la imagen a reconstruirse en caso de presionar el botón CAPTURE.

3.2.3.6 Ventana “RECTIFICACIÓN”

La ventana “RECTIFICACIÓN” permite ver la imagen rectificada que se han aplicado los factores de corrección y los parámetros extrínsecos, dentro de la cual se puede visualizar las líneas epipolares de color verde, con esta imagen se puede saber cuándo ha habido un movimiento de las cámaras, y se han descalibrado para volverlas a calibrar correctamente.

Es importante poder visualizar la imagen rectificada, debido a que cualquier variación de la una cámara con respecto a otra puede conllevar a un error en la calibración, y por ende a una reconstrucción errónea que también se refleja en los otros resultados, pero con esta imagen se sabe exactamente que es por una mala calibración de las cámaras.

En la Figura 2.10 se puede observar una imagen rectificada y sus líneas epipolares.

3.2.3.7 Visualizador 3D ^[34]

El visualizador 3D permite observar la reconstrucción 3D en forma de nubes de puntos en colores RGB, para manipular el visualizador se puede utilizar el ratón para alejar o acercar las vistas se debe hacer rotar el SCROLL para adelante o hacia atrás, para hacer rotar las vistas se debe presionar el botón izquierdo del mouse y al mismo tiempo mover hacia el lado izquierdo o derecho depende a qué lado se desea rotar la imagen.

Adicionalmente se puede utilizar el teclado con las siguientes funciones:

- j, J : Tomar una foto instantánea en .png de la vista actual del visualizador.

- g, G : Mostar el grid de escala (on/off).

En la Figura 3.5 y Figura 3.6 se detalla el diagrama de flujo del funcionamiento de la interfaz gráfica de usuario.

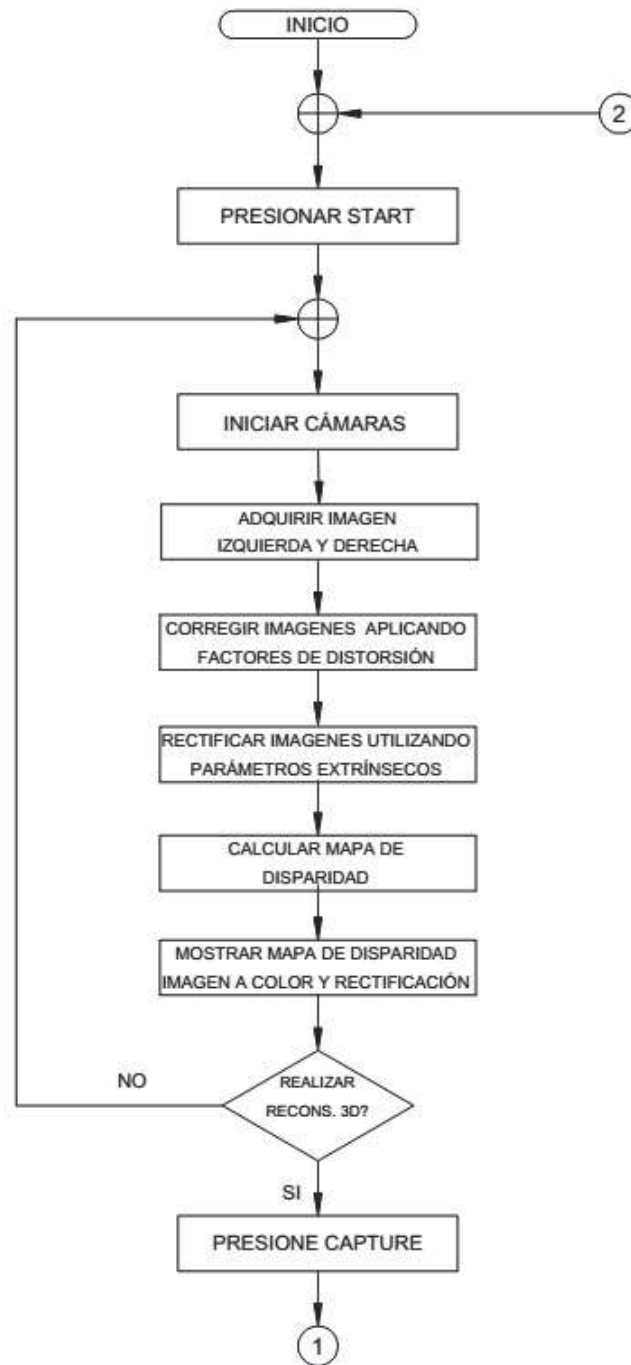


Figura 3.5: Diagrama de flujo del funcionamiento del GUI (Parte 1)

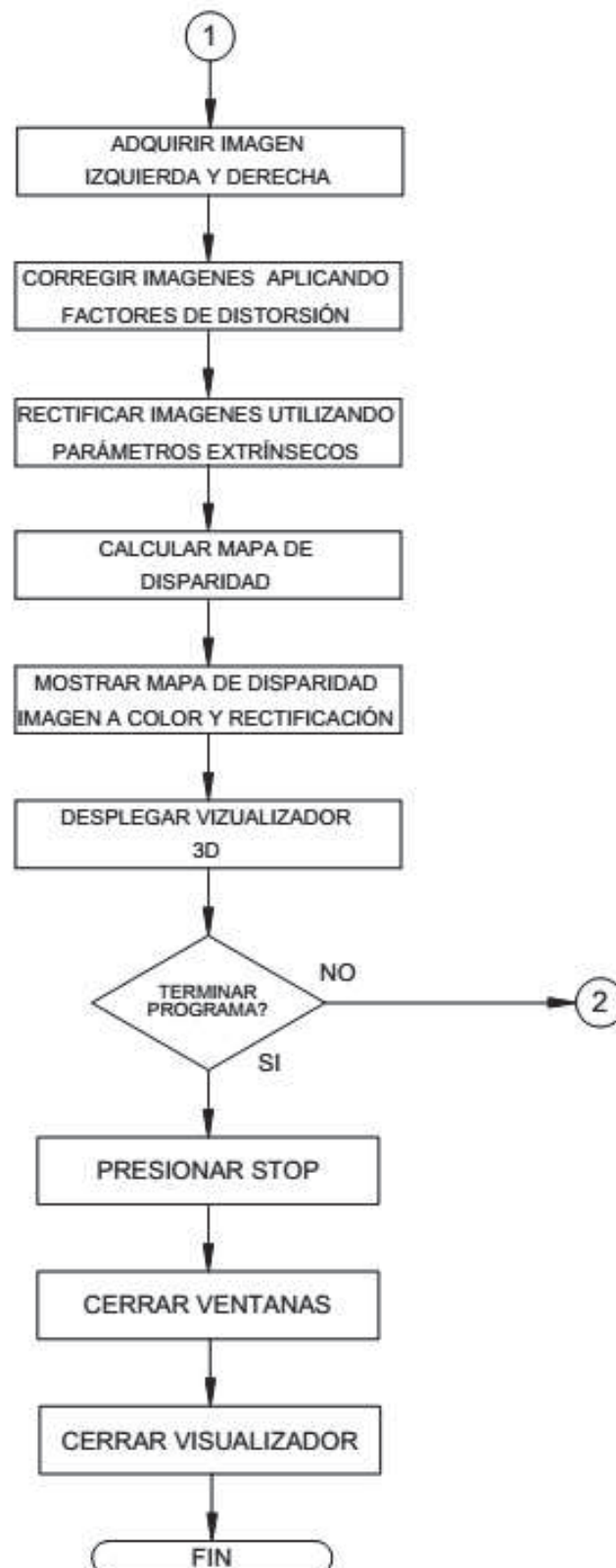


Figura 3.6: Diagrama de flujo del funcionamiento del GUI (Parte 2)

CAPÍTULO 4 PRUEBAS Y RESULTADOS

En este capítulo se muestran las pruebas que se realizaron para comprobar el funcionamiento, la precisión, y el alcance del sistema con los correspondientes resultados de los mismos.

4.1 PRUEBAS DE CALIDAD DE IMÁGENES

Para obtener un mapa de disparidad con suficiente nitidez, es importante que las imágenes usadas en el proceso del mismo sean de una calidad aceptable, de manera que se pueda detectar suficientes esquinas en el proceso de correspondencia.

Antes de usar las cámaras con la que se realizó este trabajo (Logitech HD Pro Webcam C920), se utilizó la cámara Minoru 3D *webcam*. Aunque esta cámara se usa como una cámara *web* convencional en 3D para video-llamadas, también puede ser usada para aplicaciones de visión estéreo de bajo costo como un sensor de rango.



Figura 4.1: Cámara Minoru 3D Webcam.

En las pruebas realizadas con estas cámaras, el mapa de disparidad obtenido con esta cámara no detectaba objetos con facilidad, y aparecían abundantes puntos donde no se pudo calcular el valor de disparidad, esto debido a que las imágenes obtenidas eran borrosas y tenían un alto contenido de ruido. Por estas razones se optó por cambiar la misma por un par de cámaras de mejor resolución, con las

cuales se continuó trabajando durante el resto del proyecto. Se muestra 2 mapas de disparidad obtenidos con ambos tipos de cámaras donde se puede apreciar las diferencias:

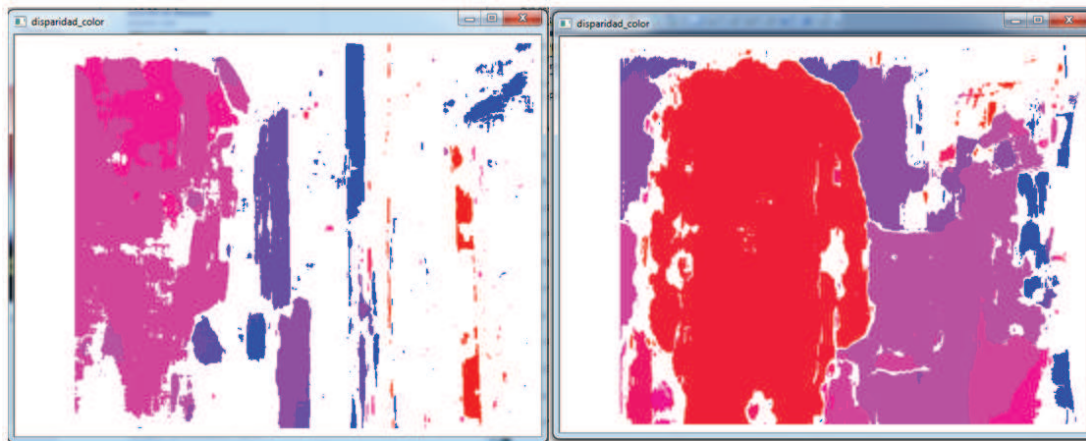


Figura 4.2: Derecha: Mapa de disparidad con Minoru 3D Webcam, Izquierda: Mapa de disparidad con par Logitech HD Pro Webcam C920.

En cámaras de mejor calidad el sistema de lentes es más trabajado con lo cual se obtiene imágenes con colores más reales, menor ruido, mejor enfoque y mayor nitidez, de manera que se puede reproducir la misma escena en diferente tomas. Estas características facilitan el proceso de correspondencia, con lo que se obtiene mejores resultados reflejados en el mapa de disparidad, al tener mayor cantidad de puntos característicos y menor número de falsas correspondencias.

Con las cámaras usadas para este proyecto, se garantiza en lo posible que la reconstrucción 3D va a contar con suficientes puntos, como para que pueda ser apreciada.

4.2 PRUEBAS DE CALIBRACIÓN

El proceso de calibración afecta directamente al proceso de rectificación, al mapa de disparidad y al cálculo de coordenadas x, y y z , de aquí la importancia de que sea realizado correctamente para obtener reconstrucciones 3D aceptables, donde se pueda distinguir los diferentes objetos y las diferentes distancias entre ellos.

Con un proceso de calibración poco exitoso la rectificación del par de imágenes contienen alta distorsión, y la superposición de la escena entre el par de

imágenes se reduce considerablemente, lo que dificulta el proceso de correspondencia dando como resultado un mapa de disparidad con un alto porcentaje de error, o con muchos puntos donde la disparidad no pudo ser calculada.

Con un mapa de disparidad con pocos puntos reconocidos, se obtiene una reconstrucción 3D donde gran parte de las coordenadas no se pueden calcular, debido a que no se tiene su valor de disparidad, dando como resultado muchos espacios vacíos.

Las razones por las que no se puede obtener un resultado positivo, son que no se tengan los suficientes pares de imágenes, que el tablero no sea visto por completo en el par de imágenes, falta de iluminación al momento de la adquisición de imágenes, que no se tenga suficiente variación del tablero entre los pares de imágenes o que se tenga imágenes borrosas.

Se recomienda que al adquirir las imágenes se cambie considerablemente de posición el tablero entre toma y toma, ya que esto se ayuda al algoritmo de calibración a obtener menores errores en los cálculos que realiza.

En el proceso de adquisición de imágenes, se recomienda que el tablero de ajedrez se encuentre estático al momento de adquirir el par imágenes, para obtener imágenes claras, sin rastros de movimiento y que sean lo más próximo a imágenes tomadas al mismo tiempo.

En este subcapítulo se muestran los resultados obtenidos del proceso de calibración, que garantizan una correcta rectificación, cálculo del mapa de disparidad y reconstrucción 3D.

Para el proceso de calibración se usó 15 pares de imágenes de un tablero de ajedrez de 9x6 esquinas, y se obtuvo los resultados presentados en las imágenes siguientes:

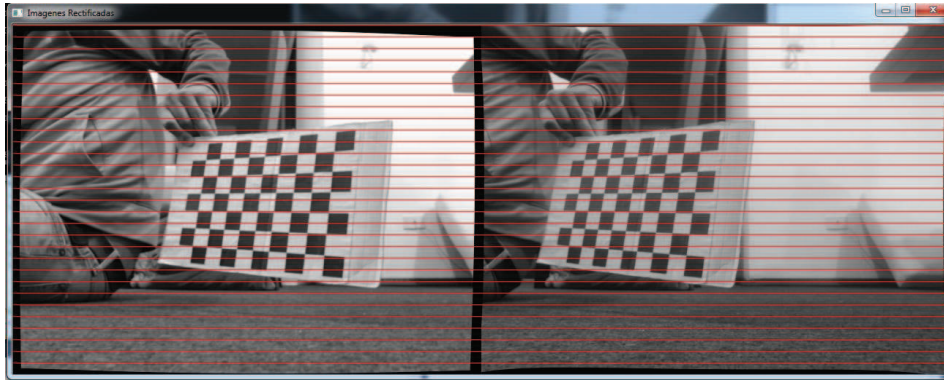


Figura 4.3: Rectificación de una escena.

Se comprobó que el proceso de calibración es correcto, verificando que los mismos puntos en ambas imágenes tienen la misma coordenada y , con lo que las líneas epipolares se encuentran ubicadas horizontalmente con los epipolos en el infinito.

Cuando el proceso de calibración no fue correcto, las imágenes rectificadas presentan alta distorsión, y el proceso debe de ser repetido antes de continuar con el cálculo del mapa de disparidad. Se presentan resultados donde el proceso de calibración no fue correcto.

En la Figura 4.4 se muestra la rectificación de una calibración realizada con solo 3 pares de imágenes, por lo que el algoritmo de calibración tiene muy poca información y el cálculo de los parámetros tienen un alto error, que se ve reflejado en una mayor distorsión.

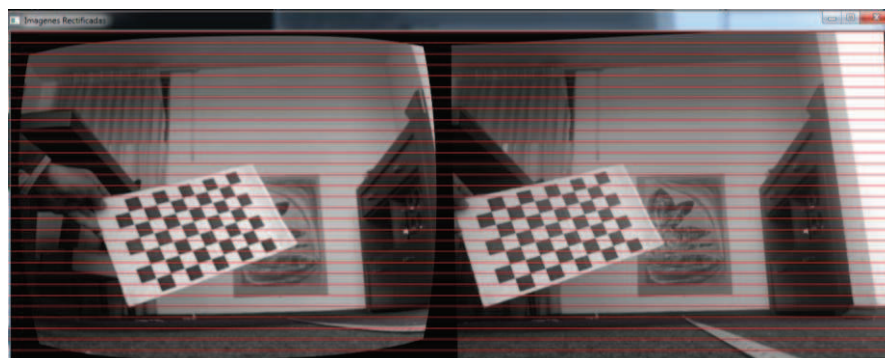


Figura 4.4: Rectificación con 3 pares de imágenes.

En la Figura 4.5 se muestran la rectificación de una calibración con falta de iluminación, por lo que las esquinas no fueron detectadas en todos los pares de imágenes y las imágenes rectificadas presentan alta distorsión

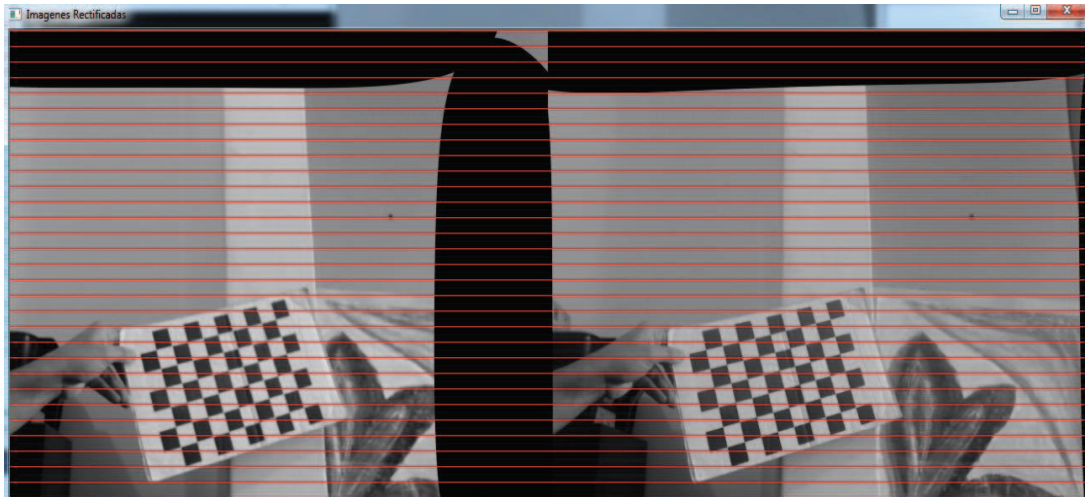


Figura 4.5: Rectificación con falta de iluminación.

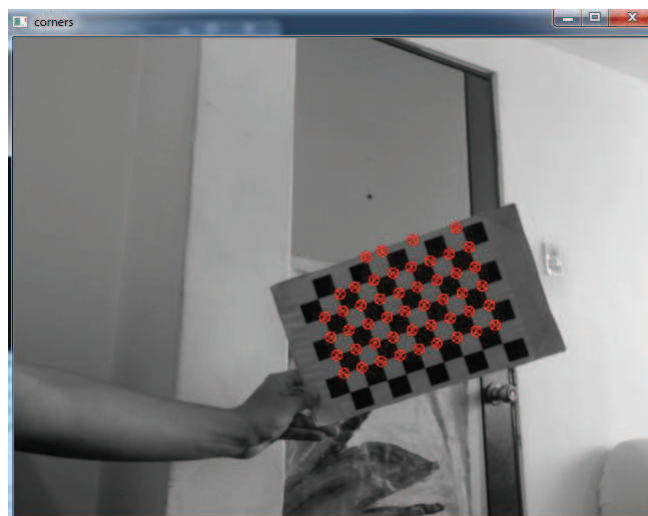


Figura 4.6: Esquinas no detectadas por falta de iluminación.

En la Figura 4.7 se muestran la rectificación de una calibración en donde no se tiene suficiente variación del tablero entre los pares de imágenes, este hace que la rectificación sea altamente distorsionada.

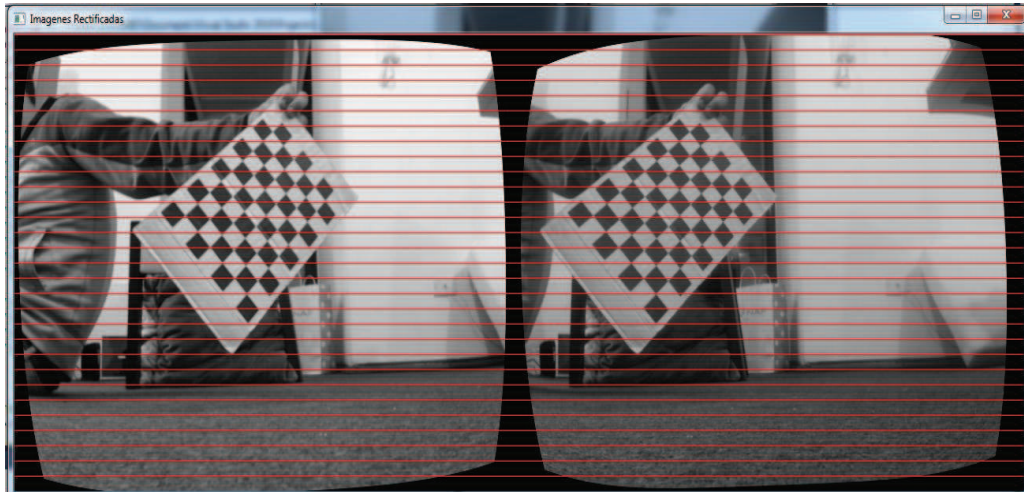


Figura 4.7: Rectificación con poca variación de movimiento del tablero entre tomas.

4.3 PRUEBAS DEL MAPA DE DISPARIDAD

El mapa de disparidad es la base de la reconstrucción 3D, de aquí se obtiene el dato de disparidad para proceder al cálculo de distancias, por lo que es de esencial importancia obtener un mapa de disparidad con abundantes píxeles, donde la disparidad fue calculada.

El objetivo de esta prueba es obtener los valores de los parámetros que controlan la generación del mapa de disparidad, para esto se probó con varios valores a manera de prueba y error y se eligió los valores que mejores resultados expulsaron para distancias de 1 a 4 m.

Se realizó un programa en C++, que nos ayude a obtener los valores de los parámetros utilizados para el cálculo del mapa de disparidad. Para agilizar el proceso, el programa calcula el mapa de disparidad de un par de imágenes almacenadas en el disco duro, en lugar de adquirir imágenes en tiempo real.

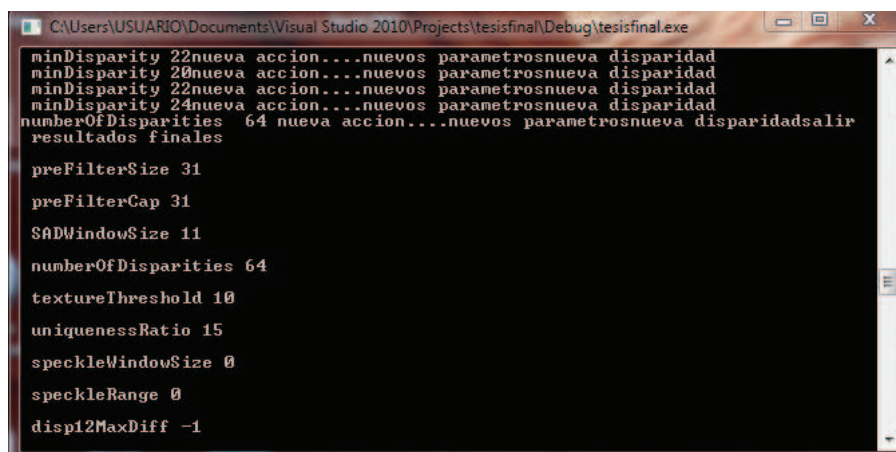
Para estas pruebas se muestra un mapa de disparidad normalizado, en un lugar del mapa de disparidad que se obtiene directamente luego del procedimiento descrito en 2.6. Se utiliza un mapa normalizado, ya que el mapa de disparidad a

escala de grises sin normalizar no cuenta con suficiente contraste, como para poder diferenciar las diferentes distancias a simple vista.

El proceso de normalización consiste en buscar el valor mínimo y máximo en píxeles dentro de la imagen, y cambiarlos por dos nuevos límites mínimo y máximo predeterminados. Luego de que se ha realizado este proceso, el resto de valores son escalados a valores intermedios calculados en función de los límites mínimo y máximo.

Se normalizó el mapa de disparidad a una imagen a escala de grises a 8 bits con los nuevos valores mínimo y máximo de 0 y 255 respectivamente; donde 0 representa el negro y 255 el blanco.

En este programa por medio de teclado se cambia cada uno de los parámetros en tiempo real, de manera de que cada vez que se realiza un cambio en algún parámetro se actualiza el mapa de disparidad inmediatamente, y se muestra el nuevo valor en la pantalla, adicionalmente al final del programa se muestra los valores finales que se usaron para el último cálculo del mapa de disparidad.



```

C:\Users\USUARIO\Documents\Visual Studio 2010\Projects\tesisfinal\Debug\tesisfinal.exe
minDisparity 22nueva accion...nuevos parametrosnueva disparidad
minDisparity 20nueva accion...nuevos parametrosnueva disparidad
minDisparity 22nueva accion...nuevos parametrosnueva disparidad
minDisparity 24nueva accion...nuevos parametrosnueva disparidad
numberOfDisparities 64 nueva accion...nuevos parametrosnueva disparidadsalir
resultados finales

preFilterSize 31
preFilterCap 31
SADWindowSize 11
numberOfDisparities 64
textureThreshold 10
uniquenessRatio 15
speckleWindowSize 0
speckleRange 0
disp12MaxDiff -1

```

Figura 4.8: Pantalla del programa usado para calcular los parámetros del mapa de disparidad.

Se eligió los valores que dieron menor cantidad de píxeles negros (píxeles donde la disparidad no pudo ser calculada), y donde se podía distinguir la forma de objetos dentro del mapa, como personas, mesas y estantes de una manera completa.

Se tiene algunos parámetros que controlan la generación del mapa de disparidad; control de pre-filtrado, post-filtrado y métodos de búsqueda de correspondencia; los valores más importantes son:

- Tamaño de la ventana SAD.
- Mínimo de búsqueda de correspondencia (MINBC).
- Máximo de búsqueda de correspondencia (MAXBC).

Los valores de estos parámetros dependen de los objetivos que se quiera; de la distancia de percepción requerida, de la precisión para la búsqueda de correspondencias y de la velocidad de procesamiento; por lo que deben de ser calibrados correctamente para obtener la máxima velocidad de procesamiento cumpliendo con todos los requerimientos de búsqueda propuestos.

El tamaño de la ventana SAD afecta de manera determinante la forma en que las correspondencias son calculadas, el valor de la ventana afecta la nitidez y el número de detalles que se pueden calcular en el mapa de disparidad.

Ventanas SAD muy grandes provoca un mapa de disparidad suave, donde no se puede diferenciar claramente la forma del objeto que se encuentra en la imagen, al mismo tiempo que se pierde precisión en los cálculos de disparidad.

Ventanas muy pequeñas permiten apreciar mayor cantidad de detalles, pero también aumenta la posibilidad de encontrar falsas correspondencias, lo que conlleva a tener un mapa de disparidad con gran número de huecos dentro de un objeto identificable.

De aquí la importancia de elegir una ventana SAD de un tamaño adecuado, donde se tenga los mayores beneficios, estos son: tener suficientes detalles, pocas falsas correspondencias, y una precisión aceptable en el mapa de disparidad.

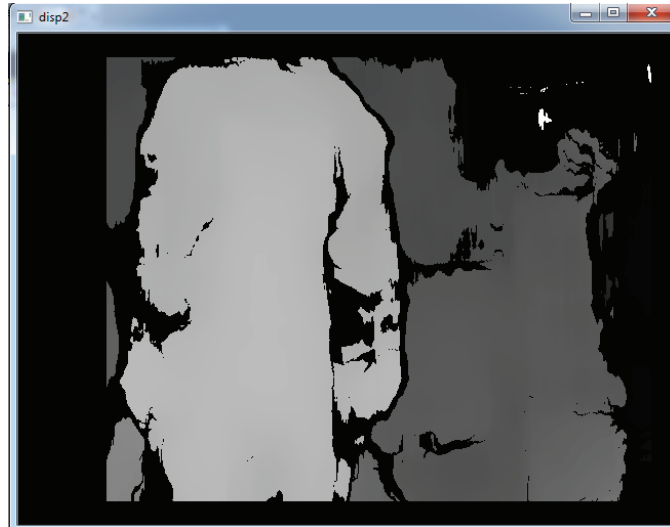


Figura 4.9: Mapa de disparidad con una ventana SAD de 47.

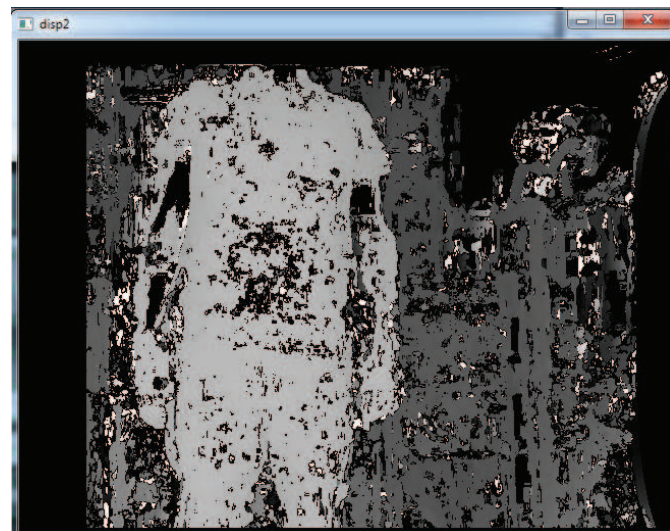


Figura 4.10: Mapa de disparidad con una ventana SAD de 7.

El mínimo y máximo de búsqueda de correspondencia controlan los límites de búsqueda. El control de estos dos valores, deben ser regulados dependiendo del rango de distancia que se quiere percibir con el sistema de cámaras.

Para distancias más cercanas se obtiene valores de disparidad más grandes, por lo que el valor máximo debe aumentar para poder distinguir objetos más cercanos, de manera análoga para distancias lejanas el sistema debe de tener la capacidad de detectar disparidades más pequeñas, por lo que el mínimo de búsqueda debería disminuir si se quiere distinguir distancias mayores.

Cuando no se encuentra la correspondencia en los límites establecidos ese pixel aparece de color negro, y no se conoce el valor de disparidad del mismo por lo que no se podrá realizar la reconstrucción en ese punto, de aquí la importancia de elegir límites de búsqueda que contengan la gran mayoría de puntos de interés, sin aumentar el tiempo de procesamiento.

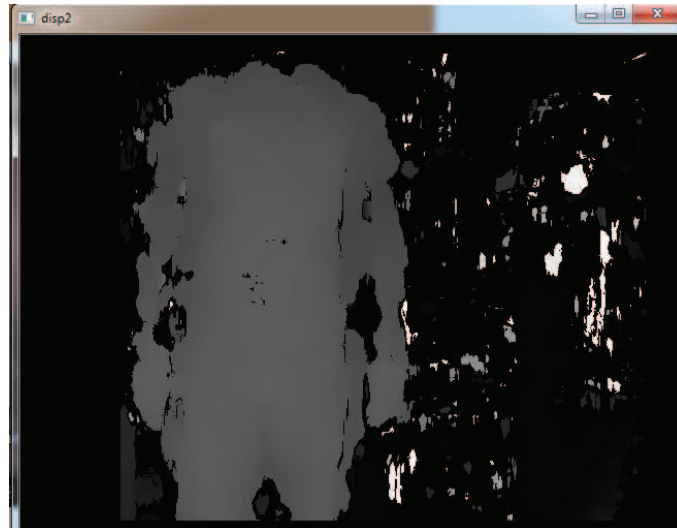


Figura 4.11: Mapa de disparidad con un mínimo de búsqueda de correspondencia de 24.

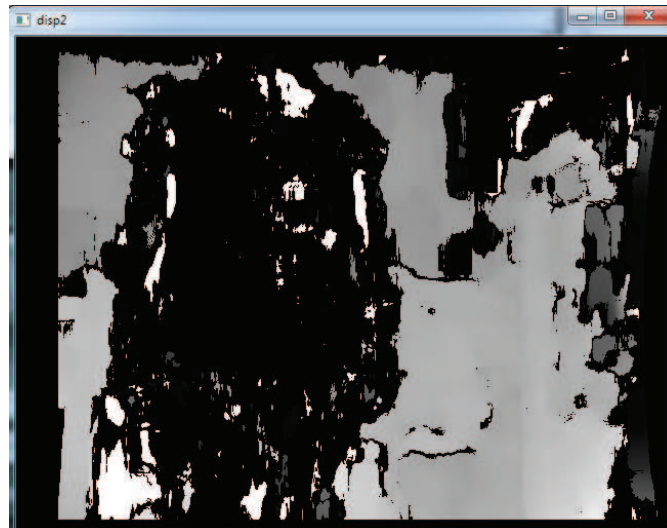


Figura 4.12: Mapa de disparidad con un máximo de búsqueda de correspondencia de 32.

Luego de algunas pruebas se obtuvo los siguientes valores de calibración:

- Tamaño de la ventana SAD = 21

- Mínimo de búsqueda de correspondencia = 0
- Máximo de búsqueda de correspondencia = 64

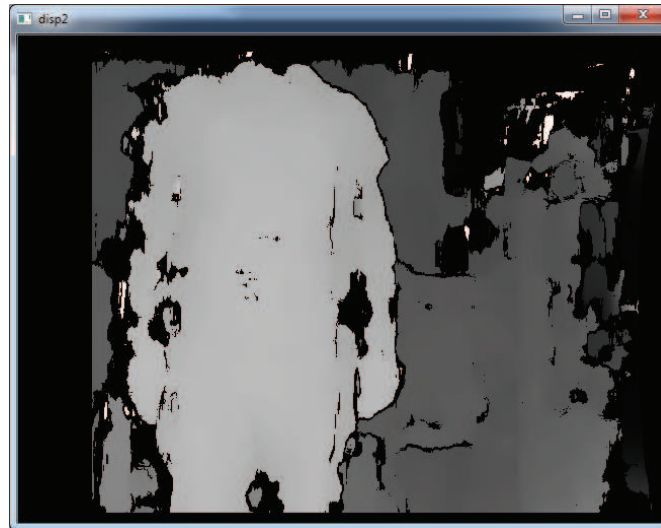


Figura 4.13: Mapa de disparidad con los parámetros utilizados para este proyecto.

4.4 ERRORES DE DISTANCIA DEL SISTEMA ESTÉREO

En este subcapítulo, se presentan los resultados de las pruebas de exactitud en la medición de distancias con el uso del sistema estéreo. Se realiza esta prueba con el objetivo de conocer la precisión del sistema, a diferentes distancias desde la cámara y a diferentes ubicaciones del mapa de disparidad.

Para esta prueba se realizó un programa en C++ que muestra el valor de distancia de un pixel previamente establecido dentro del mapa de disparidad.

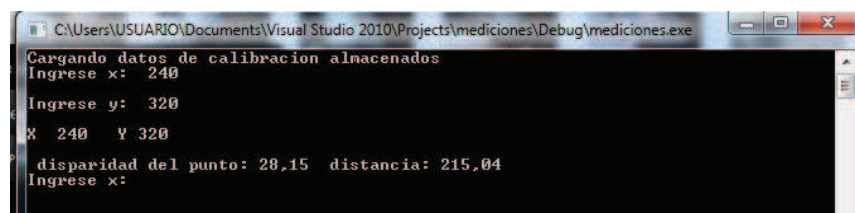


Figura 4.14: Ingreso de coordenadas dentro del mapa de disparidad.

Para controlar la ubicación del pixel del cual se muestra el valor de distancia, por teclado se ingresa la ubicación deseada en coordenadas x, y , de acuerdo a la

resolución del mapa de disparidad (640px, 480px) y de acuerdo al sistema de referencia que se muestra en la Figura 1.3.

Para tener una referencia visual del valor de distancia que se está calculando en el mapa de disparidad a colores descrito en 2.6, y que se muestra en la Figura 2.15, se ubica un cuadrado negro representado la ubicación ingresada. Este cuadrado negro es de 10x10 píxeles donde el centro del mismo es la ubicación ingresada anteriormente, y el resto solo se añade por cuestiones de visualización.

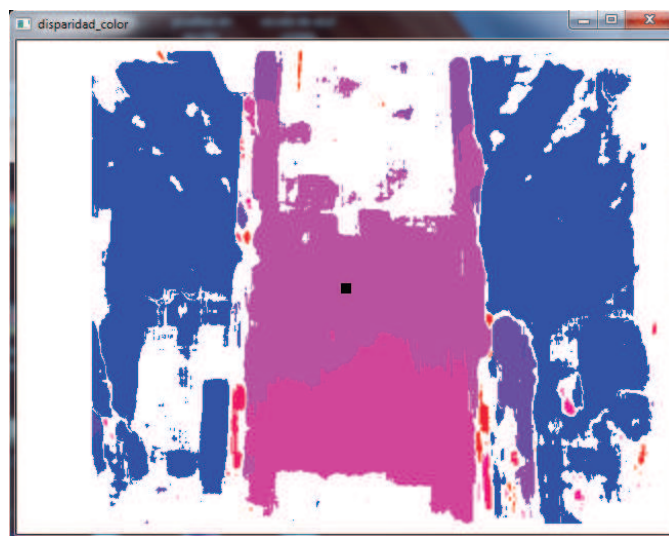


Figura 4.15: Ubicación del pixel ingresado por teclado en el mapa de disparidad.

El ingreso de coordenadas por teclado, y su representación dentro del mapa de disparidad se repite consecutivamente hasta cuando se finalice el programa, para permitir calcular diferentes valores de distancia en la misma ejecución del programa.

Para esta prueba se utilizó una superficie plana de cartón, en la cual se pegó una imagen con abundantes puntos característicos, de manera que sea fácilmente reconocida por el sistema de visión en su totalidad.

Esta superficie plana se colocó sobre una pared, de manera de que se encuentre perpendicular al piso y todos sus puntos estén a la misma distancia desde la cámara. La cámara se colocó a diferentes distancias fijas desde la pared, y se procedió a tomar datos de distancias con el programa en C++ antes mencionado.

Se tomó datos para diferentes coordenadas a la misma distancia, estas coordenadas fueron elegidas, de manera, de cubrir la mayor área posible de la superficie. Ya que imágenes adquiridas de la misma escena en tiempos diferentes nunca son iguales, por lo que el valor de disparidad cambia ligeramente, se adquirió tres mediciones de la misma coordenada.

Para obtener la distancia real, se realizó marcas en el piso a las distancias de los valores de interés medidas por medio de un flexómetro colocado perpendicularmente con la cámara izquierda. Con estos valores medidos y los calculados por el sistema de visión, se determinó los errores absolutos y relativos para diferentes medidas de acuerdo a las ecuaciones 4.1. y 4.2 , se presenta los resultados obtenidos en el anexo B. [40]

$$E_{absoluto} = |x_{promedio} - x_{medido}| \quad (4.1)$$

$$\%E_{relativos} = \frac{E_{absoluto}}{x_{real}} * 100\% \quad (4.2)$$

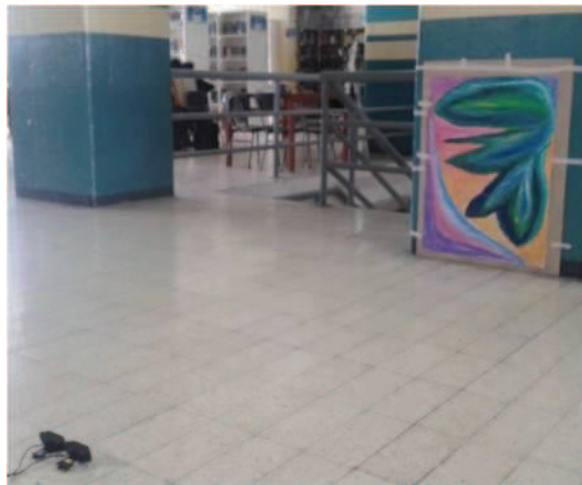


Figura 4.16: Sistema utilizado para medidas reales

De los resultados obtenidos se concluye que el error aumenta a medida que se aleja del centro de la imagen, porque la distorsión introducida por el proceso de rectificación aumenta para estos puntos.

Se calculó el promedio de los errores relativos para cada distancia de las tablas mostradas en el anexo B, y se realizó una gráfica con estos valores en función de la distancia real a la que se realizó la prueba, dichos resultados se presentan en la Figura 4.17.

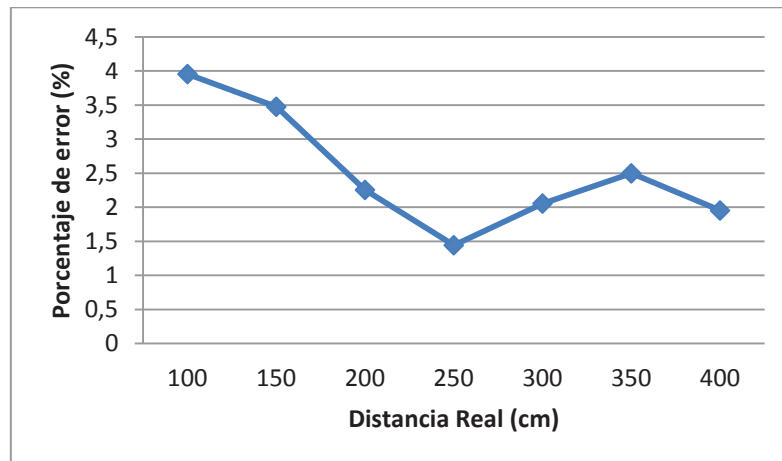


Figura 4.17: Grafico % Error vs Distancia real.

Acorde a la Figura 4.17, se concluye que el error en el rango de 100 a 400 cm tiende a disminuir en la mayoría de su rango, a medida que se aumenta la distancia medida, y la máxima diferencia entre porcentajes de error es de 2,5108%, por lo que se pueda afirmar que el error se mantiene constante en gran proporción.

El error promedio a los diferentes valores de distancia es de 2,5187 %, el cual se encuentra en un rango aceptable, y puede ser considerado para posibles aplicaciones que estén acordes que este nivel de error.

En el proceso de rectificación se calculó la rotación que debe tener cada imagen, para obtener un arreglo con las líneas epipolares en el infinito, este procedimiento introduce distorsión en las imágenes rectificadas, lo que aumenta el error en el cálculo de disparidad y afecta directamente a los valores de distancia.

Los procesos de rectificación siempre introducen distorsión, que son más notables a medida que se alejan del centro de la imagen. Lo que se trata es de minimizar

este efecto por medio de un proceso de calibración con el menor error posible siguiendo los procedimientos descritos en 4.2.

4.5 PRUEBAS DE RECONSTRUCCIÓN 3D

Las reconstrucciones 3D son el resultado final de todos los procesos y pruebas antes mencionadas. Para obtener reconstrucciones 3D apreciables, se trabajó mejorando el proceso de calibración, correspondencia, generación del mapa de disparidad y la forma en que se manejan estos datos en el momento de formar la nube de puntos.

Se realizó varias pruebas en donde se concluyó que la calidad de la reconstrucción es dependiente del mapa de disparidad. Si se obtuvo un mapa de disparidad con pocos puntos reconocibles la reconstrucción es muy pobre, hasta llegar al punto en donde no se diferencia los objetos dentro de la reconstrucción.

Se presentan algunos de los resultados que se obtuvieron desde diferentes posiciones (frontal, lateral y superior), en estos resultados se aprecia diferentes objetos a diferentes distancias. Adicionalmente, se muestra una imagen a color del resultado de la reconstrucción, como una nube de puntos para una mejor comprensión de los resultados.



Figura 4.18: Prueba 1 de reconstrucción 3D. Der: Imagen a color. Izq: Vista frontal.

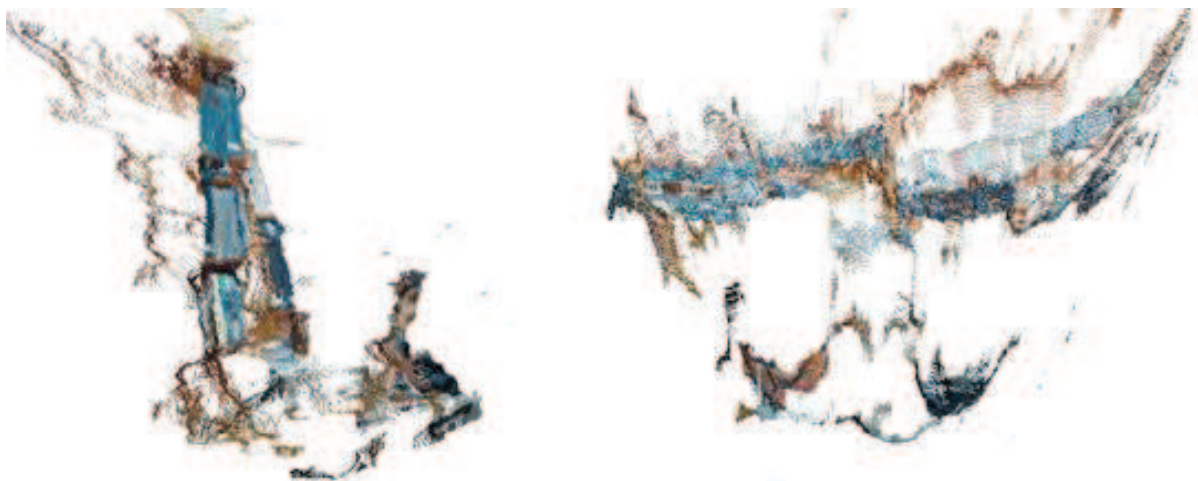


Figura 4.19: Prueba 1 de reconstrucción 3D. Der: Vista lateral. Izq: Vista superior.



Figura 4.20: Prueba 2 de reconstrucción 3D. Der: Imagen a color. Izq: Vista frontal.



Figura 4.21: Prueba 2 de reconstrucción 3D. Der: Vista lateral. Izq: Vista superior.



Figura 4.22: Prueba 3 de reconstrucción 3D. Der: Imagen a color. Izq: Vista frontal.



Figura 4.23: Prueba 3 de reconstrucción 3D. Der: Vista lateral. Izq: Vista superior.

4.6 PRUEBAS DE RECONSTRUCCIÓN 3D DESDE 2 VISTAS

Al realizar una reconstrucción 3D desde un solo punto de vista, se obtiene información de distancia de lo que las cámaras captan en ese momento, por lo que la reconstrucción se ve limitada a los puntos paralelos del eje x de la cámara, debido a que el resto de puntos están fuera del alcance de la cámara.

Para aumentar el alcance de la reconstrucción, y obtener una reconstrucción con mayor número de puntos, se realizó dos reconstrucciones desde un par de ubicaciones separadas y rotadas la una con respecto a la otra, a una distancia y ángulo conocido.

Con los valores de distancia se calcula la matriz de rotación y traslación de una posición respecto a la otra, y se desplaza una reconstrucción en el sistema de coordenadas de la otra por medio de las matrices de rotación y traslación calculadas de manera que se pueda apreciar en la reconstrucción final lo más cercano posible a una reconstrucción adquirida en una sola toma.

Para asegurarse de tener valores reales de distancia y ángulo, y mantener las mismas matrices en el tiempo se construyó una estructura con dos barras de aluminio unidas físicamente a 90 grados con agujeros para sujetar las cámaras a posiciones fijas conocidas.



Figura 4.24: Sistema utilizado para reconstruir dos vistas.

Se calculó las matrices de rotación de acuerdo a 1.3.5.1 y 1.3.5.2, tomando en consideración que el eje y es constante, por lo que el sistema se reduce a una rotación y traslación en 2 dimensiones, con un ángulo de rotación de 90 grados y la traslación en función de la diferencia de ubicaciones en centímetros entre los centros de coordenadas de los sistemas de referencia.

Las matrices de rotación se calcularon para trasladar la reconstrucción del sistema de referencia P_o al sistema P_c , de acuerdo a los ejes mostrados en la Figura 4.25.

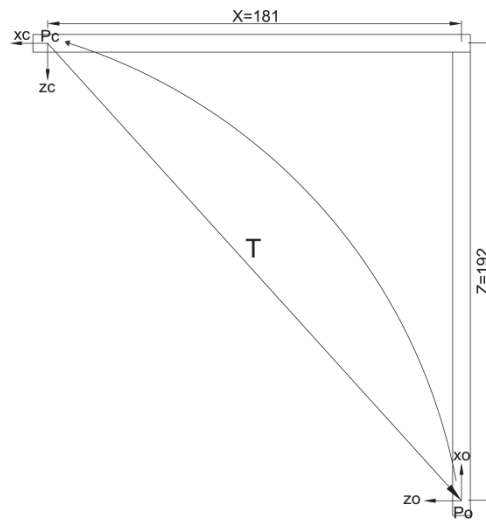


Figura 4.25: Sistemas de referencia Po y Pc.

De acuerdo a la Figura 4.25 calculamos la matriz R y T que satisfacen la ecuación 1.16.

$$R = \begin{bmatrix} \cos(\theta) & \sin(\theta) \\ -\sin(\theta) & \cos(\theta) \end{bmatrix} = \begin{bmatrix} \cos(90) & \sin(90) \\ -\sin(90) & \cos(90) \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix} \quad (4.3)$$

$$T = \begin{bmatrix} -x \\ z \end{bmatrix} = \begin{bmatrix} -182 \\ 196 \end{bmatrix} \quad (4.4)$$

$$P_c = R * P_o + T \quad (4.5)$$

Se realizó pruebas y se obtuvo los resultados mostrados en las siguientes Figuras:



Figura 4.26: Prueba 1 de reconstrucción 3D desde dos vistas.

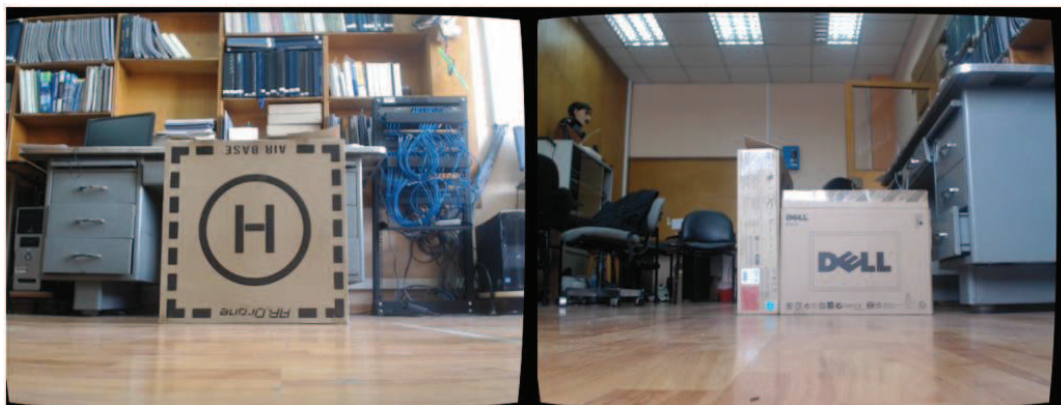


Figura 4.27: Prueba1, Der: Imagen adquirida desde Po. Izq: Imagen adquirida desde Pc.



Figura 4.28: Prueba 2 de reconstrucción 3D desde dos vistas.



Figura 4.29: Prueba 2. Der: Imagen adquirida desde Po. Izq: Imagen adquirida desde Pc.



Figura 4.30: Prueba 3 de reconstrucción 3d desde dos vistas



Figura 4.31: Prueba 3. Der: Imagen adquirida desde Po. Izq: Imagen adquirida desde Pc

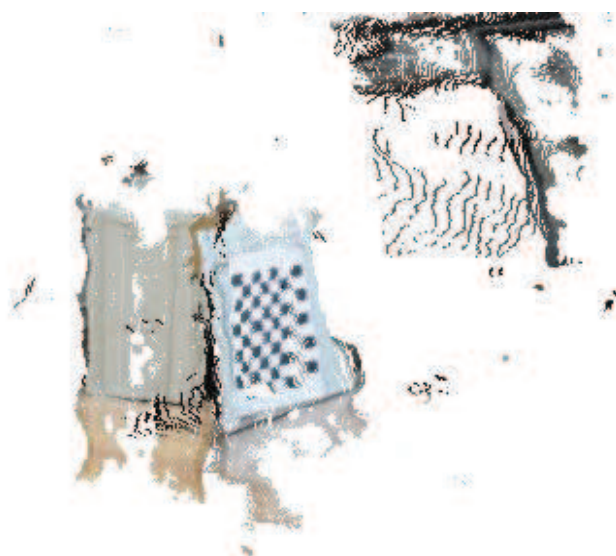


Figura 4.32: Prueba 4 de reconstrucción 3d desde dos vistas.



Figura 4.33: Prueba 4. Der: Imagen adquirida desde Po. Izq: Imagen adquirida desde Pc

Esta prueba fue realizada por medio de los cálculos de rotación y traslación entre los dos sistemas de coordenadas. Los resultados obtenidos con este método no son del todo satisfactorios, la reconstrucción no se aprecia como una sola toma, porque se tiene errores en el sistema de medición y en los cálculos en la matriz de rotación y traslación.

Para aumentar los resultados se debe usar el proceso de registración, el cual consiste en calcular puntos característicos en ambas nubes de puntos los cuales permitan calcular la matriz de rotación y traslación que unen ambas nubes de puntos.

El proceso de registración calcula las matrices mencionadas de manera de que se tenga la menor distorsión posible, y que se conserve la mayor cantidad de información por medio de una aproximación inicial.

En sistemas SLAM la aproximación inicial es obtenida por medio de odometría del robot, por lo que el sistema estéreo de visión ayuda a disminuir este error al tiempo que representa el entorno de una manera 3D. Se realizó esta prueba de manera de que sea de utilidad para estudios futuros de reconstrucción de varias tomas, y sea un aporte para estudios futuros en el campo de investigación de SLAM, pero no se incluye dentro del GUI presentado en este proyecto.

CAPÍTULO 5 CONCLUSIONES Y RECOMENDACIONES

5.1 CONCLUSIONES

- El uso de las librerías *OpenCV* y *PCL*, que simplifican el tratamiento de datos en el campo de la visión artificial, en conjunto con las herramientas que proporciona *Microsoft Visual Studio* han permitido el desarrollo de un software confiable con una interfaz gráfica para el manejo del usuario.
- El *hardware* que se utiliza dentro de la visión artificial toma un rol importante, los resultados varían dependiendo de la calidad del lente de las cámaras utilizadas. Con imágenes claras el reconocimiento de puntos característicos se facilita, por lo que, los resultados en los algoritmos de visión mejoran.
- La iluminación usada en el momento de adquirir datos con cámaras, interviene en gran medida en la información que se obtiene de las mismas. Para obtener resultados satisfactorios es importante tener un ambiente con iluminación constante, el sistema de reconstrucción se ve seriamente afectado por excesivos reflejos de luz, y por ambientes poco iluminados.
- El proceso de calibración simplifica el proceso de triangulación, por lo que influye directamente en los resultados obtenidos. De aquí la importancia de que sea realizado cuidadosamente, garantizando que se obtenga el menor error en los cálculos de los parámetros intrínsecos y extrínsecos del sistema de estereovisión, dando como resultado imágenes rectificadas con baja distorsión.
- El algoritmo de correspondencia, que permite el cálculo del mapa de disparidad, necesita que los objetos en la escena cuenten con puntos característicos reconocibles, caso contrario la disparidad no puede ser calculada en la mayoría de puntos, y se obtiene una reconstrucción de baja calidad.

- Si se conoce el rango de distancias de interés a las que se quiere que funcione el sistema estéreo, se puede limitar los rangos de búsqueda de disparidad, excluyendo valores no deseados, simplificando el proceso y reduciendo los tiempos de procesamiento para aplicaciones en tiempo real.
- Es importante utilizar una resolución en píxeles adecuada según la aplicación de interés. La resolución de las imágenes influyen directamente en la calidad de la reconstrucciones 3D, al aumentar la resolución se obtiene mayor número de puntos característicos reconocibles. Aumentar la resolución tiene una consecuencia negativa en el incremento del tiempo de procesamiento, por lo que se debe tener un balance entre calidad y tiempo.
- Los algoritmos de correspondencia evaluados para el desarrollo de este proyecto fueron los proporcionados por *OpenCV*, *Block Matching* (BM) y *Semi – Global Block Matching* (SGBM). De estos dos se seleccionó el algoritmo de BM por ser el más rápido y no presentar diferencias considerables en la calidad del mapa de disparidad con respecto a SGBM.
- Los errores en la medición de profundidad son casi constantes a diferentes distancias, sin embargo se incrementa a medida que la distorsión aumenta al alejarse del centro de la imagen. Tener una calibración adecuada disminuye los errores, y el efecto de distorsión en los extremos de la imagen, pero siempre va a estar presente y se lo debe considerar todo el tiempo.
- Los proyectos MFC creados desde *Microsoft Visual Studio* permiten tener un control de la ejecución de los algoritmos de estero visión, y la presentación de sus resultados desde una interfaz gráfica. Este tipo de proyectos, permite tener compatibilidad con las librerías utilizadas sin alterar el tiempo de procesamiento.
- Las ventajas de realizar reconstrucciones 3D con cámaras a modo de estéreo visión, con respecto a sistemas similares con el uso de sensores

activos de rango (infrarrojos, laser), son su menor costo, se tiene mayor cantidad de información en una sola toma, y se puede realizar una representación a colores sin el uso de ningún dispositivo adicional. Sin embargo, tiene la desventaja de su dependencia a la iluminación del entorno y es necesario tener muchos característicos dentro de la escena para poder tener un buen resultado.

- Si se une varios conjuntos de nubes de puntos se puede tener una representación global de un entorno, para lograr este propósito se debe tener un sistema de localización, y aplicar algoritmos de registración que permitan calcular las matrices de rotación y traslación, que disminuyen las distancias entre puntos correspondientes.

5.2 RECOMENDACIONES

- Para los procesos de calibración y de correspondencia es importante evitar excesivos reflejos de luz. Estos generan que el ambiente no tenga una iluminación constante, por lo que se tiene problemas en el cálculo de los parámetros intrínsecos y extrínsecos, afectando al resultado del mapa de disparidad y disminuyendo considerablemente la calidad de los resultados de la reconstrucción 3D.
- Para mantener la distancia focal de la cámara constante se recomienda no usar cámaras con autoenfoco, o desactivar esta característica en caso de usar cámaras de este tipo. Esto es debido a que este parámetro interviene directamente en el cálculo de la profundidad, y al ajustarse el enfoque automáticamente este parámetro se ve afectado, por lo que, se obtiene mediciones erróneas.
- Es importante construir un sistema en donde se pueda evitar movimientos de cada cámara, esto causa que los parámetros de calibración cambien y el proceso de rectificación no sea completado con éxito. Cuando el sistema

no se encuentra calibrado, no se puede determinar el valor de disparidad, y en consecuencia no se tiene información de profundidad del entorno.

- El proceso de calibración no siempre se completa exitosamente, y esto no permite obtener un resultado óptimo en el cálculo de correspondencia, por lo que es importante que se compruebe sus resultados antes de continuar con los algoritmos posteriores al mismo. Para mejorar el proceso de calibración es importante que se tome el suficiente número de imágenes, variando la posición del tablero, y que se lo realice con una iluminación adecuada.
- Para realizar reconstrucciones con varias nubes de puntos unidas entre sí, se debe incorporar un sistema de localización que proporcione las matrices de rotación y traslación que unen ambas vistas. A su vez este sistema de localización, se debe complementar con algoritmos de correspondencia 3D que disminuyen el error en las matrices, y permita que las áreas se alineen perfectamente.
- Se recomienda usar la cámara *Kinect* desarrollada por *Microsoft*, para reconstrucciones en 3D para interiores, de acuerdo a su coste y la calidad de resultados. Para exteriores, lo más recomendable es utilizar sensores de rango (laser), los cuales son inmunes a los cambios de iluminación, pero en la actualidad la mejor alternativa es la combinación de sensores de rango y sistemas de estéreo visión, esto permite incorporar las ventajas de los sensores de rango con las ventajas de sistema de estéreo visión, aumentando la calidad y resolución de las reconstrucciones realizadas.

REFERENCIAS BIBLIOGRAFICAS

- [1] Introductory Techniques for 3-D Computer Vision – Emanuele Trucco/Alessandro Verri
- [2] http://homepages.inf.ed.ac.uk/rbf/CVonline/LOCAL_COPIES/CANTZLER2/range.html. Evolving, Distributed, Non-Proprietary, On-Line Compendium of Computer Vision- The University of Edinburgh ,Bob Fisher
- [3] Introduction to Video and Image Processing - T.B. Moeslund
- [4] <http://www2.cmp.uea.ac.uk/Research/compvis/ColourIntro/ColourIntro.html>. The colour group -University of East Anglia
- [5] http://www.mathworks.com/help/matlab/creating_plots/image-types.html
- [6] <http://processing.org/reference/environment/>. The Processing Foundation
- [7] Medición de distancias por medio de procesamiento de imágenes y triangulación haciendo uso de cámaras de video. – Emmanuel Jiménez.
- [8] <http://www.epixea.com/research/multi-view-coding-thesisse8.html> Acquisition, Compression and Rendering of depth and texture for multi-view video. - Yannick Morvan.
- [9] Learning Opencv – Gary Bradski y Adrian Kaehler
- [10] http://homepages.inf.ed.ac.uk/rbf/CVonline/LOCAL_COPIES/OWENS/LECT9/nod-e2.html. Evolving, Distributed, Non-Proprietary, On-Line Compendium of Computer Vision- The University of Edinburgh ,Bob Fisher
- [11] Correcting Lens Distortions in Digital Photographs - Wolfgang Hugemann
- [12] <http://mathworld.wolfram.com/RotationMatrix.html>
- [13] Mathematical Methods for physicists – George Arfken.

- [14] Camera Calibration - Juan Andrade Cetto
- [15] Machine Vision - Ramesh Jain, Rangachar Kasturi, Brian G. Schunck
- [16] An Efficient Projective Rectification for Trinocular Stereo Vision Qing Zhu, Dan Luo
- [17] Rectification of images for binocular and trinocular stereovision. – Nicholas Ayache and Charles Hasen.
- [18] A Scalable Video Rate Camera Interface - Jon A. Webb, Thomas Warfel, Sing Bing Kang.
- [19] Theory and practice of projective rectification. – Richard I. Hartley,
- [20] A study of depth estimation and depth based applications – Tran Anh Tu
- [21] Small Vision Systems: Hardware and Implementation - Kurt Konolige
- [22] <http://opencv.org/about.html>
- [23] <http://pointclouds.org/about/>
- [24] <http://msdn.microsoft.com/es-es/library/ba1z7822.aspx>
- [25]
http://docs.opencv.org/2.4.3/doc/tutorials/calib3d/camera_calibration/camera_calibration.html
- [26] <http://homepages.inf.ed.ac.uk/rbf/HIPR2/adpthrsh.htm>
- [27]
<http://www.robots.ox.ac.uk/~vgg/presentations/bmvc97/criminispaper/node2.html>
- [28]
http://docs.opencv.org/2.4.5/modules/calib3d/doc/camera_calibration_and_3d_reconstruction.html#findhomography
- [29]
http://docs.opencv.org/2.4.6/doc/tutorials/calib3d/camera_calibration/camera_calibration.html#cameracalibrationopencv

[30]

http://docs.opencv.org/2.4.7/modules/calib3d/doc/camera_calibration_and_3d_reconstruction.html#stereocalibrate

[31] <http://www.bituh.com/2012/04/19/11aii-explain-the-characteristics-of-the-graphical-user-interface-8/>

[32] PROSISE, Jeff. *Programming Windows with MFC*, 2 ed., Microsoft Press, Washington, 1999.

[32] <http://msdn.microsoft.com/en-us/library/d06h2x6e.aspx>

[33]

<http://wiki.opencv.org.cn/index.php/CvvlImage%E7%B1%BB%E5%8F%82%E8%80%83%E6%89%8B%E5%86%8C>

[34] <http://pointclouds.org/documentation/overview/visualization.php>

[35] <http://www.juanhidalgoreina.com/2011/11/10-pasos-para-iniciar-el-diseno-de-un-interfaz/>

[36] 3D Cameras: 3D Computer Vision of Wide Scope. – Stefan May, Kai Pervozelz y Hartmut Surmann. <http://cdn.intechopen.com/pdfs-wm/352.pdf>

[37] Pagina oficial del PhD Ali Kashani. - <http://ali.kashani.ca/active-passive-fusion>

[38] Fusion of Time-of-Flight Depth and Stereo for High Accuracy Depth Maps - Jiejie Zhu, Liang Wang, Ruigang Yang, James Davis.

[39] Fusion of Active and Passive Sensors for Fast 3D Capture - Qingxiong Yang, Kar-Han Tan, Bruce Culbertson y John Apostolopoulos.

[40] Statistical method for sub-pixel interpolation function estimation - I. Haller, C. Pantilie, T. Mariña and S. Nedevschi.

[41] http://fisica.udea.edu.co/~lab-gicm/Labradorio_Fisica_1_2012/2012_Cuantificacion%20de%20errores.pdf

ANEXO A

MANUAL DE USUARIO

MANUAL DE USUARIO

Se busca que este manual sea una guía para la utilización de las librerías *OpenCV* y *Point Cloud Library*, para la configuración que se debe tener dentro de *Microsoft Visual Studio* para la correcta ejecución de este proyecto.

Instalación de Hardware

Las cámaras utilizadas para este proyecto fueron colocadas sobre una estructura acrílica, de manera que estén lo más alineadas posibles entre ellas. Esto ayuda considerablemente, a disminuir los errores en los cálculos de los parámetros intrínsecos y extrínsecos durante el proceso de calibración.

Se utilizó cinta adhesiva de manera que las cámaras se encuentren fijas a la estructura acrílica, y se mantengan constantes los parámetros en el tiempo, sin embargo, se debe comprobar que el proceso de rectificación sea correcto con los parámetros usados en el momento, y se debe realizar el proceso de calibración nuevamente cada vez que se note que la calidad de los resultados han disminuido considerablemente.

Para evitar que la distancia focal cambie automáticamente, a medida que cambian los objetos en la escena se configuró las cámaras, de manera de que no tengan la características de autoenfoque por medio del software incluido por el fabricante de las cámaras.

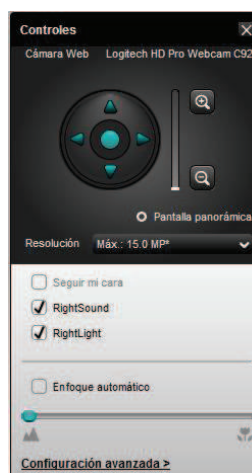


Figura A. 1: Ventana del Configuración de la Webcam Logitech C 920.

En caso de usar un hardware diferente al usado en este proyecto, se debe tener en consideración las menciones citadas anteriormente, para poder reproducir los resultados obtenidos en este proyecto.

Instalación de *software*

Para poder utilizar el *software* desarrollado, se debe instalar previamente las librerías utilizadas en el mismo. El uso de librerías dentro de *Microsoft visual Studio* requiere de una serie de procedimientos previos, que nos permita el uso de las mismas. Para esto se debe añadir todos los archivos y direcciones necesarios en la configuración de *Microsoft Visual Studio*.

Se describirá una serie de procesos que se deben seguir para tener éxito en el uso de librerías abiertas en C++, el procedimiento que se describirá funciona con cualquier librería para C++ pero para el caso de este proyecto nos limitamos al uso de *OpenCV* y *PCL*.

Lo primero que se debe realizar es descargar los instaladores de las respectivas páginas web las librerías *OpenCV* y *Point Cloud Library*. Para nuestro caso se utilizó la versión 2.4.3 de *OpenCV*, de preferencia se debe usar la versión más reciente, mientras para el caso de *Point Cloud Library* se utilizó la descarga *ALL-IN-ONE* en 32 bits.

Luego se agrega los archivos .dll (dynamic-link library) a las variables de entorno del sistema. Para agregar los archivos .dll se ingresa a mi PC, propiedades, configuraciones avanzadas del sistema donde aparecerá una ventana. En esta ventana se debe dar un *click* en variables de entorno, y posteriormente editar la variable "*path*", donde se añadirá las direcciones de las carpetas donde se encuentran los .dll.

Los archivos .dll para el caso de *OpenCV* se encuentran en subcarpetas de la carpeta *build* la misma que se encuentra en la dirección dada en el momento de instalación. La elección de las subcarpetas correctas depende del número de bits

en la que se desee realizar la compilación y de la versión de *Microsoft Visual Studio* en la que se realice la misma, para este caso se eligió 32 bits en *Microsoft Visual Studio 2010*.

Para el caso de *Point Cloud Library*, en el momento de la instalación se puede seleccionar las opciones “*Add PCL to the system PATH for all user*” o “*Add PCL to the system PATH for current user*”. En caso de elegir cualquiera de estas dos opciones ya no es necesario añadir los archivos en las variables de entorno de *Microsoft Windows*.



Figura A. 2: Ventana de Instalación de PCL.

En caso de elegir la opción “*Do not add PCL to the system PATH*”, se debe añadir los archivos .dll de la misma forma como se explicó para *OpenCV*. La carpeta con los archivos .dll se encuentra dentro de la carpeta bin que se encuentra dentro de la dirección de instalación de PCL. A continuación se muestra una copia de la pantalla donde se puede apreciar este proceso:

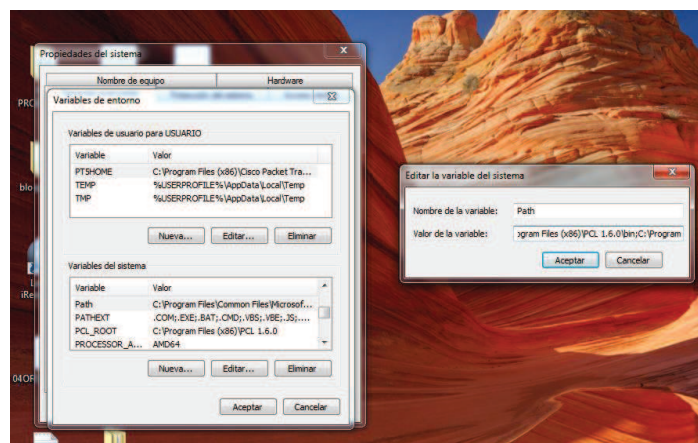


Figura A. 3 Ventana de propiedades del sistema en el sistema operativo Windows 7.

Funcionamiento de archivos ejecutables.

Dentro del cd de usuario se incluyen los archivos ejecutables. Para el funcionamiento de los mismos, es necesario que se encuentren dentro de la misma carpeta todos los documentos a los que llama durante se ejecución, y que se haya completado la instalación del *software*.

Para correr estos archivos no es necesaria la instalación de *Microsoft Visual Studio*, pero tampoco permiten ver el código fuente ni editar el contenido del mismo. En caso de que requerir algún cambio del programa, se debe seguir las instrucciones de configuración de *Microsoft Visual Studio*, y de generación de nuevos proyectos.

Configuración de Microsoft Visual Studio

Una vez que se ha realizado este proceso, se ingresa a *Microsoft Visual Studio* a realizar las posteriores configuraciones para el uso de las librerías. Una vez que se tenga un proyecto nuevo (tanto aplicación de WIN32 como MFC, que son los utilizados para el caso de este proyecto), se debe incluir tanto las direcciones de los archivos “*include*”, y de librerías adicionales como los nombres de las dependencias adicionales (las librerías usadas).

Los archivos “*include*”, conocidos como archivos de cabecera .h, contienen la información de definiciones de tipos de datos, prototipos de funciones y comandos del preprocesador de C. Estos archivos se incluyen al inicio de los programas C++ para indicar las definiciones que se van a usar en el mismo, y si no se incluyen estos archivos de cabecera, el compilador no reconocerá las comandos que estaban definidos en la misma.

Los archivos de bibliotecas adicionales .lib contienen los archivos objetos que no son más que un conjunto de archivos fuentes compilados, donde los archivos fuentes contienen las subrutinas que se van a usar dentro del programa C++ a través de la librería. [24]

Se puede agregar estas direcciones, y estos archivos dentro de las propiedades de *Microsoft Visual Studio*, o se puede crear una página de propiedades y guardarla en una carpeta dentro del disco duro. La ventaja de las páginas de propiedades, es que se puede usar la misma página en varios proyectos, sin tener que realizar todo el procedimiento nuevamente.

Las direcciones de los archivos “*include*” que se debe añadir, son las de carpetas con los nombres *include* que se encuentran dentro de subcarpetas en las direcciones de instalación de *OpenCV* y *PCL*. De igual forma las direcciones de las librerías que se debe añadir, son las de las subcarpetas con los nombre bin.

Las dependencias adicionales que se debe agregar, son los nombres de los archivos .lib que se encuentran dentro de las carpetas bin, y que se añadieron como direcciones de librerías.

Generación de nuevos proyectos.

En caso de que se requiera editar o ver el código fuente desde *Microsoft Visual Studio*, se incluye el código fuente de cada uno de los botones y del proyecto general del GUI. Una vez realizada la configuración de *Microsoft Visual Studio*, se puede usar el código fuente, para recrear el proyecto tipo MFC que genere el archivo ejecutable presentado en este proyecto.

De igual forma se incluye el código fuente de cada uno de los programas que se mencionan en este proyecto, y de igual forma se pueden recrear los mismos como Aplicaciones de Win32.

Una vez que se tenga el código fuente dentro del proyecto, se debe configurar *Microsoft Visual Studio* en modo *Release* y en Win32 antes de proceder a la compilación del programa, este se realiza desde la barra estándar.

Ahora se puede compilar el programa sin problemas dentro del entorno de *Microsoft Visual Studio*, con el botón de iniciar depuración dentro de la barra

estándar, o se puede presionar F5. Al realizar esto aparecerá la interfaz gráfica con todos sus botones y opciones listo para el uso con el usuario.

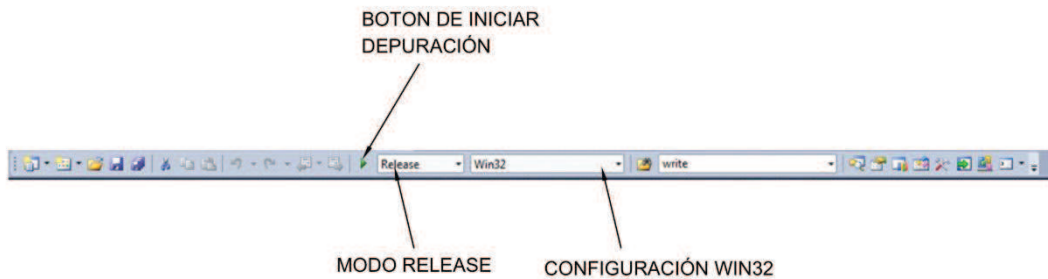


Figura A. 4 Barra de configuración de compilación en *Microsoft Visual Studio*.

Funcionamiento del GUI.

Para operar el GUI seguir el siguiente procedimiento:

1. Conectar las cámaras USB al computador en el orden correcto, enfocando hacia la escena que se desea reconstruir.
2. Ejecutar el archivo .exe del GUI.
3. Presionar el botón *START*, identificar en el mapa de disparidad si la escena a reconstruir es la adecuada, verificando la rectificación, caso contrario reubicar cámaras, o si es necesario volver a calibrar las cámaras.
4. Una vez que este el mapa de disparidad sea el adecuado presione el botón *CAPTURE*, para proceder a mirar la escena reconstruida en el visualizador 3D.
5. Para finalizar el proceso presionar el botón *STOP*, y cerrar el visualizador 3D.

ANEXO B

TABLAS DE ERRORES

TABLAS DE ERRORES

Se presentan las tablas de errores obtenidas en las pruebas de distancia, el procedimiento usado para realizar esta prueba se explica en 4.4.

Coordenadas		n medida	Dist. Real	Dist. Calculada	Error absoluto	% Error relativo	Promedio de errores relativos
x	y						
200	200	1	100	97,43	2,57	2,57	2,41
		2	100	97,33	2,67	2,67	
		3	100	98,01	1,99	1,99	
320	240	1	100	97,33	2,67	2,67	2,74
		2	100	96,91	3,09	3,09	
		3	100	97,54	2,46	2,46	
300	300	1	100	101,04	1,04	1,04	0,96
		2	100	100,81	0,81	0,81	
		3	100	101,04	1,04	1,04	
420	350	1	100	93,59	6,41	6,41	6,38
		2	100	93,59	6,41	6,41	
		3	100	93,69	6,31	6,31	
550	350	1	100	92,72	7,28	7,28	7,28
		2	100	92,72	7,28	7,28	
		3	100	92,72	7,28	7,28	

Tabla B. 1: Errores para la distancia de 100 cm

Coordenadas		n medida	Dist. Real	Dist. Calculada	Error absoluto	% Error relativo	Promedio de errores relativos
x	y						
200	150	1	150	149,71	0,29	0,19	0,12
		2	150	149,96	0,04	0,03	
		3	150	150,21	0,21	0,14	
320	240	1	150	148,72	1,28	0,85	1,34
		2	150	148,23	1,77	1,18	
		3	150	147,02	2,98	1,99	
300	300	1	150	142,15	7,85	5,23	5,38
		2	150	142,38	7,62	5,08	
		3	150	141,26	8,74	5,83	
300	500	1	150	140,38	9,62	6,41	5,67
		2	150	141,6	8,4	5,60	
		3	150	142,5	7,5	5,00	
100	350	1	150	155,93	5,93	3,95	4,86
		2	150	158,11	8,11	5,41	
		3	150	157,84	7,84	5,23	

Tabla B. 2: Errores para la distancia de 150 cm

Coordenadas		n medida	Dist. Real	Dist. Calculada	Error absoluto	% Error relativo	Promedio de errores relativos
x	y						
250	250	1	200	199,84	0,16	0,08	1,46
		2	200	195,5	4,5	2,25	
		3	200	195,93	4,07	2,04	
320	240	1	200	208,14	8,14	4,07	4,16
		2	200	208,66	8,66	4,33	
		3	200	208,14	8,14	4,07	
300	300	1	200	203,9	3,9	1,95	1,04
		2	200	202,08	2,08	1,04	
		3	200	200,28	0,28	0,14	
500	300	1	200	194,24	5,76	2,88	2,90
		2	200	194,14	5,86	2,93	
		3	200	194,24	5,76	2,88	
100	350	1	200	203,9	3,9	1,95	1,72
		2	200	203,44	3,44	1,72	
		3	200	202,99	2,99	1,50	

Tabla B. 3: Errores para la distancia de 200 cm

Coordenadas		n medida	Dist. Real	Dist. Calculada	Error absoluto	% Error relativo	Promedio de errores relativos
x	y						
250	250	1	250	251,05	1,05	0,42	0,70
		2	250	251,75	1,75	0,70	
		3	250	252,42	2,42	0,97	
320	240	1	250	247,6	2,4	0,96	0,86
		2	250	248,28	1,72	0,69	
		3	250	247,7	2,3	0,92	
300	300	1	250	255,81	5,81	2,32	1,62
		2	250	253,88	3,88	1,55	
		3	250	252,45	2,45	0,98	
300	420	1	250	244,91	5,09	2,04	1,66
		2	250	244,21	5,79	2,32	
		3	250	248,4	1,6	0,64	
450	200	1	250	256,77	6,77	2,71	2,38
		2	250	255,31	5,31	2,12	
		3	250	255,78	5,78	2,31	

Tabla B. 4: Errores para la distancia de 250 cm

Coordenadas		n medida	Dist. Real	Dist. Calculada	Error absoluto	% Error relativo	Promedio de errores relativos
x	y						
250	100	1	300	295,27	4,73	1,58	1,68
		2	300	295,27	4,73	1,58	
		3	300	294,34	5,66	1,89	
320	240	1	300	301,89	1,89	0,63	0,53
		2	300	300,93	0,93	0,31	
		3	300	301,93	1,93	0,64	
200	200	1	300	291,69	8,31	2,77	2,77
		2	300	291,69	8,31	2,77	
		3	300	291,69	8,31	2,77	
400	100	1	300	303,84	3,84	1,28	1,40
		2	300	304,82	4,82	1,61	
		3	300	303,9	3,9	1,30	
400	200	1	300	286,29	13,71	4,57	3,90
		2	300	288,04	11,96	3,99	
		3	300	290,56	9,44	3,15	

Tabla B. 5: Errores para la distancia de 300 cm

Coordenadas		n medida	Dist. Real	Dist. Calculada	Error absoluto	% Error relativo	Promedio de errores relativos
x	y						
220	120	1	350	362,51	12,51	3,57	3,45
		2	350	361,16	11,16	3,19	
		3	350	362,51	12,51	3,57	
320	240	1	350	363,88	13,88	3,97	3,97
		2	350	363,88	13,88	3,97	
		3	350	363,88	13,88	3,97	
340	150	1	350	381,07	31,07	8,88	8,87
		2	350	381,07	31,07	8,88	
		3	350	381,04	31,04	8,87	
400	100	1	350	366,63	16,63	4,75	4,76
		2	350	369,43	19,43	5,55	
		3	350	363,88	13,88	3,97	
200	350	1	350	348,17	1,83	0,52	0,87
		2	350	343,23	6,77	1,93	
		3	350	349,43	0,57	0,16	

Tabla B. 6: Errores para la distancia de 350 cm

Coordenadas		n medida	Dist. Real	Dist. Calculada	Error absoluto	% Error relativo	Promedio de errores relativos
x	y						
350	150	1	400	396,69	3,31	0,83	0,42
		2	400	399,96	0,04	0,01	
		3	400	398,26	1,74	0,44	
320	240	1	400	403,3	3,3	0,83	0,55
		2	400	398,32	1,68	0,42	
		3	400	401,62	1,62	0,41	
450	150	1	400	398,32	1,68	0,42	1,63
		2	400	391,87	8,13	2,03	
		3	400	390,27	9,73	2,43	
450	250	1	400	381,07	18,93	4,73	4,73
		2	400	382,57	17,43	4,36	
		3	400	379,57	20,43	5,11	
340	300	1	400	390,29	9,71	2,43	2,43
		2	400	390,29	9,71	2,43	
		3	400	390,29	9,71	2,43	

Tabla B. 7: Errores para la distancia de 400 cm

ANEXO C

HOJA DE DATOS DE CÁMARAS

HOJA DE DATOS

C920 Technical Specifications



Logitech HD Pro Webcam C920

General Product Information

Category	Webcam	
Software	Logitech Webcam Software Version 2.4	
OS Support	Win XP, Win Vista, Win 7 (x32/x64)	
System Requirements	Basic:	Core 2 Duo 2.4 Ghz, 2GB RAM
	HD Mode	i7 Quad core 2.6 Ghz, 4GB RAM
Connection Type	USB	
USB Protocol	USB 2.0	
USB VID_PID	082D	
Lens	Carl Zeiss®	
Lens & Sensor Type	Glass	
Focus Type	Auto 20 steps	
Optical Resolution	True:3MP	
	Software Enhanced:15MP	
Diagonal Field of View (FOV)	78°	
Focal Length	N /A	
Image Capture (4:3 SD)	N/A	
Image Capture (16:9 W)	2.0 MP, 3 MP*, 6 MP*, 15 MP*	
Video Capture (4:3 SD)	N/A	
Video Capture (16:9 W)	360p, 480p, 720p, 1080p	
Frame Rate (max)	1080p@30fps	
Right Light	RightLight 2	
Video Effects (VFX)	N/A	
Buttons	N/A	
Indicator Lights (LED)	Yes	
Privacy Shade	No	
Tripod Mounting Option	Yes	
Cable length	6 feet	

Product Dimensions

Product component	Width	Depth/Length	Height	Weight
Webcam	94 mm	24 mm	29 mm	162g

NOTE: Information is for reference only and may be subject to change.